

Contact Us

Criterion Software Ltd.

For general information about RenderWare Graphics e-mail info@csl.com.

Developer Relations

For information regarding Support please email devrels@csl.com.

Sales

For sales information contact: rw-sales@csl.com

Acknowledgements

Contributors	RenderWare Graphics Development and Documentation Teams
--------------	---

The information in this document is subject to change without notice and does not represent a commitment on the part of Criterion Software Ltd. The software described in this document is furnished under a license agreement or a non-disclosure agreement. The software may be used or copied only in accordance with the terms of the agreement. It is against the law to copy the software on any medium except as specifically allowed in the license or non-disclosure agreement. No part of this manual may be reproduced or transmitted in any form or by any means for any purpose without the express written permission of Criterion Software Ltd.

Copyright © 1993 - 2002 Criterion Software Ltd. All rights reserved.

Canon and RenderWare are registered trademarks of Canon Inc. Nintendo is a registered trademark and NINTENDO GAMECUBE a trademark of Nintendo Co., Ltd. Microsoft is a registered trademark and Xbox is a trademark of Microsoft Corporation. PlayStation is a registered trademark of Sony Computer Entertainment Inc. All other trademark mentioned herein are the property of their respective companies.

Foreword

About the User Guide

This is the RenderWare User Guide for release 3.3. We have reorganized the documentation into three volumes of general, platform independent material. Each of Xbox, GameCube and PlayStation 2 has a separate platform specific addendum, containing material which is only useful for that platform.

Volume I contains the core library and the world library, giving basic immediate mode and retained mode functionality.

Introduction

From the core library:

- Fundamental Types
- Initialization & Resource Management
- Plugin Creation and Usage
- The Camera, covering the basics of rendering
- Rasters, Images and Textures
- Immediate Mode
- Serialization
- Debugging & Error Handling

From the world library:

- World and Static Models
- Dynamic Models
- Lights

In Volume II elements of the animation systems, special effects and world management functionality are discussed

- Skinning
- Fundamental Types for Animation
- Hierarchical Animation
- Morph
- Delta Morphing
- Material Effects
- PTank
- B-splines and Bézier Patches
- Collision Detection
- Potentially Visible Sets (PVS)

Volume III covers the utility libraries, which offer a variety of useful functionality and an in-depth coverage of PowerPipe, the key to customizing RenderWare for ultimate performance and unique functionality for your application.

- 2D Graphics Toolkits
- The User Data Plugin, on exporting user data
- PowerPipe Overview
- Pipeline Nodes

As a new addition, we are offering documentation specific to three of the platforms we support, in this case PlayStation 2, Xbox and GameCube.

From PlayStation 2:
PS2All Overview

From Xbox:
Multi-texturing in MatFX
Multi-texturing on Xbox
Xbox state cache

From GameCube:
Multi-texturing in MatFX
Multi-texturing on GameCube

We realize that this User Guide does not cover every single feature in RenderWare Graphics, but we hope you will find it useful.

Please let us know what you think and feel free to offer any suggestions.

Regards,

The RenderWare Graphics Team

Table of Contents

Chapter 1 - Introduction.....	11
1.1 Welcome to RenderWare Graphics.....	12
1.1.1 What you Should Know	12
1.1.2 What is RenderWare Graphics?	13
1.1.3 What is RenderWare Graphics not ?	14
1.1.4 Design philosophy.....	15
1.2 The RenderWare Graphics SDK	17
1.2.1 Libraries and Header Files.....	17
1.2.2 Examples.....	18
1.2.3 Documentation	19
1.2.4 Exporter Source Code.....	20
1.2.5 Artists Tools	20
1.3 RenderWare Graphics Architecture	21
1.3.1 The Core Library, Plugins and Toolkits	21
1.3.2 PowerPipe.....	23
1.3.3 Namespaces.....	24
1.3.4 Just Graphics	24
1.3.5 Objects.....	25
1.4 Creating A Scene	27
1.4.1 Step-by-Step	27
1.4.2 Platform Abstraction.....	28
Part A - Core Library.....	29
Chapter 2 - Fundamental Types.....	31
2.1 Introduction	32
2.2 RenderWare Graphics & Objects.....	33
2.2.1 RenderWare Graphics Objects	33
2.2.2 Object Instantiation	33
2.2.3 Object Destruction & Reference Counters.....	34
2.3 The Boolean Type	36
2.4 Characters	37
2.5 Integer Types	38
2.6 Real Types	40
2.7 Vectors	41
2.7.1 Two Dimensional Vectors	41
2.7.2 Three Dimensional Vectors.....	42
2.8 Coordinate Systems	43
2.8.1 Right-handed Coordinates.....	43
2.8.2 Object Space.....	45
2.8.3 World Space.....	46
2.8.4 Camera Space	46
2.8.5 Device Space	46

2.9	Matrices.....	48
2.9.1	Matrix Mathematics in RenderWare Graphics.....	48
2.10	Frames.....	50
2.10.1	Hierarchical Models & RenderWare Graphics.....	51
2.10.2	Traversing Frame Hierarchies.....	52
2.10.3	Matrix Combination Flags in RenderWare Graphics.....	53
2.11	Bounding Boxes.....	55
2.12	Lines.....	56
2.13	Rectangles.....	57
2.14	Spheres.....	58
2.15	Colors.....	59

Chapter 3 - Initialization & Resource Management 61

3.1	Introduction.....	62
3.2	Basic Housekeeping.....	63
3.2.1	Initialization.....	63
3.2.2	Shutting down RenderWare Graphics.....	68
3.2.3	Changing Video Modes after Initialization.....	68
3.3	Memory Management.....	69
3.3.1	The OS-level Memory Interface.....	69
3.3.2	FreeLists.....	70
3.3.3	Resource Arenas.....	72
3.3.4	Locking and Unlocking data.....	73
3.4	Summary.....	74
3.4.1	Starting The Engine.....	74
3.4.2	Shutting Down The Engine.....	74
3.4.3	Memory Handling.....	75
3.4.4	Plugins.....	75

Chapter 4 - Plugin Creation & Usage 77

4.1	Introduction.....	78
4.2	Using Plugins.....	79
4.2.1	Attaching Plugins.....	79
4.3	Creating Your Own Plugins.....	81
4.3.1	Introduction.....	81
4.3.2	Anatomy of a Plugin.....	81
4.3.3	The "Plugin" Example.....	81
4.3.4	Using the Physics Plugin.....	84
4.4	Plugin Design.....	87
4.4.1	Introduction.....	87
4.4.2	Extension vs. Derivation.....	87
4.4.3	Deriving new objects.....	87
4.4.4	Plugins & C++.....	88

Chapter 5 - The Camera..... 91

5.1	Introduction.....	92
5.1.1	The Camera Example.....	92

5.2	The RenderWare Graphics Camera object.....	93
5.2.1	Camera Properties	93
5.2.2	The View Frustum	94
5.2.3	The View Window	95
5.2.4	The View Offset	97
5.2.5	Fog	98
5.3	The Camera View Matrix	101
5.4	Rasters & Cameras	102
5.5	Creating a Camera	103
5.5.1	Orienting and Positioning a Camera in Scene Space	103
5.6	Rendering to a Camera	105
5.7	Sub-Rasters	107
5.8	Other Features	108
5.9	World Plugin Extensions	109
5.9.1	Automatic & Manual Culling	109
5.9.2	Clump Cameras	109
5.9.3	Iterators	109
Chapter 6 - Rasters, Images and Textures		111
6.1	Introduction	112
6.2	Bitmaps & Textures	113
6.2.1	Bitmaps.....	113
6.2.2	Images.....	113
6.2.3	Rasters.....	113
6.2.4	Textures	114
6.3	The Image Object	115
6.3.1	Image Dimensions	115
6.3.2	Stride.....	115
6.3.3	Palettes	115
6.3.4	Gamma Correction	115
6.3.5	Creating Images	116
6.3.6	Example: Reading a BMP file	117
6.3.7	Reading the Image	118
6.3.8	Image Processing	119
6.3.9	Raster Conversion.....	121
6.3.10	Destroying Images.....	121
6.4	The Raster Object	122
6.4.1	Basic Properties.....	122
6.4.2	The Raster as a Display Device	125
6.4.3	Rendering Rasters	126
6.4.4	Accessing Rasters	127
6.4.5	Reading Rasters from disk	127
6.5	Textures & Rasters	128
6.5.1	Introducing Textures	128
6.5.2	Loading Textures	133
6.5.3	Texture Dictionaries	134
6.5.4	Using Texture Dictionaries	135

6.5.5	Textures & Binary Streams	136
Chapter 7 - Immediate Mode.....		137
7.1	Introduction	138
7.1.1	Properties & Render States	138
7.2	2D Immediate Mode.....	139
7.2.1	Basic Concepts	139
7.2.2	Initializing an Rwlm2DVertex object	141
7.2.3	Primitives	143
7.2.4	Triangle Winding Order.....	144
7.2.5	Primitives vs. Indexed Primitives	145
7.2.6	Example 1: Rendering A Line.	146
7.3	3D Immediate Mode.....	148
7.3.1	Preparation for Rendering.....	148
7.3.2	Rendering.....	150
7.3.3	Closing the pipeline.....	152
7.3.4	3D Immediate Mode & PowerPipe	152
7.3.5	Platform Specific Information	153
7.3.6	Camera-space and Z-buffer depth equations.....	154
7.3.7	Rt2D	155
7.4	Render States	156
7.4.1	Key Features.....	156
7.4.2	API	156
7.4.3	Blending.....	159
7.4.4	Sorting Alpha Primitives	160
7.4.5	160	
Chapter 8 - Serialization.....		161
8.1	Introduction	162
8.2	File I/O API	163
8.3	RenderWare Binary Streams.....	165
8.3.1	Binary Stream Structure.....	165
8.3.2	Serializing Objects	167
8.3.3	Explicit Streaming Functions	172
8.3.4	Stream Types.....	176
8.4	Summary.....	177
8.4.1	File I/O API.....	177
8.4.2	RenderWare Binary Streams	177
Chapter 9 - Debugging and Error Handling.....		179
9.1	RenderWare Graphics Errors.....	180
9.2	RenderWare Graphics Builds.....	181
9.3	The Debug Object	182
9.3.1	The Default Debug Stream Handler	182
9.3.2	Sending a Message to the Debug Stream.....	183
9.4	Tracing RenderWare Graphics Activity	184
9.5	Replacing The Stream Handler.....	186

9.5.1 Example	186
Part B - World Library.....	189
Chapter 10 - World and Static Models	191
10.1 Introduction	192
10.2 Scenes & Static Models	193
10.2.1 Scenes	193
10.2.2 RpWorld Object	193
10.2.3 RpWorldSector Object	194
10.3 World Offset	195
10.4 Iterator Functions	196
10.4.1 RpWorld Iterators	196
10.4.2 RpWorldSector Iterators	197
10.4.3 Collision Detection	197
10.5 Modeling Tools.....	198
10.5.1 Viewers	198
10.6 Creating Worlds	199
10.6.1 Creating Worlds from Foreign Data	199
10.6.2 What is Pre-lighting?	205
10.7 Rendering	207
10.7.1 How to Render Worlds.....	207
10.7.2 Instancing	207
10.7.3 Pre-instancing Static Geometry	208
10.8 Serialization	210
10.8.1 Writing	211
10.8.2 Reading.....	212
10.9 Destruction	213
Chapter 11 - Dynamic Models	215
11.1 Introduction	216
11.2 The World Plugin.....	217
11.2.1 The Geometry Object	217
11.2.2 The Atomic Object	217
11.2.3 The Clump Object	218
11.2.4 Clump Destruction	221
11.3 Creation of Dynamic Models	222
11.3.1 Model Creation Overview	222
11.4 Modeling Tools.....	223
11.4.1 Exporters.....	223
11.4.2 Viewers	224
11.4.3 Procedural Model Creation	224
11.4.4 Vertices & Triangles	225
11.4.5 Textures & Materials	227
11.4.6 Surface Properties & Geometry.....	229
11.4.7 Morph Targets	230
11.4.8 Pentagons & Hexagons	231

11.4.9	Bounding Spheres & Transformations	232
11.4.10	Atomics and Clumps	234
11.5	Objects in more detail	235
11.5.1	Reference Counting	235
11.5.2	Texture coordinates	235
11.5.3	Prelighting	236
11.5.4	Surface properties	236
11.5.5	Meshes.....	237
11.6	Atomics, Clumps & Transformations	239
11.6.1	Worlds	239
11.6.2	Cloning	239
11.6.3	Iterator functions	240
11.6.4	Sorting Geometry objects by Material.....	242
11.6.5	Animation.....	243
11.6.6	Skinned Models	244
11.7	Optimization.....	245
11.8	Rendering	247
11.8.1	How to Render Dynamic Objects.....	247
11.8.2	Instancing	248
11.8.3	Pre-instancing Dynamic Geometry	249
11.8.4	Converting Model Data to RenderWare Graphics	251
Chapter 12 - Lights.....		253
12.1	Introduction	254
12.2	Dynamic Lights.....	255
12.2.1	Dynamic Lights Representation	256
12.2.2	Creating a dynamic light.....	257
12.2.3	Clump Lights & Streaming	260
12.2.4	Platform-Specific Lighting Models	260
12.3	Static Lights	262
12.3.1	Creating Static Lights.....	262
12.3.2	Static Lighting Techniques	263
12.4	Related Examples	265
12.5	Summary.....	268
12.5.1	Dynamic Lights	268
12.5.2	Static Lights.....	269
Index		271

Chapter 1

Introduction

1.1 Welcome to RenderWare Graphics

Welcome to the RenderWare Graphics User Guide!

RenderWare Graphics is a powerful 3D graphics library. This User Guide is aimed at helping newcomers to RenderWare Graphics become familiar with the product. This is in addition to the on-line help, and support that is offered with the purchase of any RenderWare Platform component.

RenderWare Graphics is the result of many years of development, which began in 1991. The power and flexibility of the product has been increased with every release. The RenderWare Graphics is a multi-platform Application Programming Interface (API). RenderWare Graphics is constantly being improved and updated to keep it at the cutting edge of 3D graphics.

To support the community of developers in the field, we offer our Fully Managed Support System. Accessible via a personalized web interface, this allows our technical support personnel to be contacted, the status of outstanding queries to be tracked, and our knowledge base to be searched and viewed, plus lots of other useful information. To find out more and register for the service, point your browser at <https://support.renderware.com/>.

If you are new to RenderWare Graphics, it is strongly suggested that you read this User Guide as you go along to acquaint yourself with its operation.

Similarly, if you've already been using RenderWare Graphics for a while, stick around! The User Guide is written by the people who created the library, so you might gain new insights, tricks and tips. Additionally it has been revised and expanded to cover all our latest new technology.

RenderWare Graphics is a module of Criterion Software's RenderWare Platform, the middleware solution offering tightly integrated game development framework. For further information on the other Components (including RenderWare Audio and RenderWare Physics) please contact www.renderware.com or your account manager.



1.1.1 What you Should Know

This User Guide makes some assumptions about your level of proficiency with real-time 3D graphics programming:

This Guide is not aimed at complete newcomers to the field of computer graphics. If you are new to all this, check out the Recommended Reading appendix for links to online articles and books you can use as a starting point.

Secondly, this User Guide will not go into detail about the mathematics upon which most 3D graphics programming is founded. The whole point of graphics libraries such as RenderWare Graphics is to do the math for you.

That said, some college-level math is required to understand some concepts. If your knowledge of the principles and practices of matrix and vector manipulation is limited, the Recommended Reading appendix contains pointers to relevant texts that can help you with this area.

This User Guide assumes you are an experienced programmer with a thorough knowledge of the C (or C++) programming language. You should also be comfortable with the concepts behind object-oriented programming.

The User Guide is platform-neutral so it will not cover any specific development environment. Platform-specific variations and optimization notes will be provided in an appendix.

1.1.2 What is RenderWare Graphics?

A graphics library

RenderWare Graphics is a 2D and 3D graphics API. It is used by programmers to create real-time 3D graphics applications, such as computer games and simulations.

Multi-platform

RenderWare Graphics is multi-platform. This means that the same function calls are provided to programmers on many different computers.

RenderWare Graphics is already available for: Microsoft Windows (Direct3D), Microsoft Windows (OpenGL), Apple MacOS (OpenGL), Sony PlayStation 2, Microsoft Xbox and NINTENDO GAMECUBE.

Customizable

RenderWare Graphics has a component-based approach to its architecture based around a small-footprint, thin-layer core library, supplemented by a number of Plugins and Toolkits .

Plugins are the key to RenderWare Graphics' power: they can extend existing objects and add new objects of their own that can also be further extended.

Even the Retained Mode API is a Plugin, which gives RenderWare Graphics the unique feature of being the only 3D graphics library that can support any number of Retained Mode APIs.

Further, this Plugin mechanism is fully exposed. You can write your own Plugins, extending and adding objects, for your own requirements; we even encourage you to do so as we do not claim to have thought of everything!

Compatible with many third-party tools and middleware

RenderWare Graphics is a module of Criterion Software's RenderWare Platform, the middleware solution offering a tightly integrated game development framework and including graphics, audio and physics, with other Components planned for availability in the future.

Additionally, since we provide a number of tools, including dedicated RenderWare Graphics data exporters for the major 3D modeling packages, RenderWare Graphics will co-operate with a variety of middleware components, both from Criterion Software, and from third parties. These include physics, sound and network products.

1.1.3 What is RenderWare Graphics not ?

A Magic Bullet

Perhaps the most important point to note is that some programming is required to get anything of use out of RenderWare Graphics. You will not be able to simply click a button labeled "A Miracle Occurs" and get an instant game.

Optimized for any particular genre

RenderWare Graphics' component-based architecture makes it suitable for use for most consumer-level real-time 3D graphics applications. If you feel the Retained Mode Plugin we provide is not ideal for your purposes, there is nothing to stop you replacing it with your own. We have tried hard to make RenderWare Graphics genre-neutral. Game developers may need to extend RenderWare Graphics to provide the functionality that is required for a particular genre. However, RenderWare Graphics does provide objects and algorithms that make adding such features relatively straightforward.

In fact, you're quite welcome to write your own Plugins for release to other RenderWare Graphics licensees!

A Closed Architecture

Wherever feasible, we have attempted to create a library that is as open and accessible as possible to third parties. The source code to RenderWare Graphics itself is even available as an optional extra.

Thread-safe

RenderWare Graphics has not been written to make use of multi-threading techniques. However, it is possible to use RenderWare Graphics in a multi-threaded application on the condition that all RenderWare Graphics calls made by the application are made from the same thread.

A combined graphics, physics, AI, audio and networking API

RenderWare Graphics is a component of the complete RenderWare Platform middleware suite. This suite includes complementary components covering audio and physics that are available separately from Criterion Software Ltd.

RenderVision

RenderWare Graphics is not a point-and-click prototyping tool. For most tasks, some programming is required. RenderVision, written using the RenderWare Graphics API, is a point-and-click prototyping tool, integrated with 3ds max. RenderVision is a separate product sold by Criterion Software.

1.1.4 Design philosophy

The design brief for RenderWare Graphics was to create a 3D graphics library that would never be obtrusive. We have tried hard to make sure the library is the most powerful multi-platform 3D library available.

Platform Independent Development

RenderWare Graphics has been designed from the ground up to let you get the most out of all its supported platforms – with no compromises. APIs are provided which expose low-level features and optimization opportunities to the developer, so the best performance can be obtained from your projects.

The price for this is a little extra work during the porting process: each platform has different hardware, and different advantages and disadvantages. RenderWare Graphics gives you the freedom to choose how far you go down the optimization route:

Need to create a product quickly for more than one platform? No problem: use the common API features – the facilities provided as standard across all platforms – and treat it as an ordinary cross-platform library.

Alternatively, writing custom PowerPipe nodes and plugins enables RenderWare Graphics to be fine-tuned to specialized performance requirements. The purchase of a source code license gives the developer ultimate control over RenderWare Graphics.

RenderWare Graphics gives you the freedom to take either route, or indeed any path between these two extremes.

Such flexibility does come at a price: where platform-specific features of RenderWare Graphics are chosen, the code for each target platform will need to be changed when porting.

C vs. C++

One of the most common questions asked about RenderWare Graphics is the choice of programming language: C.

There are two reasons for choosing to write RenderWare Graphics in the C programming language. The first is that there is no standard for C++ libraries; RenderWare Graphics would have to be shipped as a separate set of libraries for each supported compiler, as well as each platform. Clearly, this would complicate product support.

Secondly, while C++ has many, many great features, most new platforms – particularly consoles – don't get a stable, mature C++ compiler until long after they get a good C compiler. In order for a new platform to be worked on as soon as possible, the fastest way to achieve this is to use a mixture of highly optimized C and assembly language.

That said, it is quite possible to mix C and C++. The Plugin mechanism lets you add space for 'this' pointers to RenderWare Graphics objects with minimal fuss, and all RenderWare Graphics' header files have the requisite "extern 'C'" directives to allow the two languages to mix seamlessly.

Several of our licensees have also produced their own C++ wrapper classes to encapsulate RenderWare Graphics in order to develop in a 'pure' C++ environment.

1.2 The RenderWare Graphics SDK

1.2.1 Libraries and Header Files

RenderWare Graphics is supplied as a number of libraries on the Software Development Kit (SDK). Applications will need to link against these libraries and `#include` the associated header files.

All RenderWare Graphics libraries are static. DLLs have not been provided, as there is no standard support for these on most platforms. This also avoids difficulties thrown up by having different versions of a DLL installed on a platform like the PC.

Each platform is provided with its own headers and libraries. In addition, the SDK contains null libraries, which are used by exporters and other tools. These are provided with all platforms. The null libraries contain all the PC functionality, but do not perform any rendering.

Null Libraries

On Xbox, GameCube and PlayStation 2 null and nullplatform libraries are also built. For example, PlayStation 2 RenderWare Graphics SDK is supplied with null and nullsky libraries. These PC libraries are required for certain tools that process platform specific data. They can be used for the generation of texture dictionaries.

It should be noted that nullplatform libraries can not create pre-instanced and geometry data.

Debug, Metrics and Release Libraries

For each platform, separate debug, metrics and release builds of the RenderWare Graphics libraries are also provided. The debug and metrics libraries live in separate folders inside the release build folder. `RWDEBUG` and `RWMETRICS` preprocessor symbols must be used to indicate which build is being used.

It is crucial that you do not mix symbols and libraries between these builds. To illustrate, some API calls are implemented in release builds as macros. In debug builds these calls are really implemented as functions. This can mean that if you define the `RWDEBUG` preprocessor symbol, but link with the release build of the library you will get numerous "undefined symbol" link errors.

Compilers supported

Please see the appropriate platform specific top-level readme files supplied with the RenderWare Graphics SDK for information on the compilers supported by RenderWare Graphics.

SN Systems, Visual Studio IDE integration and Project Files

The RenderWare Graphics SDK provides full support for the SN Systems Visual Studio Integration features.

Project Build Target settings are included for all platforms supported via Visual Studio. Developers should ensure the correct Project Build Target is selected.

1.2.2 Examples

The SDK contains nearly 50 source code examples. Examples are small and designed to illustrate a particular technique or feature of the RenderWare Graphics API. They are intended as a means of education: we encourage you to look through the source code and play with them.

Tools & Viewers

Also included are tools and viewers for you to use during development. Of particular note is the new RenderWare Visualizer viewer, which allows you to easily view RenderWare Graphics artwork on any target hardware.

The existing viewers 'wrldview', and 'clmpview' are still included in the SDK. 'wrldview' displays static models created for the Retained Mode API, and 'clmpview' displays dynamic models.

RenderWare Graphics uses a single file format, using chunk-IDs, which is capable of storing any one or more RenderWare Graphics objects. The RenderWare Graphics Binary Stream format can be viewed using the 'strview' applet supplied with the SDK.

A Microsoft Visual Studio 6 AppWizard is also supplied with RenderWare Graphics that can be used to generate an MFC framework-based clump or world viewer. This AppWizard can be incorporated into Microsoft Visual C++ and used in the same manner as the standard Microsoft AppWizards. Please see the accompanying documentation describing how to build a RenderWare Graphics viewer.

Building the Examples

The examples that are shipped on the SDK should not need compiling to run them. If you modify them, you will want to re-build the executables.

The Project files provided with the tools and examples are not necessarily set for your particular platform so make sure you select the correct build configuration prior to compilation. The two screen shots below show the Integrated Development Environments (IDEs) and the build targets that they offer. In Visual Studio, you will need to select the correct Project Configuration; in CodeWarrior, you must select the correct Target. Further, there are separate release, debug, and metrics builds. Unless you have good reason not to, it is suggested that you select:

Win32 D3D8 Release project in Visual Studio for the PC

Xbox Release in Visual Studio for Xbox.

PS2 Release in CodeWarrior for the PS2

GCN Release in CodeWarrior for GameCube

Visual Studio

CodeWarrior

1.2.3 Documentation

Documentation is provided in both online and PDF formats. (PDF format is intended for printing only and is not hyperlinked.)

The following documentation is provided in the SDK:

This User Guide

API Reference

Tutorials (PC only)

White Papers

Artist Guides

Tools and Viewers documentation

Examples document listing all examples with brief explanation

readme_xxx.pdf

All the examples and tools have a related readme.txt file. It is highly recommended that you read these for information about any last-minute changes or features.

There is also the top-level SDK readme_xx.pdf file, where xxx is the platform name, which lists last-minute changes, fixes and known issues and can be found in the root of the SDK.

1.2.4 Exporter Source Code

In RenderWare Graphics 3.3 we have removed the source for the Exporters.

With the introduction of the user data feature, rather than changing the RenderWare Graphics exporter source, you can now attach arbitrary user data to any exported object to achieve your custom results.

1.2.5 Artists Tools

The RenderWare Graphics SDK installer on the CD can be used to install the artists' tools. These tools are exporter plugins for exporting 3D model data from packages such as 3ds max and Maya.

When installed, artists will find:

- The modeling package's RenderWare Graphics exporter plugin(s)

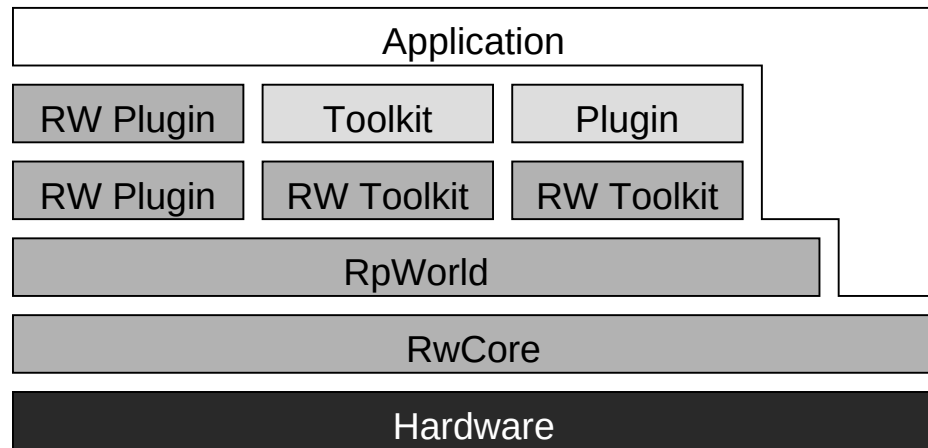
- Sample artwork demonstrating optimal modeling techniques

- Documentation covering how to create and export models successfully for RenderWare Graphics.

It is strongly recommended that programmers also read the documentation supplied with the artists tools. The installer adds links to these docs from the start menu.

1.3 RenderWare Graphics Architecture

The diagram below shows how the RenderWare Graphics library fits into a typical application.



Abstracting the underlying hardware is the RenderWare Graphics Core Library. Above this is the world plugin. This is the largest and most widely used of the optional RenderWare Graphics modules. Various plugins provided with the SDK are shown adding functionality. Also shown are toolkits, both supplied with the SDK, and non-RenderWare Graphics Toolkits supplied by third parties. A third party plugin is also shown. Above these components is the application which uses the services (functions) exposed by the various components.

The gray boxes above that labeled `RwCore` are optional. Unlike monolithic 3D graphics libraries, most of the higher-level features can be omitted. Although it's not explicitly stated in the diagram, `RpWorld`, which provides the Retained Mode API, is itself just a Plugin.

1.3.1 The Core Library, Plugins and Toolkits

The RenderWare Graphics components can be broken down:

1. Core Library
2. Plugins
3. Toolkits

The first is the Core Library. The Core Library must always be linked into your application as it provides the glue that joins all the components together, as well as basic rendering functionality.

Plugins

These are the keys to RenderWare Graphics extensibility. Plugins can extend existing objects in both the Core Library and other Plugins – and add their own objects. This feature is what differentiates them from ordinary libraries. RenderWare Graphics' high-level APIs are all implemented as Plugins.

Supplied Plugins

The following Plugins are supplied as standard with all releases of the SDK:

PLUGIN	DESCRIPTION
RpCollision	Collision-detection extensions
RpDMorph	Delta morphing and delta animation extensions
RpHAnim	Hierarchical animation plugin
RpLODAtomic	Level Of Detail extensions for RpWorld's "RpAtomic" object
RpLtMap	Render geometry using detailed static lighting information from lightmap textures.
RpMatFX	Multi-pass special effects, such as environmental mapping, bump mapping, 2-pass
RpMipmapKL	Texture mipmap "K" and "L" value extensions
RpMorph	Morph-target animation extensions
RpPatch	Bézier patch rendering engine
RpPrtStd	Particle animation plugin
RpPTank	Creation, management and rendering of user customizable particles
RpPVS	Fast visibility culling extension for RpWorld, using Potentially Visible Sets
RpRandom	Platform-neutral random number generator
RpSkin	Skinned model rendering extensions with multiple bone weights per vertex
RpSpline	Spline manipulation extensions
RpStereo	Stereoscopic display extension
RpUserData	Provides functionality for storing user defined data with geometry
RpWorld	Provides RenderWare Graphics' Retained Mode API – specifically, the scene graph portion of it

Toolkits

A Toolkit is an ordinary library that just happens to make use of RenderWare Graphics features. A Toolkit usually provides conversion functions or other utilities.

Supplied Toolkits

The following Toolkits are supplied as standard with all releases of the SDK:

TOOLKIT	DESCRIPTION
Rt2d	Advanced 2D Graphics API utilizing underlying 3D graphics hardware
Rt2dAnim	Animation of 2D objects
RtBary	Mapping points between the barycentric space and Cartesian space
RtBezPat	Bézier patch generation utility library
RtBMP	Microsoft® Windows® Bitmap image format handling
RtCharset	A bitmapped character library
RtIntersection	Polygon and line intersection testing functions
RtLtMap	Generation of lightmap textures - used with RpLtMap
RtMipK	Texture mipmap "K" value calculation functions
RtPick	Object-picking functions
RtPITexd	Platform independent texture dictionary streaming
RtPNG	Portable Network Graphics image format handling
RtQuat	Quaternion manipulation functions
RtRAS	Sun® Raster image format handling
RtRay	Ray-casting functions used for picking
RtSlerp	Spherical Linear Interpolation functions
RtSplinePVS	Utility functions to allow PVS generation using spline paths
RtTIFF	Tag Image File Format image format handling
RtTile	Tiled rendering functions (used mainly for very high-resolution renderings)
RtWorld	Utility functions to be used in conjunction with RpWorld
RtWorldImport	Utilities for creating RpWorld objects from foreign data formats

1.3.2 PowerPipe

PowerPipe provides a means of overloading the rendering subsystem either wholly or piecemeal. You can replace or even create entirely new rendering pipeline nodes and clusters.

1.3.3 Namespaces

All the RenderWare Graphics functions and objects carry two-letter prefixes to prevent naming clashes with your own code. This has been used to provide the best compromise between readability and name length.

The prefixes depend on whether the object in question is part of the Core Library, part of a Plug-in, or part of a Toolkit. They are:

PREFIX	DESCRIPTION
'Rw'	Indicates a function situated in the RenderWare Graphics Core Library ("rwcore.h" / "rwcore.lib"). These functions are always available as long as the core RenderWare Graphics library is linked to your application. Examples: RwEngineStart() RwCameraCreate()
'Rp'	Indicates a function situated in a Plugin library (e.g. RpMorph.) In most cases, the name of the Plugin follows the prefix, but this guideline is sometimes ignored to keep function names sensible. The appropriate Plugin must be attached if you intend to use these functions and linked to the appropriate header and library files. Examples: RpMorphPluginAttach() RpPVSAAtomicVisible()
'Rt'	Indicates a Toolkit. Syntax is similar to that of Plugins, described above. Many Toolkits rely on one or more Plugins being attached, but Toolkits themselves do not need to be attached. Examples: RtSlerpCreate() RtTileRender()
'Rx'	Used by the PowerPipe API. Examples: RxHeapFree() RxPipelineExecute()
'Rs'	Source code to a simple (and very basic) platform abstraction layer used for all examples is provided. Major functions in this layer use an 'Rs' prefix.
Exceptions	These include constants, such as #define's and enum values. These generally use the same prefixes as above, but entirely in lower case, followed by an all-caps name, such as: rwRASTERTYPECAMERA

1.3.4 Just Graphics

This may seem obvious, but it is important to remember that RenderWare Graphics only provides multi-platform 3D graphics features. While some utilities are provided, such as an abstraction layer known as the 'Skeleton', this code is not officially supported.

Most developers will create their own abstraction layers and write suitable plugins for RenderWare Graphics to provide a consistent programming interface across their supported platforms.

1.3.5 Objects

As RenderWare Graphics is written in C and assembler, it makes object-oriented design that little bit harder to implement. C++ classes might need to be written to wrap the RenderWare Graphics API.

One important issue to consider is the definition of an object in RenderWare Graphics.

In C++, objects are an explicit part of the language's design. RenderWare Graphics 'objects' are either intrinsic, such as `int` and `char`, or an ordinary C struct. These usually have a noun for a name – e.g. `World`, `Clump`, and `Vector`. The methods or member functions associated with these objects are ordinary C functions that live outside these structures, but with names that begin with the same name as the 'object'.

For instance, a hypothetical object called `RwThing` might have methods with names like `RwThingGetProperty()`.

RenderWare Graphics objects have been designed to operate in much the same way as C++ objects. The main difference is that there are no member selection operators to separate object names from their associated methods.

Objects & Properties

Transparent vs. Opaque Objects

RenderWare Graphics developers sometimes make a simple object transparent . This means that you will find complete documentation of the internals of said object in the API Reference and (usually) no property-access functions. The point of this is to reduce unnecessary function calling overheads: you are free to directly change the individual object elements.

This reveals the element of trust enshrined with RenderWare Graphics' design: where a data structure is not explicitly documented it should be considered an opaque object. Usually, such objects will have no documentation for their individual members; so only their associated property access methods (using the traditional 'Get' and 'Set' convention) should be used.

The table below shows some examples of opaque objects and access to their members:

OBJECT	PROPERTY	'METHOD' NAME
RwCamera	ViewWindow	RwCameraGetViewWindow()
		RwCameraSetViewWindow()
RpAtomic	Geometry	RpAtomicGetGeometry()
		RpAtomicSetGeometry()

The object name always forms the root of the function name. This pattern is followed consistently throughout the RenderWare Graphics API.

Obviously, the various header files document all members of a particular struct . In general, these members should not be modified in your code, but instead use the functions that are supplied in the library.

Of course, C doesn't automatically pass the object instance to the functions, so you still have to do this explicitly. Usually, the first parameter of a function will be a pointer to the object.

1.4 Creating A Scene

1.4.1 Step-by-Step

To render scenes in RenderWare Graphics applications, a number of steps need to be performed:

1. Create assets in a modeling package.

This includes the background scenery and all the characters, props and other animated models that will populate that scenery. Textures may need creating in a paint package.

2. Export assets in the RenderWare Graphics format.

The RenderWare Graphics Retained Mode API supports two kinds of model: static models that live in World objects, and dynamic models that live in Atomic objects.

Backgrounds and other fixed scenery models are generally considered static; all other models are dynamic. To add dynamic scenery elements, you should use dynamic models and position them in the scene accordingly.

These objects are part of the high-level RpWorld Plugin that encapsulates our Retained Mode API. More on this powerful Plugin can be found in the two chapters: Worlds & Static Models and Dynamic Models .

3. Load assets into the RenderWare Graphics application.

RenderWare Graphics includes a multi-platform file serialization API – RwStream – that is used for this purpose.

4. Position them using frames .

Frames, described in the Fundamental Types chapter, are an essential feature of the RenderWare Graphics architecture. They are attached to objects so they can be positioned in world space . Frames also manage model hierarchies.

5. Create lights .

RenderWare Graphics supports a number of lighting models, as well as both static and dynamic lights, and the standard light model can be overridden.

6. Create a camera object and orient it.

RenderWare Graphics uses the standard virtual camera metaphor in the Retained Mode API. This API, and Cameras in general, can be found in the following chapters: Cameras ; Worlds & Static Models and Dynamic Models .

7. Take a picture.

This process involves the actual rendering of the scene.

8. Update the scene.

If you're producing real-time animation in 3D, you will need to update the models between renderings.

9. Repeat steps 7 and 8 until the user tells the application to quit.

This is the rendering process in short. Aside from some of the terminology specific to RenderWare Graphics, this is much the same as in any other 3D graphics API.

1.4.2 Platform Abstraction

In order to be able to write our examples and other sample code just once, without the need to rewrite it for each target platform, the hardware abstraction layer called the 'Skeleton' has been developed. Almost all the sample code supplied with the SDK will use this code as its foundation.

The Skeleton was developed for our sample code. It is not suitable for anything other than similarly simplistic test-beds and prototyping. It is absolutely, categorically not intended for use as the basis for a professional application. However, it does provide a convenient set of functions that can be used for rapid prototyping of games.

The Skeleton code is completely unsupported. The source code to it is provided to show you that (a) it can be done, and (b) so that all our tools and examples can be presented in a consistent way.

Put another way: the Skeleton is provided as-is and with no guarantee for any suitability or fitness for purpose. You use it entirely at your own risk.

File I/O

File handling is an important facet of RenderWare Graphics and for this reason we provide a file system that can be overloaded. The `RwOsGetFileInterface()` can be used to obtain a structure containing the pointers to the file operation functions. The file pointers in this structure can be replaced by pointers to your own, which makes it easier to divert file-handling to the DVD, host machine and possibly even over a TCP/IP link.

Part A

Core Library

Chapter 2

Fundamental Types

2.1 Introduction

This chapter covers many of the basic types that are exposed by RenderWare Graphics.

Some types are simple and transparent , meaning you can access them and their elements directly. Others may be opaque and you should use the API provided to manipulate them.

If your development is intended to run on multiple platforms then it is suggested that you use RenderWare Graphics' data types throughout your application. The Core Library implements a number of basic data types, such as `RwChar`, `RwUInt16` and so on, which means that you can rely on RenderWare Graphics to ensure that on the different platforms the sizes of these types are correct.

For example, you can rely on the `RwChar` to be the correct size on a particular platform to store characters as these are not guaranteed to be eight bits on all supported platforms.

2.2 RenderWare Graphics & Objects

The first chapter explained that RenderWare Graphics is designed along object-oriented principles. As a result, the concept of objects plays an important part in understanding how the API works.

Before covering the basic data types then, it is worth looking at the ramifications of this design in a little more detail.

2.2.1 RenderWare Graphics Objects

C is not an object-oriented language, so one question frequently asked of the development team is why use C? This question was answered in the previous chapter, but it leaves open the question of how an object-oriented design is implemented.

By design, the API looks rather like a C++ based one, without all the extra punctuation. By necessity, the API supports a plugin mechanism which is used to extend the "objects". This extension mechanism is managed programmatically rather than as an intrinsic feature of the programming language.

For example, a look at the RpWorld plugin's API will reveal a number of base objects: RpTexture , RpClump, RpWorld and so on. Each of these objects is actually defined as a standard C struct datatype with their "methods" – functions – defined as ordinary functions separate from the data structure.

2.2.2 Object Instantiation

This is usually a two-stage process. The first is to define a variable of the object's type. For example:

```
RpTexture myTexture;
```

It is rare for developers to create automatic instances; allocation on the heap is far more common. This example therefore defines myTexture as an object of type RpTexture .

However, C does not support a constructor mechanism in the way C++ does, so myTexture has no valid data in it.

To reduce bugs caused by referencing uninitialized objects, RenderWare Graphics' API usually provides some form of default object creation function. In the case of Texture objects, the function is RpTextureCreate() , which generates a new Texture from the given Raster object and returns a pointer to the Texture on success.

```
RpTexture myTexture;  
RpTexture * pmyTexture = RpTextureCreate();
```


Similar creator functions are provided for most RenderWare Graphics objects and it is advisable to use them where possible.

2.2.3 Object Destruction & Reference Counters

As in C++, RenderWare Graphics objects must be destroyed when you are finished with them. This is particularly important if you are working on platforms with limited memory available. However, care is needed to ensure that you do not destroy objects that are still being referenced elsewhere in your code...

Although the documentation for RenderWare Graphics talks about things like container objects, these just boil down to objects that contain lists of pointers to the objects they contain. The word pointer is emphasized here because it implies that an object can be referenced by more than one object.

For example, a Texture can be "contained" by multiple Materials (part of the World Plugin and a useful container object for Textures), but this just means these Materials would each contain a pointer to the same Texture.

So far, so obvious, but multiple references can be a problem when it comes to destroying objects. If a Texture is referenced by multiple objects, some method is needed to prevent it being destroyed before the other objects have finished with it.

To avoid this, RenderWare Graphics uses a fairly standard reference counting system.

For example, if a Texture is referenced by another object, the Texture's reference counter will be incremented using a call to the Texture's ...AddRef() method. (RpTextureAddRef()) When the Texture is removed from that object, its counter is decremented. So if the Texture is referenced by, say, five other objects, its counter will be equal to five.

When the Texture is no longer needed by an object, it should be destroyed by calling the object's ...Destroy() method. (For Textures, the full function name is RpTextureDestroy() .) This function will decrement the reference counter and, if it is zero, finally destroy the object referred to.

It is important to note that the reference counting system is not fully automatic. For example, you will need to call ...AddRef() and ...Destroy() methods directly if you add a reference to a RenderWare Graphics object to a structure of your own.

Destruction & Destruction Order

The order in which objects are destroyed is an important consideration.

RenderWare Graphics programming often involves the use of a number of container objects. A common bug can be brought about by deleting such containers before deleting the contained objects.

For example, take the Clump and Atomic objects. These are part of the scene graph API provided by the World Plugin (RpWorld). For now, it's important to know only that Clumps are container objects for Atomics.

A common cause of bugs is to destroy a Clump, then destroy each of its contained objects by referencing through the Clump . Obviously, the Clump no longer exists at this point, but many programmers assume that the pointer is still valid as no code has been executed to overwrite the data yet.

This is a bad assumption to make: some platforms, including Microsoft Windows, have background tasks running which can easily trigger the overwriting of such data. This is a common cause of intermittent bugs and crashes, so you should always destroy objects in the correct order.

2.3 The Boolean Type

RenderWare Graphics supports one Boolean type:

TYPE	DESCRIPTION	RANGE	SIZE
RwBool	A standard Boolean type with the usual two states	FALSE, TRUE	32 bits

2.4 Characters

TYPE	DESCRIPTION	RANGE	SIZE
RwChar	Character type, in either ANSI or Unicode format		8 bits (ANSI char) or 16 bits (Unicode)

RwChar is intended solely for storing individual characters and character strings (usually 8-bit for ANSI libraries and 16-bit for Unicode libraries).

You should never use `RwChar *` as pointers to memory as it is wrong to assume it will be equivalent to the C Language "char" type.

Use `RwInt8 *` or `RwUInt8 *` instead.

2.5 Integer Types

RenderWare Graphics is intended to run on a number of platforms. To ensure consistent behavior, new types are defined to replace the base C Language ones. The RenderWare Graphics replacements are designed to behave as consistently as possible over all supported platforms.

RenderWare Graphics defines a number of integer types for specific bit widths, which are shown in the table on the following page.

These data types are designed to behave identically across all supported platforms. It therefore makes sense to use these instead of the standard C data types in your own applications.

Integer types:

TYPE	DESCRIPTION	RANGE
RwInt8	signed byte (8 bits)	-128 to +127
RwUInt8	unsigned byte (8 bits)	0 to 255
RwInt16	signed word (16 bits)	RwInt16MINVAL (-32768) to RwInt16MAXVAL (32767)
RwUInt16	unsigned word (16 bits)	RwUInt16MINVAL (0) to RwUInt16MAXVAL (65535)
RwInt32	signed long (32 bits)	RwInt32MINVAL (-2^{31}) to RwInt32MAXVAL ($2^{31}-1$)
RwUInt32	unsigned long (32 bits)	RwUInt32MINVAL (0) to RwUInt32MAXVAL ($2^{32}-1$)
RwInt64	64 bit, signed integers should only ever be used on platforms with native and compiler support for 64 bit data types. On other platforms, this data type is defined using a struct so no mathematical operations are available.	-2^{63} to $2^{63}-1$. No range defining macros are defined.
RwUInt64	64 bit, unsigned integers should only ever be used on platforms with native and compiler support for 64-bit data types. On other platforms, this data type is defined using a struct so no mathematical operations are available.	0 to $2^{64}-1$. No range defining macros are defined.
RwInt128	128 bit, signed integers should only ever be used on platforms with native and compiler support for 128 bit data types. On other platforms, this data type is defined using a struct so no mathematical operations are available.	-2^{127} to $2^{127}-1$. No range defining macros are defined.
RwUInt128	128 bit, unsigned integers should only ever be used on platforms with native and compiler support for 128 bit data types. On other platforms, this data type is defined using a struct so no mathematical operations are available.	0 to $2^{128}-1$. No range defining macros are defined.

2.6 Real Types

RenderWare Graphics supports the following real number types:

TYPE	DESCRIPTION	RANGE	SIZE
RwReal	Usually equivalent to the C Language's single-precision "float" type.	RwRealMINVAL to RwRealMAXVAL	32 bits
RwFixed	16 bit integer, 16 bit fractional fixed point value. Rarely used, as true floating-point math is usually faster in current hardware.	RWFIX_MIN to RWFIX_MAX	32 bits

A common mistake is to forget to provide a trailing "f" on floating point constants. Some platforms have no double precision hardware support. If the "f" is omitted from an expression compiled for these platforms the expression will be assumed to be double precision and it will be emulated in software. This can dramatically reduce performance. It is recommended that you get into the habit of either using type prefixes:

```
RwReal g = 9.8f ;  
RwReal f = m * g;
```

or make use of type-casting in expressions, which is more portable:

```
RwReal g = 9.8;  
RwReal f = m * (RwReal) g
```

2.7 Vectors

RenderWare Graphics supports two and three dimensional vector types and arithmetic.

These types should be considered opaque as they can sometimes map directly onto underlying hardware vector processing units.

2.7.1 Two Dimensional Vectors

The 2D Vector type is `RwV2d`. It contains x and y coordinates.

The following functions are provided to manipulate 2D Vectors:

FUNCTION	OPERATION
<code>RwV2dAssign()</code>	Assignment (Copy) from a source vector to a target vector
<code>RwV2dAdd()</code>	Addition
<code>RwV2dSub()</code>	Subtraction
<code>RwV2dLength()</code>	Length
<code>RwV2dNormalize()</code>	Return a unit normal vector calculated from the original vector
<code>RwV2dLineNormal()</code>	Find a Unit Normal line between two vectors
<code>RwV2dScale()</code>	Scale
<code>RwV2dDotProduct()</code>	Calculate Dot Product
<code>RwV2dPerp()</code>	Calculate a 2D vector perpendicular to the given 2D vector

2.7.2 Three Dimensional Vectors

The 3D Vector type is `RwV3d`. It contains x, y and z coordinates.

The table below shows the functions available to work with these vectors:

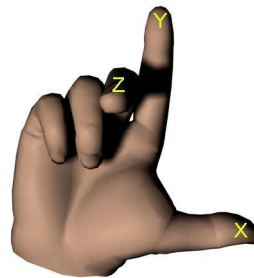
FUNCTION	OPERATION
<code>RwV3dAssign()</code>	Assignment (Copy) from a source vector to a target vector
<code>RwV3dAdd()</code>	Addition
<code>RwV3dSub()</code>	Subtraction
<code>RwV3dLength()</code>	Length
<code>RwV3dNormalize()</code>	Calculate unit normal vector from the original vector
<code>RwV3dScale()</code>	Scale
<code>RwV3dIncrementScaled()</code>	Multiplies the second 3D vector by the given scalar then adds the resulting vector to the first vector
<code>RwV3dDotProduct()</code>	Calculate Dot Product
<code>RwV3dCrossProduct()</code>	Calculate Cross Product
<code>RwV3dNegate()</code>	Negation
<code>RwV3dTransformPoints()</code>	Transform an array of points or vertices by the specified matrix
<code>RwV3dTransformVectors()</code>	Transform an array of vectors or normals by the specified matrix

2.8 Coordinate Systems

3D graphics programming requires understanding of a number of common coordinate systems, known as spaces. Some basic conventions need to be covered.

2.8.1 Right-handed Coordinates

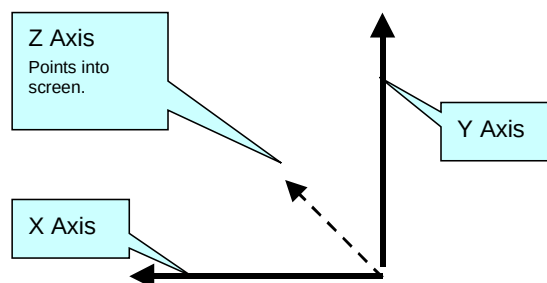
RenderWare Graphics uses an orthogonal right-handed coordinate system for its 3D spaces. A right-handed coordinate system is illustrated below:



A right-handed coordinate system

As you can see, it has the same orientation as that given by taking the first and second fingers of the right hand as the x and y axes respectively, and the thumb as the z axis. The fingers point in the positive direction along each of the axes. (In other words, to draw to the left of the origin, you need to use negative values for x.)

It is important to understand that as long as the axes are rotated together, the coordinate system is always right-handed. For example, RenderWare Graphics' virtual camera points into the screen along the positive z axis. If you hold your fingers as shown in the picture above, then rotate your hand about the y axis (forefinger) such that the z axis (middle finger) points away from you, you'll get the axis orientation used within RenderWare Graphics:



RenderWare Graphics' right-handed coordinate system

Axis Naming Conventions

The three axes are defined in RenderWare Graphics by three vectors. By convention, the axis names used are:

AXIS	VECTOR REPRESENTATION
X	"Right"
Y	"Up"
Z	"At"

Sometimes, vector names will be prefixed with the word "look", as in "look-at". This is most common when referring to the Camera object as this "looks" along the z axis, represented by the "at" vector, which some people call a front vector.

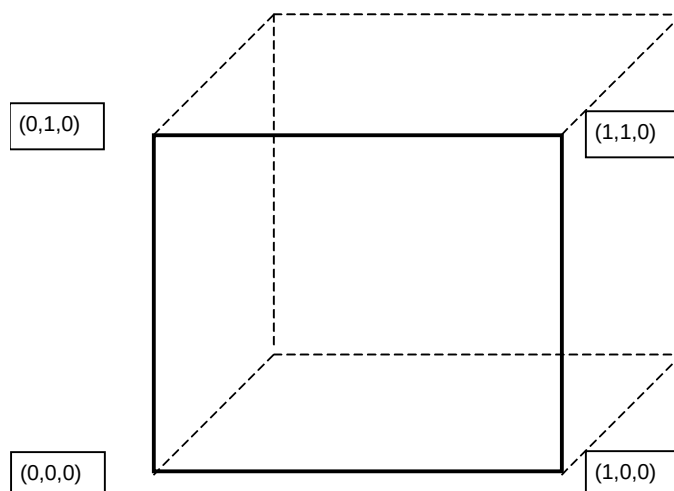
RenderWare Graphics' Camera object also exposes an oddity with this naming convention. Since the Camera faces into the scene and we're effectively standing behind it, the Camera object's right vector actually points left. Therefore, moving the Camera by incrementing its x coordinate results in it moving to the left – the opposite direction to that one would expect.

In fact, these names only make sense when you're observing the object in question by looking directly at it along its (positive) z axis.

2.8.2 Object Space

Dynamic models in RenderWare Graphics are defined in terms of Object Space. This means that the vertices that define the model are relative to an arbitrary origin.

For example, the front face of a cube of unit size could be defined thus:



The origin in this case is the lower-left corner of the front face. However, the origin could be anywhere – even the center of the cube. This could be achieved by subtracting (0.5, 0.5, 0.5) from each vertex.

The location of the origin can be important, as it is easier to align models if their origins are on a shared edge or a corner rather than at some arbitrary point in the middle or outside the models.

2.8.3 World Space

Dynamic models, defined in Object Space, need to have a frame of reference so that they can be positioned relative to other models and scene graph objects. This frame of reference is World Space .

Objects positioned relative to this system are said to reside in world space . This coordinate system is used, for example, to specify the position of cameras and lights. Geometry can be positioned in world space using transformations described later.

A bounding box defines the World Space limits. This is either generated by a modeling package exporter when exporting World data, or explicitly by the developer when calling `RpWorldCreate()` .

Frames

The RenderWare Graphics object that allows us to position objects in World Space is called the Frame (`RwFrame`). Many RenderWare Graphics objects require a Frame to be attached before they can be positioned within a World.

2.8.4 Camera Space

Cameras, like many RenderWare Graphics objects, require a Frame to define their position and orientation. However, there is another system called Camera Space which defines the Camera's viewing coordinate system. A normalized Camera Frustum defines Camera Space as follows:

for a parallel projection model the Camera Frustum space has side planes at $x = 0$, $x = 1$, $y = 0$ and $y = 1$

for a Perspective Projection model, it has side planes at $x = 0$, $x = z$, $y = 0$ and $y = z$

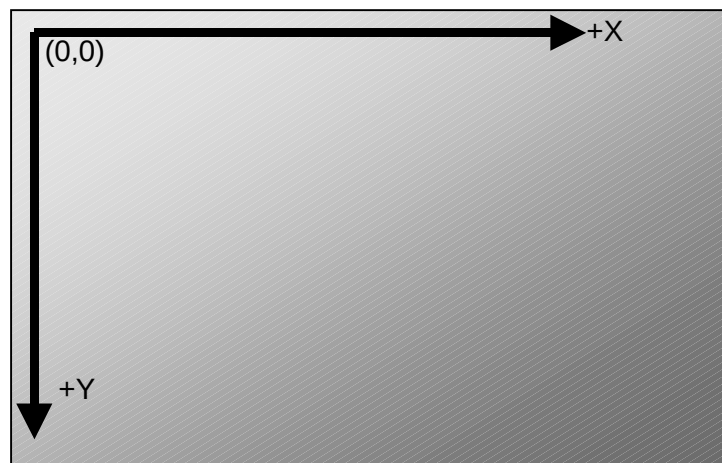
Camera Space is also a right-handed system with an origin at the Camera's position (given by the Frame's `pos` vector). The positive z axis of Camera Space is given by the view direction, which points in the direction of the Frame's `at` vector. The units of Camera Space z coordinates are the same as those for World Space.

RenderWare Graphics also recognizes a two-dimensional coordinate system.

2.8.5 Device Space

The device coordinate system defines a device space . Its units are those of the display (screen or window) to which the camera's image buffer is copied and as such, it has coordinates that only take discrete (or integer) values.

The origin of device space is located at the top-left of the display with the x-values running from left to right and the y-values running from top to bottom, as shown in the diagram:



Device space can also be considered to have a depth component that is used in the Z buffer.

2.9 Matrices

RenderWare Graphics defines a pseudo-4x4 homogeneous Matrix (`RwMatrix`) object to represent 3D transformations. The `Frame` object, see 2.10 below, makes heavy use of matrices. Matrices in RenderWare Graphics appear as shown below:

$$\begin{pmatrix} R_x & R_y & R_z & 0 \\ U_x & U_y & U_z & 0 \\ A_x & A_y & A_z & 0 \\ P_x & P_y & P_z & 1 \end{pmatrix}$$

The row vector (R_x, R_y, R_z) contains the components of the look-right vector, (U_x, U_y, U_z) are those of the look-up vector, (A_x, A_y, A_z) for the look-at vector and (P_x, P_y, P_z) are the components of the pos vector. The components of a matrix are available to the application programmer by using `RwMatrixGetRight()`, `RwMatrixGetUp()`, `RwMatrixGetAt()` and `RwMatrixGetPos()`.

2.9.1 Matrix Mathematics in RenderWare Graphics

Matrices are treated as quadruples of row vectors. Each row vector is a triple of real values. The vectors are the right, up and at vectors, which orient a Cartesian coordinate system, and the pos vector that positions this coordinate system with respect to a parent coordinate system. In conventional 4x4 matrix form, there is an implied last column of $(0,0,0,1)^T$. The equations for matrix multiplication can then be expanded as shown below.

C A B

$$\begin{pmatrix} R_x^A & R_y^A & R_z^A & 0 \\ U_x^A & U_y^A & U_z^A & 0 \\ A_x^A & A_y^A & A_z^A & 0 \\ P_x^A & P_y^A & P_z^A & 1 \end{pmatrix} \begin{pmatrix} R_x^B & R_y^B & R_z^B & 0 \\ U_x^B & U_y^B & U_z^B & 0 \\ A_x^B & A_y^B & A_z^B & 0 \\ P_x^B & P_y^B & P_z^B & 1 \end{pmatrix} = \begin{pmatrix} R_x^C & R_y^C & R_z^C & 0 \\ U_x^C & U_y^C & U_z^C & 0 \\ A_x^C & A_y^C & A_z^C & 0 \\ P_x^C & P_y^C & P_z^C & 1 \end{pmatrix}$$

Matrix multiplication is not commutative so, the order of the arguments passed to `RwMatrixMultiply()` is significant. Transformation of a position is performed mathematically in the following manner:

$v = u \cdot M$

$$\begin{pmatrix} v_x \\ v_y \\ v_z \\ 1 \end{pmatrix} = \begin{pmatrix} u_x & u_y & u_z & 1 \end{pmatrix} \begin{pmatrix} R_x & R_y & R_z & 0 \\ U_x & U_y & U_z & 0 \\ A_x & A_y & A_z & 0 \\ P_x & P_y & P_z & 1 \end{pmatrix}$$

This is the operation performed by `RwV3dTransformPoints()`. This function is used to transform position vectors. There is also an `RwV3dTransformVectors()` function, which transforms vectors (e.g. normals). In this case, the matrix is assumed to contain a zero position vector and so does not contribute to the output vectors.

Put another way: a direction vector is never translated, but only rotated.

Using `RwV3dTransformPoints()` with normals, or `RwV3dTransformVectors()` with vertices will give strange results.

The order of matrix transformations has an effect on the visible orientation of Atomics, as discussed (see 2.10.3 below).

2.10 Frames

RwFrame objects contain two matrices. These are the Local Transformation Matrix (or LTM), and the Modeling Matrix. These two matrices can be retrieved using `RwFrameGetLTM()` and `RwFrameGetMatrix()`, respectively.

When transformations are performed on a frame (see next paragraph), it is the Modeling Matrix that is affected. (The LTM describes the total transformation from Object Space to World Space.) Unless the Frame belongs to a hierarchy, the Modeling and Local Transformation Matrices are identical. Otherwise, the Modeling Matrix is relative to the Frame's parent.

Frames can be transformed using `RwFrameTranslate()`, `RwFrameRotate()`, `RwFrameScale()` and `RwFrameTransform()`. Rotation and scaling is accrued in the top-left 3x3 sub-matrix, while translation is accumulated in the bottom row.

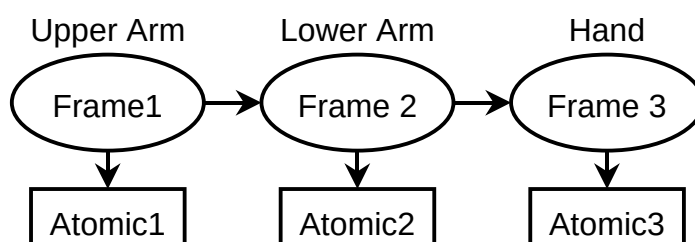
Given a transformation matrix, individual points and vectors (both of type `RwV3d`) can be transformed using `RwV3dTransformPoints()` and `RwV3dTransformVectors()`.

Hierarchical modeling is the process of building models that preserve the hierarchical structure of objects and allow the position and orientation of an object in the hierarchy to be specified relative to its parent.

2.10.1 Hierarchical Models & RenderWare Graphics

Hierarchical modeling explicitly models articulation, or joints, connecting objects. In RenderWare Graphics, these joints are represented by frames (RwFrame). These only define the hierarchy itself. The model data itself is sectioned, with each section stored in an atomic (RpAtomic). An atomic can be linked to a frame, with collections of atomics and frames thus forming a hierarchical model.

Consider, for example, the modeling of a robot arm consisting of an upper-arm, lower-arm and a hand represented by three atomics. We wish to model the arm so that movement of the upper-arm is transferred to the lower-arm and hand whilst movement of the lower-arm only affects the hand. Assume each atomic has its own frame (see API function `RpAtomicSetFrame()`). Then the required articulation can be achieved by attaching the lower-arm's frame as a child of the upper-arm's frame and, similarly, attaching the hand's frame as a child of the lower-arm's frame. The API function `RwFrameAddChild` is used to accomplish this.



The modeling matrix of each frame is now described relative to the Frame's parent Frame object and the corresponding LTM becomes the concatenation of all the modeling matrices up to the root Frame, in this case the Frame on the upper-arm's atomic.

For each case we have:

$$LTM_1 = MM_1$$

$$LTM_2 = MM_2 \ LTM_1$$

$$LTM_3 = MM_3 \ LTM_2$$

(where LTM = local transformation matrix and MM = modeling matrix.)

Only the root Frame's modeling and local transformation matrices are identical.

The object hierarchy is shown schematically above. Frame1 is the parent of Frame2 (therefore, Frame2 is the child of Frame1), while Frame2 is the parent of Frame3. Note that the arrangement of the Atomics is determined entirely by the frame hierarchy. If these Atomics were all added to a single Clump object, the Clump would impose no organization. Because a Clump is designed as a container for Atomics, it can be assumed that the Clump must order its Atomics but, as you can see, this is not how it works.

To model the robot hand in more detail by adding some fingers:

give the hand three fingers represented by three more Atomics.

build the hand hierarchy by attaching all the fingers' Frames as children of the hand's Frame (see diagram above).

All the fingers' Frames (frame4 , frame5 , frame6) are child frames of the hand's frame and siblings of each other. In terms of the LTMs we have, for example:

$$LTM_4 = MM_4 \quad LTM_3$$

This means the LTM of frame4 is the concatenation of the modeling matrices of Frames Frame1 , Frame2 , Frame3 and Frame4 – this is the same for the other fingers. Movement of the hand is now automatically transferred to all the fingers.

It is recommended, though it is by no means required, that the hierarchy is encapsulated within a Clump which has its own Frame – see `RpClumpSetFrame()` – acting as the root of the Frame hierarchy. For example, Frame1 would be attached to the Clump's frame (say, Frame0 ,) as a child of that Frame. Also, each of the Atomics must be added the Clump using `RpClumpAddAtomic()` for this organization to work correctly.

Frames and the Local Transformation Matrix

Frame objects contain a matrix called the Local Transformation Matrix , (usually abbreviated to "LTM "). This matrix is used when working with hierarchies of Frame objects.

Local Transformation Matrices are constructed by traversing the hierarchy from top to bottom. At each level in the hierarchy, the modeling matrix is pre-multiplied into the local transformation matrix from the level above to form the LTM for the current level. Because the LTM is used to post multiply vectors this means that the transformation stored in the lowest frame in the hierarchy affects the vertex first of all:

$$U \quad V \quad M_n \quad M_{n-1} \quad M_1 \quad M_0$$

In this equation, the Local Transformation Matrix at the root of the hierarchy is M_0 and that at the bottom of the hierarchy is M_n , where there are $n+1$ levels in the hierarchy.

2.10.2 Traversing Frame Hierarchies

There are two methods for traversing an object hierarchy depending on whether it is Frames or the objects hanging from them. It is assumed that a hierarchy of Frames and Atomics assembled into a Clump where the Clump's Frame acts as the root of the hierarchy.

All Frames in the hierarchy may be iterated over using the `RwFrameForAllChildren()` iterator. Starting at the root Frame, this function will apply a user-defined callback to all first generation child Frames. Repeating `RwFrameForAllChildren()` on each callback will then iterate over all second generation, i.e. grandchild, frames, and so on. Also, for each Frame encountered a call to `RwFrameForAllObjects()` applies a callback to each object attached to the Frame.

`RwFrameForAllChildren()` will only take you down one level in a hierarchy. You must use your callback function to call `RwFrameForAllChildren()` again to go down the hierarchy another level, so your callback should be designed for recursion.

If the Frame hierarchy is not important, a call to the `RpClumpForAllAtomics()` iterator will iterate over all the Atomics registered with a Clump. The supplied user-defined callback function will be applied to each Atomic in turn.

2.10.3 Matrix Combination Flags in RenderWare Graphics

Matrix and frame transformation functions in RenderWare Graphics (for example, `RwFrameTransform()` and `RwMatrixRotate()`) take a parameter, `combineOp`. This parameter is used to control the order in which the transform is applied to the matrix or Frame.

The available combine operators are: `replace`, `pre-concatenate`, and `post-concatenate`.

`rwCOMBINEREPLACE` assigns the new transformation to the matrix. The original contents of the matrix are entirely replaced by the new transform.

`RwMatrixTranslate(M, t, rwCOMBINEREPLACE)` builds a new matrix containing only a translation, and stores this matrix in `M`. Any rotation component in the matrix `M` is overwritten with a 3x3 identity sub-matrix. This is true for all matrix-transforming functions where the `rwCOMBINEREPLACE` flag is used.

`rwCOMBINEPRECONCAT` causes the transform matrix to be pre-concatenated onto the matrix. This has the effect of applying the transformation before the transformation already in the matrix. Put another way: this operator causes the transformation to be made in object space (or the coordinate space of the matrix).

`rwCOMBINEPOSTCONCAT` instructs RenderWare Graphics to post-multiply the transformation into the matrix. The new transformation will take effect after the transform already in the matrix. In other words, the transformation takes place in world space.

Example

Consider a call made to `RwFrameScale()` with a Frame `F`, and a vector `S` encoding the scale. The effect of changing the combination operator will be looked at. Firstly, however, it should be pointed out that the function alters the modeling matrix in the frame.

A RenderWare Graphics application should never modify the LTM matrix directly, since the library is responsible for computing this matrix from the Modeling Matrix and the LTMs of the Frame hierarchy.

In this example the modeling matrix, denoted M , is stored in Frame `F`.

Notionally, the `RwFrameScale()` function computes a scale transformation matrix, `S`. This will be a 4x4 zero matrix with a diagonal encoding the elements from the scale vector, and 1:

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

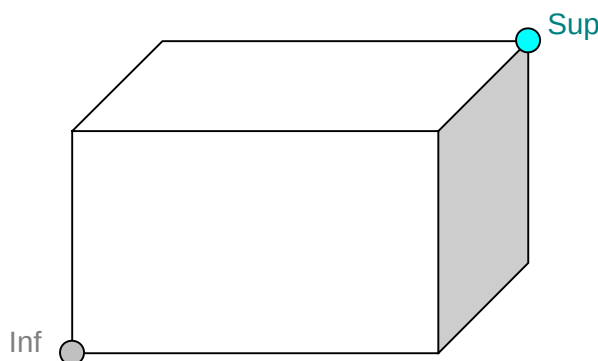
The table below illustrates the result of varying the combine operator on the matrix `M`.

OPERATOR	RESULTING MATRIX (M)
<code>rwCOMBINEREPLACE</code>	<code>S</code>
<code>rwCOMBINEPRECONCAT</code>	<code>SM</code>
<code>rwCOMBINEPOSTCONCAT</code>	<code>MS</code>

2.11 Bounding Boxes

The Bounding Box (`RwBBox`) object defines a bounding box using two points: `sup` (supremum) and `inf` (infimum). These two points define diametrically opposite corners of the box, such that for any axis, the value in the `inf` coordinate is never larger than the value in the `sup` coordinate.

The illustration below shows how this works in practise:



The resulting box therefore represents a 3D axis aligned cuboid volume.

Bounding boxes provide the basis for simple tests to determine whether specific coordinates in the coordinate space are bounded by the volume. The box may also be expanded to include another point within its volume, using the `RwBBoxAddPoint()` function.

Another test available is `RwBBoxContainsPoint()`, which can be used to check if a vertex is inside or outside a Bounding Box.

`RwBBoxAddPoint()` can also be used together with the `RwBBoxInitialize()` function. This takes a single vertex and initializes both the `inf` and `sup` elements to this point. `RwBBoxAddPoint()` can be called repeatedly to expand the Bounding Box to contain any arbitrary number of points. This technique is useful if the number of points involved is unknown.

If, on the other hand, you do have a fixed-length array of vertices, you can use the `RwBBoxCalculate()` function to perform a similar operation to the above. This function is usually the more efficient of the two systems.

2.12 Lines

RenderWare Graphics naturally supports a basic line type. This is called `RwLine` and it is defined by two `RwV3dobjects` denoting the start and end vertices. The `RwLine` object is used typically as an intersection primitive. Do not confuse the `RwLine` object with an immediate mode line, which is a renderable object.

2.13 Rectangles

Rectangles are defined by the `RwRect` type. This type takes a positioning vector to locate the rectangle, as well as two parameters defining its width and height. As with lines above, the `RwRect` object is not renderable. Rectangles are typically used to define sub-regions of the screen.

2.14 Spheres

RenderWare Graphics defines the `RwSphere` type. This contains both a `center` and a `radius` as elements to define the location and size of the sphere.

Spheres are heavily used during Retained Mode rendering to determine which models are within the Camera Frustum. Using spheres for the initial tests makes for rapid culling. Once this coarse-level checking has been performed, the remaining models can then be tested and clipped more accurately.

Note that this object is not renderable, and is used primarily for intersection testing.

2.15 Colors

RenderWare Graphics defines two color representation types: `RwRGBA` which is an integer-based primitive for describing colors, and `RwRGBAReal` which defines colors using real numbers rather than integers.

The `RwRGBA` format is the most used as it corresponds closely with most of our supported platforms. This structure contains four `RwUInt8` elements, one for each of the Red, Green, Blue and Alpha components.

Although the `RwRGBA` format is fixed, the bitmap facilities offered by RenderWare Graphics are not and may differ widely in color element format. It is therefore possible that conversion may be needed when extracting or modifying individual pixels in a bitmap and such processing should thus be kept to a minimum.

See the chapter on Rasters, Images and Textures for information on these items and their colors.

`RwRGBAReal` defines four `RwReal` elements for red , green , blue and alpha . It is intended to be used in cases where high orders of accuracy are needed while processing colors. Values range from 0.0f to 255.0f.

Chapter 3

Initialization & Resource Management

3.1 Introduction

RenderWare Graphics Core Library contains the base feature-set of the API. All other features are optional, provided in the form of either Plugins or Toolkits.

The Core Library's functionality is divided up into the following broad categories:

- Memory management

- OS-level file and memory management

- Plugin management

- Basic objects and intrinsic types, such as `RwInt32` , `RwBBox`, `RwMatrix` , `RwImage`, etc.

- 2D and 3D immediate modes

- PowerPipe extensible rendering pipeline

- Platform-Specific APIs (where applicable)

- Managing rendering time through the Lock and Unlock commands.

This chapter focuses on the memory management, plugin management and file management features of the core library.

3.2 Basic Housekeeping

At the heart of RenderWare Graphics is the `RwEngine` object. This engine performs a number of housekeeping functions, including:

- Initialization and shut-down procedures
- Selecting the display device and video mode
- Querying the underlying hardware about its capabilities
- Updating metrics information (when linked to the Metrics libraries)
- Managing Plugins

All RenderWare Graphics applications have to initialize the engine before calling any other RenderWare Graphics functions. We'll look at this process next.

3.2.1 Initialization

Getting the RenderWare Graphics engine up and running is a three-step process:

1. Initialization of memory management and default file system interface
2. Setting the video mode
3. Starting RenderWare Graphics

and there are three functions you need to call:

1. `RwEngineInit()`
2. `RwEngineOpen()`
3. `RwEngineStart()`

These functions must be called in the above order and you cannot start rendering until after the `RwEngineStart()` call.

Why the three steps?

RenderWare Graphics has to make allowances for the many platforms it supports. Splitting the start-up process into three stages lets us open up such features as file and memory management, as well enabling the plugin architecture to operate properly.

Let's look at these steps in terms of what they're for...

Step 1 - Initialization of Memory Management and Default File System Interface

This first step concerns the API function `RwEngineInit()` and involves setting up the memory and file-handling subsystems.

Why Does the Engine need to be Initialized First?

`RwEngineInit()` must always be the first RenderWare Graphics function your application calls as all other functions assume the memory management facilities have been set up. For example, the plugin mechanism needs to access these memory functions, so they need to be set up before any Plugins are attached.

Similarly, the display device your application will use may also need some memory allocated for it. So again, we have to make sure the memory subsystem is ready first.

The Memory Functions

By default, RenderWare Graphics uses the standard ANSI memory functions, as implemented by the target platform's compiler, so any replacement functions must have identical prototypes to their ANSI counterparts.

In theory, the application can replace the default RenderWare Graphics handlers with its own at any time. In practice the application should change the memory handler only at this initialization stage, rather than later.

Changing the memory and file handling subsystems within the main loop of your application is possible, but not recommended as it can make debugging much more difficult.

Pointers to the functions are stored in the `RwMemoryFunctions` structure returned by `RwOsGetMemoryInterface()`. If necessary, the entries in this structure can be overwritten directly with your own function pointers, but this is not recommended as the change takes place immediately—if memory has already been allocated with the original functions, then it is likely that memory will be "lost" to your application.

Your application should create a new `RwMemoryFunctions` structure with pointers to your own functions, then pass a pointer to the structure to `RwEngineInit()`.

The File Functions

The file system is implemented in a similar way to the memory system, with a structure containing pointers to default POSIX functions used by default. Again, any replacement functions will need to share the same prototypes as the functions they will be replacing.

A file functions structure filled with the default functions can be retrieved using `RwOsGetFileInterface()` . To replace these default function pointers with your own, overwrite the entries in the structure. Your functions will then be used when accessing files.

Step 2 - Setting the Video Mode

At this stage, you will need to:

- Open the RenderWare Graphics Engine

- Determine which graphics subsystem the application should be using, as some platforms can support more than one

- Determine the video display mode the application should use.

Video hardware varies greatly from platform to platform, so this procedure requires some additional steps of its own.

Opening the RenderWare Graphics Engine

The RenderWare Graphics Engine can be opened using `RwEngineOpen()` .

Choosing the Graphics Subsystem

The RenderWare Graphics API represents a graphics device as a graphics subsystem .

The application must first determine how many graphics subsystems are available. This requires a call to `RwEngineGetNumSubSystems()` .

With this information, the application iterates through the available subsystems, checking each one for suitability. This is achieved by calling `RwEngineGetSubSystemInfo()` and interrogating the structure returned.

The following code fragment illustrates this process by printing out the identifying names of all graphics subsystems on a particular platform:

```
RwInt32 numSubSystems, i;
RwSubSystemInfo sslInfo;

numSubSystems = RwEngineGetNumSubSystems();

for(i=0; i<numSubSystems; i++)
{
    RwSubSystemInfo sslInfo;

    if( RwEngineGetSubSystemInfo(&sslInfo, i) )
    {
        printf("SubSystem %d: %s\n", i, sslInfo.name );
    }
}
```


(The `RwSubSystemInfo` structure contains only the name element.)

Once the subsystem has been chosen, the application must inform RenderWare Graphics of its choice by calling `RwEngineSetSubSystem()` with the requisite index number.

Setting the Video Mode

With the graphics subsystem selected, the application must now set the video mode. The first step is to determine how many video modes are supported, and what these modes are.

To determine the number of video modes supported, the application should call `RwEngineGetNumVideoModes()`, then iterate through the available modes by calling `RwEngineGetVideoModeInfo()`.

The code fragment below iterates through the available video modes and prints the information for each one to `stdio`.

```
RwInt32 numVideoModes, j;
RwVideoMode vmodeInfo;

numVideoModes = RwEngineGetNumVideoModes();

for(j=0; j<numVideoModes; j++)
{
    if( RwEngineGetVideoModeInfo(&vmodeInfo, j) )
    {
        printf("Video Mode %d: %dx%dx%d %s\n", j,
            vmodeInfo.width, vmodeInfo.height, vmodeInfo.depth,
            (vmodeInfo.flags & rwVIDEOMODEEXCLUSIVE ?
"(EXCLUSIVE)" :
            "" ) );
    }
}
```

The `RwVideoMode` object contains the following elements:

`width` – the width, in pixels, of the video mode;

`height` – the height, in pixels, of the video mode;

`depth` – the number of bits per pixel supported by the video mode;

`flags` – contains one or more of the following flags:

`rwVIDEOMODEEXCLUSIVE` if set, the video mode is full-screen rather than windowed.

Windowed modes are only available on DirectX and OpenGL platforms.

`rwVideomodeInterlace` if set, the video mode is interlaced, with alternate fields being displayed each frame.

Interlaced video modes are more common on platforms that use televisions for their video output.

`rwVideomodeEffInterlace` if set, the video mode is interlaced, but the signal is processed to reduce or eliminate the flickering normally associated with interlaced video modes.

Interlaced video modes are more common on platforms that use televisions for their video output.

`rwVideomodeFSAA0` (PlayStation 2-specific). If set, this video mode uses full-screen anti-aliasing Mode 0.

`rwVideomodeFSAA1` (PlayStation 2-specific). If set, this video mode uses full-screen anti-aliasing Mode 1.

Owing to differences in hardware design, the flags shown above are not supported on all platforms. Please refer to the platform-specific documentation in the API Reference for information on any additional flags.

The video mode is set with a call to `RwEngineSetVideoMode()`, which requires the index number of the desired video mode.

Step 3 – Starting RenderWare Graphics

Plugins are one of the keys to RenderWare Graphics power. They usually extend or add objects and provide new functionality. The Retained Mode API, for example, is exposed by the `RpWorld` plugin.

RenderWare Graphics is written in C, not C++, so object extensions must be handled explicitly. One of the most important procedures is attaching a plugin. This process performs the following steps:

- Linking a plugin with any existing RenderWare Graphics objects;

- Adding any extensions to those objects;

- Informing the RenderWare Graphics engine about any additional memory the extended objects will need.

Attaching must take place prior to starting the RenderWare Graphics engine with `RwEngineStart()`.

3.2.2 Shutting down RenderWare Graphics

Each of the three start-up functions has a matching shutdown function, which also needs to be called in the following order when the application terminates:

1. `RwEngineStop()`
2. `RwEngineClose()`
3. `RwEngineTerm()`

Each of these functions needs to be called in turn to close down and exit a RenderWare Graphics application.

Applications running on consoles, such as the PlayStation 2 or Xbox, do not need to concern themselves with exiting cleanly as console users just switch off the machine. But developers targeting multi-purpose platforms like Windows and MacOS will need to use these functions.

A good reason for shutting down is to trigger debugging functions, which notify the developer of any memory leaks. As these are often tied to the memory handling functions set up by `RwEngineInit()` , it may be necessary to perform a complete shutdown of RenderWare Graphics to trigger the leakage tests.

3.2.3 Changing Video Modes after Initialization

Changing video modes after the initialization steps described so far is tricky. To do so, you will need to partially shut down the RenderWare Graphics Engine by calling both `RwEngineStop()` and `RwEngineClose()` , then set your new video mode by calling `RwEngineOpen()` and `RwEngineInit()` to restart the RenderWare Graphics Engine.

This procedure requires the deletion and re-initialization of many active RenderWare Graphics bitmap objects, as hardware issues may require your graphics data to be in a different format for some display modes.

3.3 Memory Management

RenderWare Graphics supports two memory management features, as well access to those of the underlying platform.

In this part of the chapter, we will take a look at these features, which are:

- The OS-level memory interface

- The Freelist memory management system

- The Resource Arena

3.3.1 The OS-level Memory Interface

As we've already seen, the RenderWare Graphics initialization function, `RwEngineInit()`, lets you set up the memory management subsystem used by the RenderWare Graphics API. This is a useful feature as you will be in a far better position to understand your application's memory usage patterns and can therefore replace the RenderWare Graphics memory manager with your own, more optimized one if you require.

To use these functions, your application should call the `RwOsGetMemoryInterface()` API function. This returns a pointer to an `RwMemoryFunctions` structure. The structure contains function pointers for OS-specific implementations of the ANSI C `malloc()`, `free()`, `realloc()` and `calloc()` functions.

RenderWare Graphics provides default implementations of the ANSI functions, which are named as follows:

- `RwMalloc()`

- `RwFree()`

- `RwRealloc()`

- `RwCalloc()`

These functions are mapped using macros. Where a specific platform's operating system provides its own, optimized forms of the ANSI functions, these will be substituted in the macros. In addition, debug versions of these functions will be substituted in the macros if a debug build is specified.

Your application can override these default functions by passing a structure populated with your own function pointers to `RwEngineInit()`. Your functions will obviously need to take the same parameters as the ANSI ones.

If you want to cut down on porting issues, it's worth using the `RwMemoryFunctions` structure to access the relevant memory allocation functions, rather than calling them directly. This also means your code can seamlessly switch to another memory-management system if you should decide to replace the default RenderWare Graphics functions with your own.

3.3.2 FreeLists

RenderWare Graphics provides two memory-management systems. The first of these is the `RwFreeList` object and is active by default.

A value of `rwENGINEINITNOFREELISTS` can be passed in the `flag` parameter for `RwEngineInit()`. This will tell RenderWare Graphics to use ANSI memory functions in place of the FreeList functions instead.

How FreeLists work

FreeLists are initialized with a block size – call it *s* – and a number – call it *n* – which specifies the minimum number of such blocks to allocate at a time. When you allocate your first object, the FreeList grabs enough memory to hold *n* blocks. You're free to allocate and free any of these blocks as often as you like.

If the FreeList is full and you try and allocate a new block, the FreeList grabs another (*n* * *s*) bytes of memory.

As you develop your application, you should monitor your usage of FreeLists to determine the best balance between speed and efficient memory usage.

Why FreeLists are useful

FreeList objects manage blocks of memory of the same size. Applications often spend most of their time creating and destroying groups of related objects. Grouping similar objects together under one FreeList object gives an economy of scale. Instead of allocating lots of small chunks of memory, the object allocates in bulk, grabbing another big chunk only if the previous chunk is completely filled.

As an example, take a game that needs to instantiate groups of missiles. Each missile structure requires the same amount of space as that for any of the other missiles, so it makes sense to avoid multiple `malloc()` calls and use a single FreeList object to handle the missiles in batches instead.

In code, our initialization for the missiles FreeList might look something like this...

```
RwBool
InitializeMissiles(void)
{
    /* First, set up an RwFreeList structure for our missiles... */
```

```

    /* "TMissile" is our missile structure (wrapped into a typedef).
    * We want to allocate them in batches of 64 at a time.
    * We need them aligned to 4-byte boundaries. */
    Globals.missilesList =      RwFreeListCreate    ( sizeof(TMissile),
(RwInt32)64, 4 );

    return (TRUE==Globals.missilesList);
}

```

Now, to use our FreeList mechanism, we can do something like this:

```

...

/* Need a new missile... */

TMissile *myMissile;

myMissile = (TMissile *)      RwFreeListAlloc    ( Globals.missilesList );

if (myMissile)
{
    /* Do something here with the missile. */
}

```

And, when we're done with all our missiles, we can destroy our FreeList storage:

```

void
DestroyMissilesList(void)
{
    if( Globals.missilesList)
    {
        RwFreeListDestroy    ( Globals.missilesList );
    }
}

```

You can see that the FreeList allocation and destroy functions are similar to `malloc()` and `free()`, with the important difference that they result in far fewer calls to those low-level functions.

While a small amount of memory is used as a stack to help manage the FreeList system, it is important to understand that `RwFreeLists` don't allocate any memory until you start using them. For our example above, the big block allocation would take place during the call to `RwFreeListAlloc()`.

FreeList usage within RenderWare Graphics

FreeLists are used extensively by RenderWare Graphics for its internal memory management, particularly for managing groups of textures, cameras, clumps and other objects. These lists are not accessible directly by the application.

3.3.3 Resource Arenas

The Resource Arena is a cache and only one is supported in an application.

When rendering geometry, RenderWare Graphics will create a platform-specific version of that geometry—a process known as *instancing*—in the Resource Arena. RenderWare Graphics can then use the cached instance of the geometry directly rather than re-generate the platform-specific data for each rendering cycle. This is faster than re-instancing the geometry data every cycle.

This system is most efficient if the model geometry changes infrequently—a common situation in many 3D graphics applications.

There are some important points to understand:

If you make your Resource Arena too small, your application will suffer from slowdown by a process known as *arena thrashing*. This occurs when objects that are being instanced require that another, previously instanced object, be kicked out to make room. This kicked out object will have to be re-instanced in the next rendering frame. This will result in another object being evicted from the cache, and so on.

Conversely, making the Resource Arena too big makes less than efficient use of your platform's memory. On platforms like the PC, this is not such a big issue, but when it comes to consoles, every byte counts.

The answer, as with FreeLists, is to spend some time optimizing the parameters you use when setting up your Resource Arena.

The Resource Arena API

The `RwResources` object represents the Resource Arena. This object represents a list of `RwResEntry` structures, each describing a block of memory in the Resource Arena. The API itself is centered on the `RwResources` object.

A Resource Arena is always present when using RenderWare Graphics. The size of the arena is set during `RwEngineInit()`, which is passed in the `resArenaSize` parameter. The arena size can be set to zero if it is not needed.

Also present in this particular API is the equivalent get function – `RwResourcesGetArenaSize()` – which returns the size of the Resource Arena. This can be used together with `RwResourcesGetArenaUsage()` to determine the optimum Arena size. This last should be called just before a call to `RwCameraShowRaster()` to obtain meaningful results.

`RwResourcesGetArenaUsage()` returns the memory used by instancing and can return a value greater than the maximum value you set. If it does so, it means that arena thrashing is taking place – geometry is being instanced, thrown away then re-instanced again because there isn't enough space to store all the instanced data needed for the frame. You should increase the maximum size of the Arena to prevent this or, if memory is tight, reduce the complexity of your geometry to keep the size of the instanced data down.

`RwResourcesSetArenaSize` does not actually reserve all the memory specified. Instead, the Arena Size value determines the maximum amount of memory the Resource Arena is allowed to use.

For example, if your application sets the size to 8 Mb, but only uses 4Mb, only 4Mb of memory will be allocated.

3.3.4 Locking and Unlocking data

The process of instancing display data into platform-dependent form generally takes a lot of processing time. The Lock and Unlock functions that are part of most plugins allow the developer to control this process.

The Lock function tells RenderWare Graphics that the developer wants to access and change some data, like a geometry, texture or palette, and have it re-instanced so that it can be rendered efficiently. The Unlock function allows re-instancing, which is generally slow (though for some data, like vertex positions and colors, re-instancing is postponed until just before rendering).

Many Lock functions have flags to indicate precisely which data is being altered and thus avoid recalculating unchanged data.

So the Lock functions should be used judiciously and be avoided when possible.

3.4 Summary

3.4.1 Starting The Engine

The initialization phase of your application needs to perform the following steps to start the RenderWare Graphics engine:

1. Call `RwEngineInit()` , passing an `RwMemoryFunctions` structure if required.

The `rwENGINEINITNOFREELISTS` flag should also be specified if the FreeList mechanism is not required, pass the `resArenaSize` parameter for the Resource Arena size;

2. Enumerate the available graphics subsystems and use `RwEngineSetSubSystem()` to select the subsystem required by the application;
3. Enumerate the available video modes and use `RwEngineSetVideoMode()` to select the video mode required by the application;
4. Call `RwEngineOpen()` ;
5. Attach any required plugins;
6. Call `RwEngineStart()` .

The RenderWare Graphics engine is now started and rendering can take place.

3.4.2 Shutting Down The Engine

When running on platforms that run multi-tasking operating systems such as Windows or MacOS, applications should exit gracefully. This is achieved by calling the following functions in the order shown:

1. `RwEngineStop()`
2. `RwEngineClose()`

(At this point, it is possible to change the graphics subsystem and video mode and restart the engine.)

3. `RwEngineTerm()`

3.4.3 Memory Handling

Default Memory Handlers

RenderWare Graphics provides two default, general-purpose memory-management mechanisms, only one of which is available at any one time:

FreeList memory management (default);

ANSI-standard memory management functions—`RwEngineInit()` must be called with the `rwENGINEINITNOFREELISTS` flag set;

You can replace the memory-management functions by creating an `RwMemoryFunctions` structure with pointers to your own functions, then passing this structure to `RwEngineInit()` .

The Resource Arena

When rendering takes place, platform-specific instances of the geometry data are created in the Resource Arena, which acts as a cache. If the same geometry data is required later, the cached data in the Resource Arena will be used directly.

3.4.4 Plugins

Plugins extend existing RenderWare Graphics objects and/or create new ones. In order for this mechanism to work, the memory management system needs to be in place.

The plugin must also be attached , by calling its associated `...PluginAttach()` function prior to the call to `RwEngineStart()` .

Chapter 4

Plugin Creation & Usage

4.1 Introduction

Plugins are an important feature of RenderWare Graphics. By providing a mechanism to extend and enhance the in-built objects of RenderWare Graphics, they give us the flexibility to consistently meet your needs in a fast-moving industry. The entire Retained Mode functionality for RenderWare Graphics is, in fact, provided as a plugin.

Toolkits

Toolkits don't extend any core objects or datatypes at all, and usually provide a set of utility functions.

In fact, a Toolkit is just a library that requires RenderWare Graphics to work. The RenderWare Graphics team uses the "Toolkit" term to differentiate a simple utility library from the more complex Plugins, which must be handled more carefully.

Plugins and Streaming

Streaming is covered in the [Serialization](#) chapter.

4.2 Using Plugins

This part of the chapter focuses on how to use plugins and how they work.

4.2.1 Attaching Plugins

Plugins have to be registered with RenderWare Graphics Core Library. The reason for this is so that any object extensions have the chance to hook into the memory handler and allocate space for their additional data structures. For example, if you wanted a plugin to extend camera objects by 64 bytes, then RenderWare Graphics needs to be told this before any cameras are created.

Most RenderWare Graphics applications follow a start-up process along the following lines (some error checking has been removed for clarity):

```
...

/* Platform-specific initialization */

... some code ...

/* Initialize RenderWare Graphics Engine: */

/* use default memory handler with Freelists
 * and set arena size */
RwEngineInit(NULL, 0, resArenaSize);

...code to choose display device, format and other parameters ...

RwEngineOpen(&openParams);

/* Attach Plugins */
if (! RpWorldPluginAttach  ())
{
    /* Something went wrong so, */
    /* either display an error message, */
    /* or just crash.*/

    return FALSE;
}
```

Other plugins should be attached at the same point in the start-up sequence. After this, accessing a plugin's API is the same as calling any other function.

After all necessary plugins are attached, the RenderWare Graphics engine can be started:

```
/* Start Engine */  
RwEngineStart(); /* "We have ignition!" */
```

With the plugins attached, calling their functions is now possible:

...

```
/* Render World */  
RpWorldRender() ; /* Render the RpWorld object. */
```

...

As with toolkits, plugins also have to have associated libraries and header files that must be included in your build. Merely attaching them is not enough.

Dependencies

Plugins must be attached in the right order. If plugin B relies on plugin A being present then plugin A should be attached before plugin B.

For example, the World plugin (RpWorld) is required by a number of other plugins. RpWorld must therefore be attached first.

4.3 Creating Your Own Plugins

4.3.1 Introduction

Your application is not limited to using the plugins supplied with the RenderWare Graphics SDK. Plugins can be written by any licensed developer.

In this part of the chapter, we will see how this is done.

4.3.2 Anatomy of a Plugin

A RenderWare Graphics plugin is divided in two parts.

The first part is the exposed API that users of the plugin will have access to. This API is the one other developers will use: it's the reason for writing the plugin.

The second part is not usually seen by the plugin's user and performs the housekeeping tasks: it performs any initialization and shut-down processes required by the plugin; it tells the core engine how much memory it needs for the new data structures, and also attaches itself to other plugin objects if needed.

For instance, when calling `RpWorldPluginAttach()` to attach the World plugin, it is the housekeeping functions that are called to create the extended objects and initialize any data structures.

4.3.3 The "Plugin" Example

Path: `examples/plugin`

This example defines both a plugin and a simple demo to exercise it.

The `main.c` file contains the demo program which exercises the plugin.

The plugin code is in `physics.c` and `physics.h`.

The `physics.h` exports the plugin's "public" API. As you can see from the plugin's API, the plugin extends the World plugin's `RpClump` object by adding some basic physical properties.

The base object name for this extension is "ClumpPhysics", so the public API uses an "RpClumpPhysics..." prefix.

Defining the Plugin

Near the start of the `physics.c` file, we see the first requirement for a plugin, the Plugin ID:


```
#define rwID_EXAMPLE 0xff
```

This example uses an ID of 255, but most developers will produce more than one plugin and so will use ID numbers starting from 0 upwards.

The ID shown above is only part of the full Plugin ID. The complete ID is created using the `MAKECHUNKID` macro, which combines your Vendor ID with the Plugin ID number.

You can see this used in the `RpClumpPhysicsPluginAttach()` function.

This function is the one called by the application just after `RwEngineOpen()` . It performs the operations required to extend the core library and any other extensible objects.

Plugin & Vendor IDs

The maximum number of plugins possible is 256 per Vendor ID. If you need more than this, you should contact Criterion Software Ltd. for another Vendor ID.

(Your Vendor ID is the same number used to log in to the RenderWare Graphics Support web site.)

Extensible Objects

These are objects which have a function in their API ending with `"PluginAttach()` ". Without this functionality, it is impossible to directly extend such an object.

Because of this, a number of the core library objects, such as `RwBBox` are not directly extensible using the plugin mechanism. Instead, you will need to define a new, extensible container object with one of the inextensible objects as a member—a wrapper object—which you can then extend.

New structures

The next step is to define the new structures with which we will extend the core engine and any other extensible objects.

In this example, we are adding some new global variables (at the core library, or "global", level), and extending the World plugin's `RpClump` object (the "local" level) to support some basic physics variables. Hence, we now define two structures: `PhysicsGlobals` and `PhysicsLocals` .

The global-level structure defines storage for a caption, gravity and a minimum speed. The caption is there because it is easy to spot when debugging.

Global structures are added to the core RenderWare Graphics engine itself.

The Local structures in this plugin are attached to each `RpClump` object created by the application.

If we were extending other objects, such as `Atomics`, we'd also need to define `Local` structures for them along with associated copiers, constructors and destructors.

Data Access Macros

One of the original design goals for C++ was that all of its functionality could be implemented using standard C. The upshot of this is that you can do object-oriented programming in C. It's just nowhere near as elegant, natural or pretty.

An important issue is the need for some macros to make accessing the new "members" of extended objects as painless as possible. In the case of the `Physics` example, these macros are defined at the top of the `physics.c` code: `PHYSICSGLOBAL` and `PHYSICSLOCAL`

These macros provide access to the base address of the new data structures within the `RpClump` object. By using these macros, it is possible to access the relevant structures directly without having to add the overhead of a function call. It also makes the code a lot more readable.

Registering the Plugin

While object creation and destruction is relatively trivial stuff for a C++ expert, this kind of thing requires a bit more work when using good, old-fashioned C. In particular, we have to explicitly define constructors, copiers and destructors because there are no defaults for these purposes.

Going back to the `physics.c` code, you'll find functions with words like "constructor", "destructor" and "copier" in their names. These functions are defined in the first half of the file and represent the housekeeping part of the plugin's API.

Just for completeness, we also define functions for serializing the data to and from `RenderWare Graphics` binary streams.

As you can see from the source code, we need to produce constructor and destructor functionality for both the global and local data structures, but only the local data structures need copiers as well.

Stream reading and writing functionality should only be provided if the base object also supports streaming. This can be determined by looking for a function name ending in `...AttachPluginStream()`. If it doesn't, you should create a new wrapper object containing the base object and provide full streaming support through this.

The stream support functionality needed is: `Read()`, `Write()` and `GetSize()` – the last allows the `RenderWare Graphics` binary stream functions to work out how much data will be read or written to the stream.

How do we tell RenderWare Graphics about these functions? This is why we need the plugin attachment process to attach a plugin and register it with the core library. Once registered in this way, RenderWare Graphics will automatically call the new functions as callbacks.

Take a look at the `RpClumpPhysicsPluginAttach()` function. A cursory glance shows that it is split into three very similar sections:

Registering the Global-Level Extensions

Here, we use the Core Library's `RwEngineRegisterPlugin()` function to add the new global variables to the engine and register the constructor and destructor functions for them.

The return value is the offset into the RenderWare Graphics engine's global data structure where the new variables will be stored.

This value is very important: it's needed by the data access macros (see [Data Access Macros](#) on page 83.)

Registering the Clump Object Extensions

This section uses the `RpClumpRegisterPlugin()` function – part of the World plugin's API. The main difference between this section and the global-level extensions section is the additional need to register a copier function which will be called when clumps are copied or cloned.

Registering the Clump's Binary Stream Extensions

Again, very similar except that functions that hook into the stream handler are registered instead of constructors, destructors or copiers.

Finishing Touches

The rest of the `physics.c` file consists of the remaining publicly-exposed API functions. These implement a Newtonian physics model using our new structures to store the state of each extended clump object.

In keeping with the object-oriented design philosophy behind RenderWare Graphics, these functions provide access to the new data members.

At this stage, we know how to create a simple plugin; now let's take a look at the other side of the coin and see the new plugin in action...

4.3.4 Using the Physics Plugin

The example plugin is used by the code found in `main.c`.

As with all of the other RenderWare Graphics Examples, the "plugin" example is built on top of a simple platform abstraction layer called "The Skeleton". This is spread through a number of files and is used to provide a platform-neutral framework for all our sample code.

For clarity, this chapter only discusses the relevant functions specific to the "plugin" example.

The full source code to the Skeleton is available for you to peruse and examine. It can be found in: `shared/skel/`

The ClumpPhysics Demo

The new plugin – called `RpClumpPhysics` – is used in a simple demo which drops some wooden buckyballs onto a simple surface. Each buckyball is stored in a clump which has been extended using the new plugin. The physics attributes are used to give each buckyball a random bounciness and initial speed.

At this point, you should build and run the demo and play with it for a bit. Stepping through functions with the debugger is also advised so you can get an idea of the program execution flow.

Most of the code is basic stuff: create a floor, create the buckyballs, initialize it all and then enter the main event loop.

"Buckyballs

A "buckyball" is a nickname given to any structure or object that looks like a C_{60} molecule. The molecule's fully qualified name is Buckminsterfullerene and was named after Richard Buckminster Fuller (1895-1983), an inventor, architect, engineer, mathematician, poet and cosmologist. His most famous invention was the geodesic dome, a structure similar to that of C_{60} .

Preparation

We have already learned that the first thing you need to do when using plugins is to make sure they're attached. The Skeleton used as a framework for this example application can make it difficult to see where this process takes place.

You'll find the plugin being attached within the aptly-named `AttachPlugins()` function. This function gets called by the Skeleton's event handler at the appropriate point during the startup phase and, as you can see, our plugin isn't the only one being attached.

Dependencies

You'll note that the World plugin, which provides RenderWare Graphics Retained Mode features, gets attached before our `RpClumpPhysics` plugin. This is because our new plugin has to extend the features of the `RpClump` object—an object that is only available when the World plugin is attached.

It is important to bear this dependency issue in mind when developing your own plugins.

Exercising The Plugin

The demo consists of a bunch of buckyballs, dropped from a height onto a surface and allowed to bounce.

The code which sets up the World itself sets up the lighting, the camera and the floor. You can see this being done in the `CreateScene()` function. See the two chapters, `Worlds & Static Models` and `Dynamic Models`, for more information on scene creation.

The interesting bit begins when the buckyballs are initialized:

The `ClumpPhysics` plugin is used to add storage space to each of the buckyballs for some basic physical attributes relating to speed and "bounciness" – elasticity in technical terms.

This initialization process takes place in the `InitClumps()` function found in the `main.c` file. Look at the section following the `/* Initialize user plugin data... */` comment and you'll see the new `ClumpPhysics` plugin in use.

In this case, we're using the exposed property `get/set` functions we saw in `physics.c`, setting most of the attributes using a random number generator. (The `Active` flag we added to the clump object is used to keep a note on which of the buckyballs is still bouncing.)

Once the initialization phase is completed, the demo starts the main update cycle. The physics calculations are handled in `UpdateClumpDynamics()`. Here, the basic Newtonian Laws of Motion are applied to make the buckyballs behave in a manner approximating reality.

4.4 Plugin Design

4.4.1 Introduction

The Physics example we have looked at is pretty simple. Most real-world plugins won't be so trivial as to contain the plugin code within the test-bed itself: full plugins are usually created as standalone libraries that are linked into your application like any other library.

In this section, we will look at some design issues that you'll likely come across when designing your own extensions.

4.4.2 Extension vs. Derivation

The `RpClumpPhysics` plugin extends an existing object. This is great for small-scale additions, but this design does have a serious drawback: all `RpClump` objects will have these extra data structures attached.

This means that the buckyballs are not the only ones affected: the floor itself is also a clump and, therefore, it too has the storage space allocated for our `RpClumpPhysics` extensions. While this kind of overhead is acceptable for such basic examples as `ClumpPhysics`, clearly this is going to be a problem if memory is tight—as is the case on a number of our supported platforms.

The trick, then, is to find some way of optimizing the design such that only objects that actually need the additional data will have it available. By far the most efficient way of doing this with RenderWare Graphics is to create a plugin that creates a new object instead of just extending an existing one.

This is the equivalent of C++'s derivation mechanism and most of the plugins supplied with RenderWare Graphics derive new objects rather than extending existing ones.

One notable exception is `RpWorld`'s extensions to the `RwCamera` object. It adds new functions that retain the "RwCamera" prefix, rather than using "RpCamera".

4.4.3 Deriving new objects

The process of creating derived objects is not very different from what you've seen already.

The first step is to decide how to link the new object to its parent object so that the two can be used to share data efficiently. Thanks to the design of the C programming language, three options are available to the developer:

1. The object can be extended
2. A container object can be created

3. An extended container object can be created—a combination of options 1 and 2.

Object Extensions

This technique is the one used by the ClumpPhysics example, covered earlier in this chapter.

The base object is extended by adding a pointer to it which connects that object to the extended data. Multiple pointers can be used if there are multiple derivations. In addition, you could add a second pointer so that a child object can access its parent. This gives you increased flexibility for such things as iterator functions, without sacrificing too much valuable memory.

The drawback is the overhead of forcing all instances of the parent object to have a pointer that may not always be used.

Container Objects

This technique involves creating a master object through which accesses to the base and derived objects are made. It would usually be a simple construct such as a pair of linked lists. The `RpAtomic` object found in the World plugin is an example of this technique.

Container objects have advantages where you need to derive a large number of interrelated objects from a single base object with a one-to-many relationship: the use of a separate linking object removes the need to store a large number of pointers in the base object.

The disadvantage with this technique is that all access to the interesting objects must be made via this link object to maintain integrity. The linking mechanism must therefore be designed carefully to keep access time overheads as low as possible.

Extended Container Objects

This option sometimes makes sense when the base object has a wildly variable number of derived objects. The disadvantage is that access to the derived objects through the list will be relatively slow. In addition, the links can be tricky to maintain when there are a large number of interdependent objects.

4.4.4 Plugins & C++

RenderWare Graphics plugin mechanism can make life a lot easier if you're using C++ as your primary development language.

As an example, let's assume you're writing a wrapper class for RenderWare Graphics clump (RpClump) object. Ignoring constructor/destructor methods and other side-issues, your class might look something like this:

```
class CClump
{
    ...

    // private data

    ...

private:
    RpClump *myClump;
    RwInt32 someData;

    ...
}
```

A problem arises when you need to pass the clump data to one of the RenderWare Graphics iterator functions, such as `RpWorldForAllClumps()`. These functions trigger a callback which expects an `RpClump` parameter, not your `CClump` class. So how does the callback used by the iterator function get a reference to the wrapper class?

The answer is to extend the clump object, using the plugin mechanism, and add a pointer which your wrapper class then initializes with a pointer to itself. Now your callback function can get at the rest of the class' data, (usually by calling a plugin-supplied access function).

Chapter 5

The Camera

5.1 Introduction

Wander into any movie studio's sound stages and you'll see a common set-up: static sets, props and actors, all lit by a lighting rig and performing in front of one or more cameras.

RenderWare Graphics defines equivalents for most of these elements:

static sets (static models) are covered in the chapter Worlds & Static Models ;

dynamic elements, such as props and actors, are covered by the chapter on Dynamic Models ;

lighting is covered in the chapter on Lights ;

...and the Camera is covered here.

5.1.1 The Camera Example

Computer software has the useful ability to be interactive and can therefore be used to great effect as an educational tool in its own right. With this in mind, our RenderWare Graphics Demos Team has created a number of educational examples that demonstrate particular features of the SDK and let you play with them to see how they work and interact. The 'camera' example is one such, and provides an interactive illustration of the Camera object's features.

This Example will be referred to frequently to highlight the many properties of the Camera object. In the second part of this chapter, the source code will be discussed, to understand, how it all fits together.

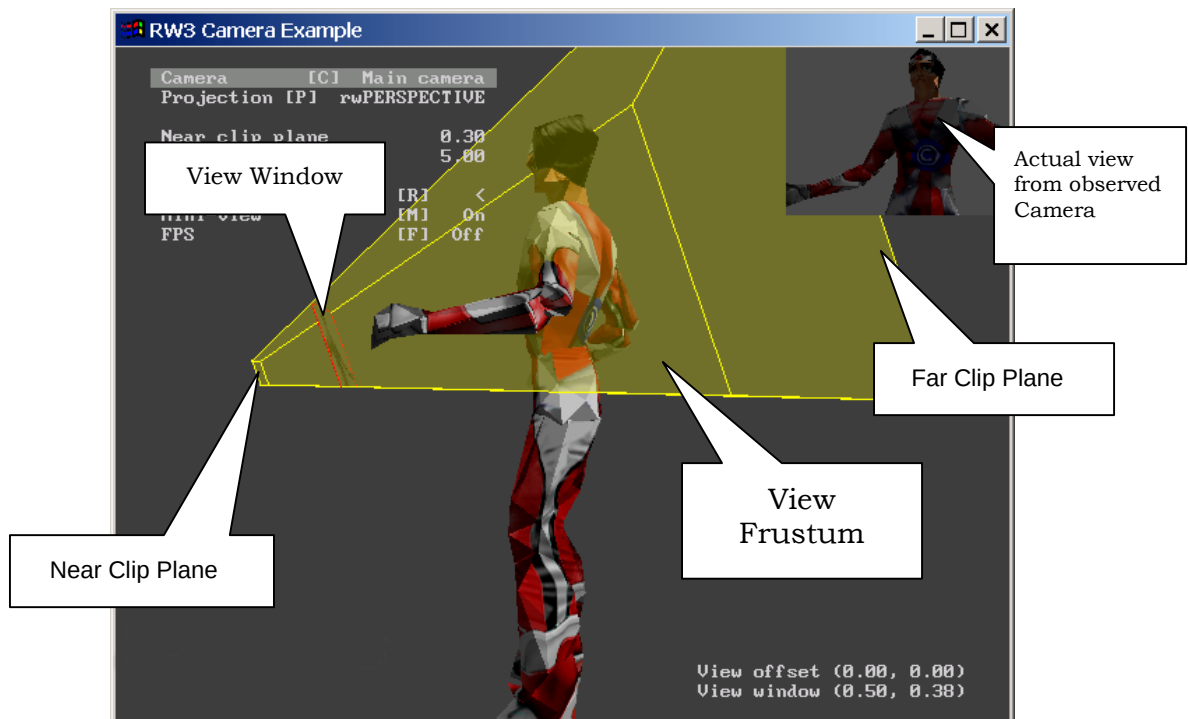
The Camera Example is in the `examples/camera/` folder. (Open up the text file to view the user interface instructions as these vary from platform to platform.)

5.2 The RenderWare Graphics Camera object

The RenderWare Graphics Camera (RwCamera) object is part of the Core Library. It is the target of all 3D rendering operations and therefore plays an important part in any 3D graphics application built on RenderWare Graphics.

5.2.1 Camera Properties

The screenshot below is taken from the 'camera' Example and it has been annotated to show the primary features of the Camera object:



Other Camera properties, such as projection modes, frame buffers, and fog handling, will be covered later.

5.2.2 The View Frustum

The highlighted volume in the diagram on the previous page represents the volume in front of the Camera within which a renderable object is considered visible. This is the View Frustum, defined by the near clip plane, the far clip plane and the four planes passing through the Viewpoint and the edges of the View Window. RenderWare Graphics uses the bounding sphere of the object to determine whether it intersects or lies within the view frustum. This coarse-grain visibility test is then used to determine whether or not to draw the object, and whether the object is clipped or unclipped. Clipping is a fine-grain visibility test.

If a model does not have a valid bounding sphere, the RenderWare Graphics engine may produce unexpected results.

The Clipping Planes

Two clip planes define the near and far extents of the view frustum.

The far clip plane defines the point along the Camera Space Z axis, beyond which objects will not be rendered.

The far clip plane is responsible for the infamous 'pop-up' found in older 3D games, whereby model geometry suddenly appears on the horizon as if from nowhere. If a platform has enough 3D graphics processing power, this plane can be pushed back so far that the 'pop-up' simply isn't visible.

The near clip plane defines the point along the Camera Space Z axis, nearer than which, objects will not be rendered. It is parallel to the far clip plane and on the opposite side of the view frustum. Again, this is just a point along the Z-axis running through the center of the camera object and into Camera Space.

The near clip plane controls clipping of polygons close to the camera object. Without it, even models behind the camera might be considered visible by the engine and be rendered to the camera's frame buffer.

Pushing the Near Clip Plane away from the Camera improves Z Buffer resolution. When feasible, the Near Clip Plane should be positioned as far away from the Camera as possible without introducing visible clipping artifacts. The distance at which this will occur depends on the zoom factor currently applied to the Camera. As a rule of thumb, dividing the distance of the far clip plane from the camera by the distance of the near clip plane from the camera should ideally result in a value of less than 1000.

The Viewpoint

When the Camera is set to the more usual Perspective view the View Frustum defines a section through a pyramid. The apex of this pyramid is the Viewpoint.

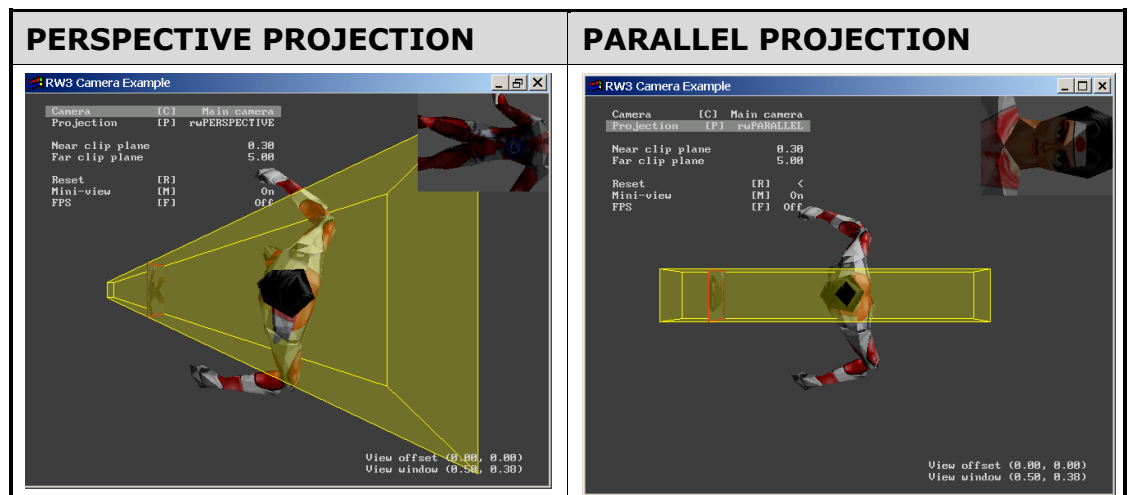
The Viewpoint is the center of projection when using a camera set to Perspective mode. It is, literally, the 'point of view' – the eye – in the scene.

Projection Modes

RenderWare Graphics provides two Projection Modes: Perspective and Parallel. Perspective projection is the most generally used and is the projection system assumed throughout most of this chapter. That said, Parallel projection mode, while not so frequently used in apps, does lend itself to a number of more specialist rendering tasks.

Perspective projection makes objects that are more distant (from the camera) appear smaller on the screen as they do in real life. The angular field of view may be controlled by calling the `RwCameraSetViewWindow()` function, as described in the API Reference Manual.

Parallel projection mode, which can be selected using the `RwCameraSetProjection()` function, changes the View Frustum to a View Cuboid.



As you can see from the screenshots above, the main difference between Perspective and Parallel projection modes is that the latter does not scale vertices in the Z axis – no matter how far away a model is, it won't get any smaller.

This mode is most often used for orthographic views or for projecting shadows from area light sources, but it can also be used to duplicate some popular 2D graphics rendering techniques, such as parallax scrolling. This has the advantage of using the 3D acceleration hardware features available on your target platform.

5.2.3 The View Window

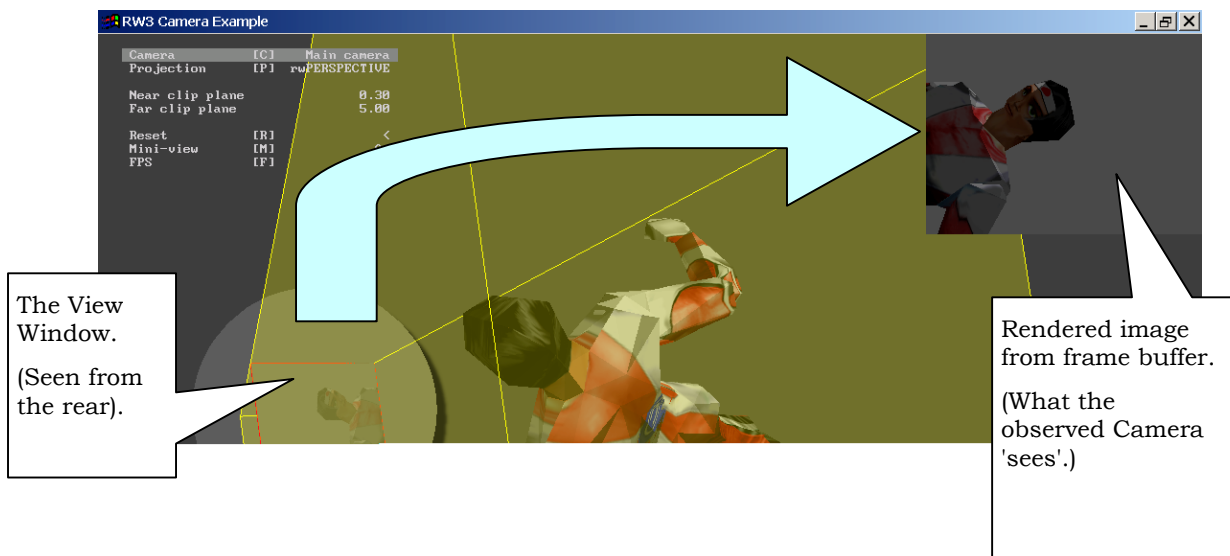
Earlier, the Viewpoint was described as being a virtual 'eye' looking into a scene. If we continue with this analogy, the View Window becomes a virtual 'retina'. Where the analogy falls down is that the View Window is in front of the Viewpoint.

Let's go back to the 'camera' Example.

With the 'camera' Example running, you can move the model around and see its image rendered into the view window. This is shown inside a red outline close to the near clip plane.

This View Window represents the frame buffer – our virtual retina – upon which the actual scene is rendered. The picture below shows this happening.

A useful feature of the 'camera' Example is that it displays the view from the observed Camera inset at the top-right corner of the screen. (Because of our view of the Camera object, the image shown in the View Window appears to be reversed because it is facing away from us.)



It is the contents of the frame buffer the View Window represents which the end-user gets to see.

View Windows & Aspect Ratio

Aspect ratio is defined by a combination of the View Window's dimensions and those of the frame buffer Raster.

While you can set the View Window to any arbitrary dimensions, the resulting image must still fit into the supplied frame buffer. Therefore, RenderWare Graphics scales the output so that it always fits into the frame buffer Raster.

This opens up a useful feature of the Camera: by manipulating the dimensions of the View Window and frame buffer Raster, we can:

Zoom into a scene by decreasing the dimensions of the View Window;

Zoom out of a scene by increasing the dimensions of the View Window;

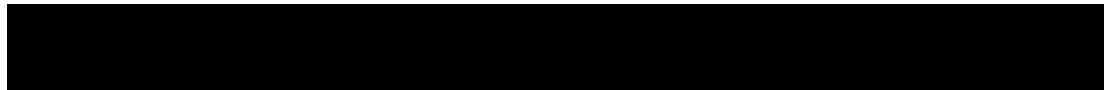
Duplicate an anamorphic lens by adjusting the dimensions of both the View Window and the frame buffer and Z buffer Rasters.

This last feature can be used to implement a typical requirement for a console game: support for anamorphic wide screen displays: Setting the View Window to a 16:9 aspect ratio, but keeping the frame buffer's dimensions to the traditional 4:3 TV standard achieves this. The rendered scenes will be squashed to fit into the frame buffer's smaller width. The resulting frame buffer can then be displayed on a wide screen TV using its anamorphic (i.e. 'stretched') setting. The result is that the squeezed frame buffer image becomes stretched across the entire width of the wide screen TV's display, restoring it to its intended aspect ratio.

The advantages of this technique are two-fold:

1. It saves memory. By removing the need for a wide screen frame buffer, we save on precious video RAM.
2. The lower memory overhead typically results in faster performance as system bus bandwidth requirements are reduced.

This technique can also be used to produce distortion effects.



The Camera View Window API

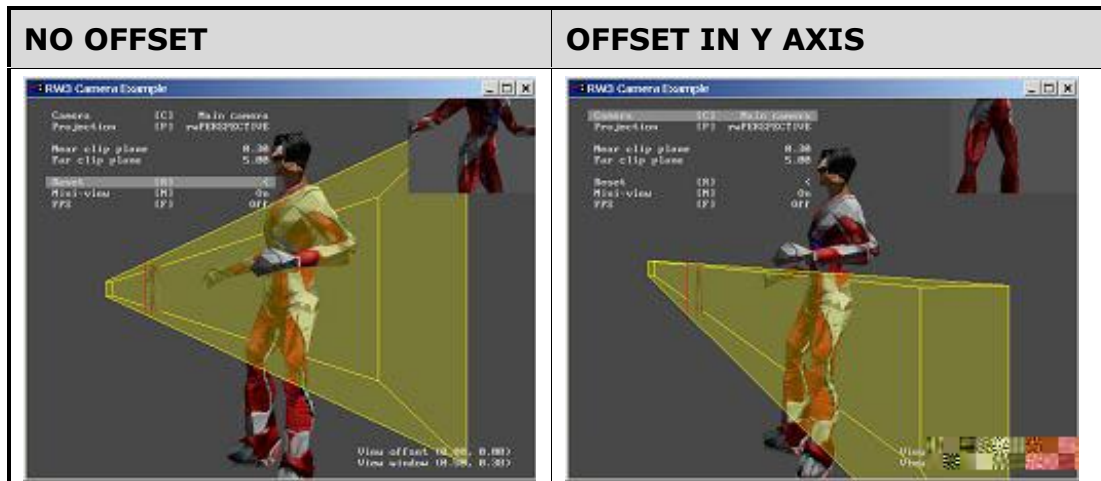
The View Window is set using an `RwV2dvector` set to half the required dimensions in each axis. The function call is `RwCameraSetViewWindow()`. The property retrieval function is `RwCameraGetViewWindow()`. Again, the returned `RwV2dvector` is set to half the actual dimensions.

The Raster object, used to provide the frame buffer and the Z buffer for Cameras, is covered in the `Rasters, Images & Textures` chapter.

5.2.4 The View Offset

The View Offset is used to shear the Camera's view frustum. This is done by specifying an X and Y (Camera Space) offset for the frustum's apex – the point defining the center of the far end of the frustum.

The screenshot below, left, shows an un-offset View Frustum. The screenshot to the right shows the View Frustum with a View Offset of 0.46 in the Y axis:



Notice how the Viewpoint – the apex of the View Frustum – is offset vertically relative to its original position. The result is the shearing of the View Frustum so that the View Window renders the lower portion of the model instead of the middle portion.

This feature can be explored in the 'camera ' example (see the associated readme.txt file for the user interface functions needed to manipulate the View Offset).

The offsets are absolute values, so repeated calls to `RwCameraSetViewOffset()` will not be accumulated.

This function has advantages for such activities as supporting stereoscopic display devices and tiled rendering to a high resolution output device.

Camera View Offset API

Two functions are supplied for this feature: `RwCameraSetViewOffset()` and `RwCameraGetViewOffset()`. Both take an `RwV2dvector` to set or store the offset.

5.2.5 Fog

Fog effects in RenderWare Graphics are tightly coupled to the Camera model.

A number of different fog models are supported by RenderWare Graphics: Linear , Exponential , Exponential ² and Table . However, some of these fog types are only supported on a sub-set of platforms.

Basic Fog Controls

Fogging starts at the distance set by `RwCameraSetFogDistance()` for Linear and Table fog types. Also, for the same fog types, the full fog effect is achieved at the far clip plane.

Fog Distance should never be set to zero.

Most of the fog handling is done through the `RwRenderState` API, so let's take a quick look at what's available...

Fog Render State

Fog is controlled through both the `RwCamera` and `RwRenderState` APIs. The Render State API provides the following fog control functionality:

`rwRENDERSTATEFOGENABLE`

Turns on fogging. All polygons rendered from this point onwards will be modified by fog processing. Clear this flag to disable fogging.

`rwRENDERSTATEFOGCOLOR`

The target color for fogging. The denser the fog around a particular polygon, the closer the polygon's color will be to this target color.

`rwRENDERSTATEFOGTYPE`

The type of fogging to use. The fog types defined are:

`rwFOGTYPELINEAR`

Selects the Linear fogging type.

`rwFOGTYPEEXPONENTIAL`

Selects the first Exponential fog type. This is available on most of our supported platforms. If the fog effect is represented by f then the function is:

$$f = 1 / e^{(d * \text{density})}$$

(where d is the distance from the Camera's View Point.)

`rwFOGTYPEEXPONENTIAL2`

Selects the second Exponential fog type. This is using a higher exponent in the formula to make the fog get denser faster. The formula is as above except that the exponent – $(d * \text{density})$ – is further raised to the power of two.

`rwRENDERSTATEFOGDENSITY`

Select the density of Exponential and Exponential² fogging. The higher the value, the denser the fog.

`rwRENDERSTATEFOGTABLE`

This is the Table form of fogging. The parameter is a 256-entry fog table placed between fog distance and far clip plane. The table format is not fixed, so check your platform-specific documentation to see what format to use if this fog type is supported.

Note that Render States can return `FALSE` for a number of reasons. Check the platform-specific information in the API Reference to ensure that a particular fog type is supported for a particular platform.

The Render State API, consisting of the two `RwRenderStateSet()` and `RwRenderStateGet()` functions should be used to set the fogging as appropriate.

The Render State represents the RenderWare Graphics state machine and is a shared resource. This means, when a particular render state is set, it stays set until it is cleared again, either by your own code, or by code elsewhere in RenderWare Graphics.

RenderWare Graphics makes heavy use of its rendering engine. Check the Render State flags if you find effects being applied inappropriately – like fog being applied to displaying text.

5.3 The Camera View Matrix

This matrix transforms world space coordinates to camera clip space. The camera's clip space is bounded by the near and far clip planes, and side planes defined as follows:

$x = 0$, $x = z$, $y = 0$ and $y = z$ for a Perspective projection model;

$x = 0$, $x = 1$, $y = 0$ and $y = 1$ for a Parallel projection model.

Camera View Matrix API

The View Matrix is accessed through the `RwCameraGetViewMatrix()` function.

5.4 Rasters & Cameras

In order to be useful, a Camera object has a couple of optional buffers attached to it that are used to store the resulting rendered image. These provide storage for both a Z Buffer and the frame buffer itself. In RenderWare Graphics, the Raster object is used for both of these buffers.

Rasters are a key feature of RenderWare Graphics and are covered in much greater detail in the Rasters, Images & Textures chapter.

The Raster types must be `rwRASTERTYPEZBUFFER` for the Z-buffer, and of type `rwRASTERTYPENORMAL` for the frame buffer.

The frame buffer Raster holds the rendered image. Since RenderWare Graphics provides a double-buffered model, this image is stored off-screen and is displayed when rendering is completed by the `RwCameraShowRaster()` function.

Z Buffer Accuracy

This can vary quite dramatically from platform to platform and application to application.

The Z buffer range is effectively dictated by both the spacing of the near and far clip planes, and the definition of the Z buffer Raster. This last might use anything from an array of 8-bit bytes to a map constructed from IEEE-standard double-precision floating-point values.

Clearly, this means you need to take care when setting up the view frustum as placing the near and far clip planes too far apart may result in rounding errors and related problems.

5.5 Creating a Camera

The process of constructing a RenderWare Graphics camera is typically as follows:

Create and allocate a Raster of type `rwRASTERTYPECAMERA`

Create and allocate another Raster of type `rwRASTERTYPEZBUFFER`

Create a Camera object using `RwCameraCreate()`

Attach the two Rasters to the Camera object using `RwCameraSetRaster()` and `RwCameraSetZRaster()`

Create a Frame object and attach it to the Camera object using `RwCameraSetFrame()`. The Frame allows for the control of the position and orientation of the Camera.

Finally, if you're using the `RpWorld` Retained Mode API, you will need to 'add' the Camera to the World. (See the two Retained Mode API chapters, `Worlds & Static Models` and `Dynamic Models` for more on this.)

At this point, the Camera is ready to use.

5.5.1 Orienting and Positioning a Camera in Scene Space

A quick skim through the `Fundamental Types` chapter reminds us that the Frame object is a container for the matrices used to position and orient objects in a virtual space.

Cameras use frames to define:

Where they are positioned

Which direction they are facing

The Camera will always face along the Frame's look-at vector. The look-up vector always points at the top of the View Window.

One oddity is that, because the Camera is facing away from us, its look-right vector is invariably pointing to the left from the user's point of view.

Transformations

Frames attached to Cameras behave just like Frames attached to anything else. Transformations are performed by using the `RwFrame` API and are used to position and orient a Camera.

Frame Origin

When attached to a Camera, the Frame's origin will be coincident with the Viewpoint. Note that this means the View Window will always be located exactly one unit along the look-at vector.

5.6 Rendering to a Camera

3D graphics rendering takes place between an `RwCameraBeginUpdate()` / `RwCameraEndUpdate()` pair. All rendering must take place between these two functions, although you can have more than one Begin/End pair operation before displaying the rendered image using `RwCameraShowRaster()`.

As many RenderWare Graphics applications use the Retained Mode (RpWorld) Plugin, a typical render cycle may look like this:

```
if ( RwCameraBeginUpdate (mainCamera) )
```

This line prepares the Camera for rendering, pushing its frame buffer Raster onto the context stack. The top-most Raster on this stack is used as the target for rendering. (Rasters are covered in more detail in the Rasters, Images and Textures chapter.)

The context stack is described in more detail in the "Immediate Mode" chapter.

At this point, Frame objects marked as 'dirty' will also have their Local Transformation Matrices (LTM) resynchronized. Any hierarchies hanging off the Frames are also updated. This ensures any rendered objects attached to the Frames are positioned and oriented correctly.

```
{
    RpWorldRender(gameWorld); /* Render the main game world */
```

We'll cover the RpWorld Retained Mode API in detail in later chapters. For now, suffice to say that this function will render an entire scene.

```
/* Now render any UI overlays, such as HUDs, high score */
... some more rendering code...
```

At this stage, you may want to add some special effects, such as lens flare or other visual gloss. Many such effects are often best achieved using the Immediate Mode APIs.

```
RwCameraEndUpdate (mainCamera);
/* End of rendering cycle. */
```

At this point, RenderWare Graphics will pop the Camera's Raster off the context stack. It also cleans up after itself and performs any necessary housekeeping.

We do not, however, have a visible result: the rendering has taken place, but because RenderWare Graphics uses double buffering, we need to swap the buffers over so we can see the results of our toil...

```
/* Display the results of the rendering. */
RwCameraShowRaster (mainCamera, (void *)win32Handle,
rwRASTERFLIPWAITVSYNC);
}
```


This function performs the buffer-swap needed to display our rendered scene. The parameters are, in order: the Camera object, a platform-specific parameter (in this case, a HWND container as the Win32 SDK is being used in this example), and finally, a flag can be specified to tell RenderWare Graphics to wait for the vertical blanking interrupt. (This last should only be used in a full-screen application.)

We can now update our scene ready for the next rendering cycle.

Destroying a Camera

Camera objects need to be explicitly destroyed when you are finished with

5.7 Sub-Rasters

Sub-Rasters can be used with Cameras to provide the following:

- Split-screen views, often seen in multi-player games

- Picture-in-picture and other inset effects, such as rear-view mirrors in driving games

These effects are achieved by sharing a single Raster across multiple Camera objects.

For full details on how Sub-Rasters work, see the [Rasters, Images & Textures](#) chapter.

5.8 Other Features

Clearing The Buffers

This process should be performed at the beginning of each frame unless your scene rendering is refreshing each pixel in the buffer every cycle anyway.

The function for this procedure is `RwCameraClear()` . Refer to the API Reference Manual for more details.

Cloning a Camera

RenderWare Graphics provides the `RwCameraClone()` function for this purpose.

As with most other 'clone' operations in RenderWare Graphics, this means both the source Camera and the target Camera objects will share the Frame, frame buffer Raster and Z-buffer Raster. Also, if the source Camera had been added to a World (`RpWorld`) object, the cloned Camera will also be in that World.

5.9 World Plugin Extensions

About the Extensions

The World Plugin adds a number of additional functions to the Core Library's Camera API. These are described here.

5.9.1 Automatic & Manual Culling

When rendering scenes using the World Plugin (RpWorld) API, the Plugin will automatically cull any Atomics and World Sectors that are located outside the Camera's view frustum. It will also cull any dynamic Lights whose spheres of influence are inapplicable within the view frustum.

However, this only applies to model data that has been explicitly 'added' to the World object. If you need to render an object, such as an Atomic, explicitly – and this is sometimes desirable – then you will need to perform the culling yourself.

This can be achieved using the `RwCameraFrustumTestSphere()` function, which will test a Bounding Sphere (`RwSphere`) object passed to it and tell you whether it is inside, outside or intersecting the specified Camera's view frustum. The values it will return are: `rwSPHEREINSIDE`, `rwSPHEREOUTSIDE` and `rwSPHEREBOUNDARY` respectively.

5.9.2 Clump Cameras

An `RpClump` is a container for dynamic objects that are associated with a frame hierarchy, and are described in more detail in a later chapter. It is possible to add Cameras to a Clump using the function `RpClumpAddCamera()` . Such Cameras are streamed with the Clump, and their location within the frame hierarchy preserved. This mechanism is used whenever Cameras are exported from the modeling packages.

5.9.3 Iterators

A number of useful iterator functions are provided to access model data that falls either within or outside the Camera's view frustum. The iterators cover a number of object types.

In all cases, the iterators can rely Bounding Spheres to perform their tests.

World Sectors

These represent static model geometry. One iterator is supplied: `RwCameraForAllSectorsInFrustum()` . This will iterate through all World Sectors that are either partially or entirely within the view frustum, calling a specified callback function for each one it finds.

The World Sector functions include a number of iterators for locating objects that lie within them, such as Atomics and Lights.

Clumps

Clumps represent dynamic objects and two iterators are supplied for these.

The first is `RwCameraForAllClumpsInFrustum()` . Like the World Sector form described above, this function will iterate through all Clumps that contain Atomics either partially or entirely within the view frustum. For each such Clump, it will call the specified callback function.

An inverse function is also supplied:

`RwCameraForAllClumpsNotInFrustum()` . As the name implies, this iterator calls the specified callback function on any Clumps that contain Atomics all lying entirely outside the view frustum.

The callback function is of type `RpClumpCallBack()` .

Chapter 6

Rasters, Images and Textures

6.1 Introduction

This chapter discusses bitmaps, images, rasters and textures and explains when and where they are used. These objects can all be found in the Core Library:

RwImage

RwRaster

RwTexDictionary

RwTexture

6.2 Bitmaps & Textures

Bitmaps play a major part in any 3D graphics engine. They form the basis for texture maps.

6.2.1 Bitmaps

RenderWare Graphics' Core Library supports two different types of bitmap objects: Images and Rasters .

While the APIs for these objects are superficially similar, there is one very important difference. The Image (RwImage) object is platform independent; the Raster (RwRaster) is not.

The reason for this duality is to give you the most flexibility without sacrificing power. Images can be easily processed and manipulated, but cannot be displayed until they have been converted to Rasters. Rasters have minimal manipulation and processing facilities, but are highly optimized for the underlying hardware.

6.2.2 Images

Images provide a platform-independent object for manipulating bitmap data.

Images are used to load bitmaps from disk or other media storage prior to conversion to a platform-optimized Raster object, which can then be manipulated.

6.2.3 Rasters

Rasters provide a platform dependent form of image and are used to hold image data. Rasters are frequently found inside other RenderWare Graphics objects. Of particular note are the Camera (RwCamera) and Texture (RwTexture) objects. For more information about cameras see the chapter on The Camera .

Rasters are the only form of bitmap that RenderWare Graphics can actually render to a display device. Image objects are normally used as intermediaries.

6.2.4 Textures

A texture is a bitmap that is stretched across a polygon. Textures use Rasters to store the actual bitmap data. The Rasters are also used to store MIPMAPs.

It is the Raster itself that supports MIPMAPping, not the Texture object.

6.3 The Image Object

Images represent a platform-independent bitmap.

They can support most common image formats and this makes them ideally suited as intermediaries.

For example, when you load a bitmap, such as a PNG or BMP, from disk, it ends up in an Image object.

6.3.1 Image Dimensions

Image objects can have any width or height. Dimensions are limited only by available memory. The color depth can be four, eight or thirty-two bits per pixel.

Images do not support packed pixel formats. This means that the bitmap actually stores only one pixel per byte, even if only four of the bits in each byte are significant.

6.3.2 Stride

An Image is associated with a stride . A stride defines the physical number of bytes needed to get from one pixel in the bitmap to the one directly below.

The stride allows a bitmap to be defined as a portion of a larger bitmap. This kind of usage is popular as it reduces loading times by combining smaller images into one larger one.

6.3.3 Palettes

Four-bit and eight-bit Image depths require palettes.

Palettes consist of:

- 16 entries for 4-bit formats

- Up to 256 entries for 8-bit formats.

Each entry is an RGBA value defining a color.

6.3.4 Gamma Correction

Image objects also support a gamma correction value. This value defines the gamma curve of the device upon which the bitmap is to be displayed. Changing this value and applying the gamma correction enables you to adjust the gamma of the bitmap.

What is Gamma Correction?

This process exists to make graphics appear as close to the original as possible, regardless of any differences or gamut limitations in the display device used. Gamma correction is similar to the "contrast" control on a television set or monitor.

When applied, the RGB values are multiplied by the gamma correction value to increase or decrease the effective level of an image's pixels.

Consumer video devices, such as televisions, have a higher native gamma than dedicated computer monitors. This means that graphics drawn on a monitor will tend to look brighter on a television set. This may not be what the graphics artist intended, hence the provision for gamma correction.

6.3.5 Creating Images

Image objects can be created either by allocating memory for the data and assigning that data to an Image, or by reading the object from a RenderWare Graphics Binary Stream.

Creating Images by Assigning Data to an Image

Initialization is fairly straightforward: a call to `RwImageCreate()` is made, supplying width, height and depth values.

This gives a dummy image object with all its members except those that point at the actual bitmap data. To include these `RwImageAllocatePixels()` is used. This looks at the width, depth and height of the image and calculates the memory required to store this Image.

If your Image uses a palette format then space also needs to be allocated for the palette. `RwImageAllocatePixels()` will handle this for us automatically by looking up the settings in the Image object.

For example, a depth of 8 bits will result in the creation of a 256-entry palette.

Loading Images

Image objects are most commonly loaded. They act as containers for bitmaps loaded from a disk or CD. Usually, bitmaps are drawn by an artist and then loaded into the game ready-made.

Bitmap files include standard formats, for example, Windows Bitmap (.BMP), Portable Network Graphics (.PNG).

RenderWare Graphics doesn't support any of these formats and `RwImageRead()` routine does not support any standard formats by default.

RenderWare Graphics does, however, have four toolkits, which add support for four popular file formats: `RtPNG`, `RtRAS`, `RtTIFF` and `RtBMP`. These provide a single callback function, which the RenderWare Graphics Image Streaming API uses to handle a particular file extension.

More information about these Toolkits is in the table below:

FORMAT	EXTENSION	TOOLKIT	SUPPORTED FEATURES
--------	-----------	---------	-----------------------

Microsoft®
Windows®
Bitmap format

For this example the Windows Bitmap (.bmp) file format will be used:

1. Add the appropriate header – `rtbmp.h` – and ensure our application is linked with the associated Toolkit library. (For Win32 developers, this library is `"rtbmp.lib"`.)
2. Register the RtBMP Toolkit's supplied callback functions with the Core Library so it knows which functions to call in order to read or write BMP-format images using the `RwlImageRegisterImageFormat()` function. This takes a string containing the associated filename extension, a pointer to the image reading function and another pointer to the image write function.

In this instance, the registration code would look like this:

```
RwlImageRegisterImageFormat ("bmp", RtBMPImageRead,  
                             RtBMPImageWrite);
```

Any number of additional image formats can be registered in this way. When reading or writing Images, RenderWare Graphics will look up the supplied filename extension against the registered extensions in its database and call the appropriate callback functions.

Search Paths

Before an image can be read we have to ensure RenderWare Graphics knows where to look for it. For this purpose, RenderWare Graphics' Core Library provides the `RwlImageSetPath()` function. This accepts a string containing one or more search paths separated by semicolons. RenderWare Graphics will use this to find the Image filename supplied to the read function.

For obvious reasons, the `RwlImageWrite()` function requires an absolute, fully qualified path, filename and extension. It makes no reference to the search path.

6.3.7 Reading the Image

It is now possible to read or write an Image object using the `RwlImageRead()` and `RwlImageWrite()` functions.

These work by passing a filename to the function and, if successful, it will return a pointer to an `RwlImage` object with your image in it. If you pass a filename without a path, RenderWare Graphics will use the search paths to attempt to locate the file.

Two important points to note are:

1. If you haven't registered the image file format you're trying to read or write the function call will fail.
2. The `RwlImageRead()` function requires the file extension so RenderWare Graphics can work out which registered reader function to use.

For example, a "BMP"-format image file is being read, we would perform the registration process just detailed, then call the `RwImageRead()` function to perform the Image reading, as below.

```
{
    ...

    /* BMP Registration must have been done prior to this point. */

    myImage = RwImageRead("myimage.bmp");

    if (!myImage)
    {
        /* Failed to read the image. Handle error condition. */
    }

    ...

}
```

Assuming all went well, a pointer to a ready-to-use Image object should be stored in `myImage`.

Any image processing or manipulation can be performed on our Image.

Masked Images

RenderWare Graphics can use Image alpha channels as masks when available. (The "A" in RGBA formats represents the alpha/mask data). Masks are covered later in this chapter.

6.3.8 Image Processing

The `RwImage` API supports a number of basic image manipulation functions. These cover such processes as re-sampling, copying, gamma correction, masking (for basic alpha channel effects) and resizing.

Creating an Image object

Sometimes, you'll need to create an empty Image object, which is to be populated with data and bitmaps later in its life-cycle. This process can be a one- or two-stage process, depending on what the object is to be used for.

To instantiate the Image object itself:

1. Call the `RwImageCreate()` function and pass it the dimensions you need and it will return an Image object that is almost, but not entirely, complete.
2. Allocate the memory for the bitmap and the palette (if this is a palletized Image). This is achieved using `RwImageAllocatePixels()`. This will look at the dimensions of the Image object and use these data to work out how much memory the Image will need. It will also allocate memory for the palette if one is required.

The reason for this two-step process is to allow multiple Image objects to share a single bitmap. This is a useful feature as you can, for instance, have a number of sprites stored within a single bitmap. To make access to each of these sprites easier, you can define multiple Images which share the same physical bitmap and point at individual sprites.

You can therefore set the bitmap data pointer in an Image object directly using `RwImageSetPixels()` .

Re-sampling

There are two re-sampling functions. Both take an original Image object and produce another Image of a different width and/or height. The main difference between them is that the first function creates and allocates a new re-sampled Image object for you, while the second does not:

`RwImageCreateResample()` function allocates and creates a new Image for the re-sampled bitmap.

`RwImageResample()` requires that you supply a valid `RwImage` object for both the source and the destination. You will need to use both `RwImageCreate()` and `RwImageAllocatePixels()` to create the Image objects.

`RwImageCreateResample()` also differs in that it allows you to specify a palletized image as the source. However, the result is always an Image of 32-bit depth.

Re-sampling actually re-scales the source image to fit the new dimensions. RenderWare Graphics always uses the CPU to perform this operation.

Accessing and modifying Image properties

RenderWare Graphics' Image API provides `get` functions to access Width, Height, Depth and other properties. These are `RwImageGet[PROPERTY]()` form. For example:

`RwImageGetHeight()` , `RwImageGetDepth()` , `RwImageGetStride()`

Most of these have equivalent `Set` forms. The only exceptions being Width, Height and Depth. These are set through the `RwImageResize()` function which takes all three as parameters.

Copying

`RwImageCopy()` will copy a source Image into a destination Image. Both Image objects must already be created (using `RwImageCreate()`), and bitmap memory allocated (using `RwImageAllocatePixels()`), as this copying function will not create the destination Image for you.

Alpha Channel Masks

All Image objects have storage for alpha channel information, either directly in the bitmap (32-bit formats), or in the palette table (4-bit and 8-bit formats). While some image file formats support alpha channels directly, many others do not so the RwlImage API includes functionality to get around this and allows you to transfer alpha channel data from one Image to another.

In particular, functions are provided for loading masks stored in a separate file (`RwlImageReadMaskedImage()`); creating masks from pixel data (`RwlImageMakeMask()`); and applying a mask stored in one Image to another (`RwlImageApplyMask()`).

At the time of writing, RenderWare Graphics supports two image file formats that can handle alpha channel data:

the Portable Network Graphics format (.PNG), supported by the "RtPNG" Toolkit

the TIFF format (.TIFF), supported by "RtTIFF ".

These can read a 32-bit RGBA PNG-format or TIFF-format file complete with alpha channel. A single Image is returned.

6.3.9 Raster Conversion

So far, Rasters haven't been covered in detail. The RwlImage API also provides functionality to convert a Raster to an Image object and vice versa. The function that performs this magic is `RwlImageSetFromRaster()` . This function will take a Raster and convert it into an Image of the same dimensions. The target Image must be valid, initialized to the right dimensions and allocated enough memory for the pixel data.

A complementary function, `RwRasterSetFromImage()` also exists to perform the reverse conversion. This is covered in more detail in the section on Rasters.

Palletized bitmaps can be more efficient on some platforms. RGBA formats may work better on others. You should refer to platform-specific documentation to determine which bitmap formats are best suited to your target platform.

6.3.10 Destroying Images

Any RenderWare Graphics image objects you have explicitly created must be destroyed.

Images should be destroyed using the `RwlImageDestroy()` function. If the bitmap and palette data were allocated using the `RwlImageAllocatePixels()` function, the destructor function will also free this memory. Otherwise, you will need to free it yourself.

6.4 The Raster Object

Images must be first converted into a platform-dependent format if they are to be rendered. This platform-dependent format is known as the Raster (RwRaster).

Rasters also provide the foundation for RenderWare Graphics' virtual camera as well as its texture-mapping features. This makes the Raster one of the most important objects in RenderWare Graphics.

6.4.1 Basic Properties

Like Images, Rasters represent bitmaps. Therefore, they contain all the basic features of a bitmap: width, height, depth, palette data (if applicable) and the bitmap data itself. As with Images, Rasters can also support alpha channel data, if the platform supports it.

The main difference between Rasters and Images is the lack of control available to you over format and other basic properties. There's a good reason for this:

Rasters will support only the formats most suited to the underlying platform, and nothing else .

This platform-dependence is a very important aspect. A few points regarding platform-dependence are below:

Just because you managed to get a 16-bit Raster on one platform, there is no guarantee that you will be able to get that same format on another platform. In fact, there is nothing to prevent Rasters using such non-RGB formats as DYUV or similar.

Another related point is that some platforms may only support Rasters with dimensions that are a power of 2 or some other arbitrary limitation.

Never rely on being able to obtain a particular Raster format.

In fact, it is quite possible for later versions of RenderWare Graphics to switch to another Raster format for a particular task.

Developers should also note that RenderWare Graphics treats PC and Apple Macintosh graphics cards as separate platforms in their own right. Rasters are optimized for specific hardware, not specific operating systems.

Creating Rasters

Rasters are created using the `RwRasterCreate()` function. This takes the usual width, height and depth settings. It also takes a `flags` value that determines what the Raster is to be used for.

Rasters are primarily defined by their purpose. For example, a Raster can be used as storage for texture data so setting the `RWASTERTYPETEXTURE` flag would produce a Raster with additional storage space for MIP-levels (on platforms that support MIPmaps).

An important point to note is that the width, height and depth values are limited to those supported by the underlying platform. For instance, some platforms may require that all Rasters have dimensions that are exact multiples of two.

In short: be prepared to have this function fail if you attempt to create an unsupported Raster format.

It is common for developers to leave the depth setting to zero as this lets RenderWare Graphics choose the best depth for the platform and means the function call is more likely to succeed.

Creating Rasters from Images

Bitmaps are often loaded from a file on disk. As Rasters cannot be read directly from any of the popular file formats – only Image objects can do this – this means that bitmaps intended for use as Rasters will need to be read as Image objects first.

The `RwRaster` API provides the `RwRasterSetFromImage()` function for this purpose. As with many other RenderWare Graphics conversion functions, a valid, initialized `RwRaster` object needs to be provided as a destination. The function will not allocate a new Raster.

`RwImageFindRasterFormat()`

This function accepts an Image and a hint flag telling it what the Image data is to be used for. It uses this information to determine the best available Raster pixel format for the Image.

Creating MIPMAP Rasters

Rasters can contain multiple bitmaps – known as MIPMAPs. This is a popular system used for texturing 3D models. A MIPMAP stores a base bitmap and zero or more re-scaled versions of the same bitmap. The rendering engine can choose to upload a smaller bitmap if the texture is being viewed from a distance, as the smaller bitmap's lower detail won't be noticed.

This has the advantage of reducing the memory bandwidth required to render the texture and, over a whole scene, this technique can dramatically increase efficiency. (Another use for MIPMAPs is to reduce the effects of aliasing.)

Creating MIPMAP Rasters can be done in two ways:

1. Manually, by reading individual Image files for each MIPMAP level.
2. Automatically, by specifying the base image and telling RenderWare Graphics to create the remaining MIPMAP levels itself.

To do this, set the raster type to either `rwRASTERFORMATMIPMAP` or `rwRASTERFORMATAUTOMIPMAP`. Then setting the `Rasters NumLevels` property to the number of MIPMAPs required.

If the type is set to `rwRASTERFORMATMIPMAP` you can:

1. Choose to lock the Raster at each MIPMAPs in turn.
2. Call `RwRasterSetFromImage()` with the requisite Image to store the MIPMAPs.
3. Unlock the Raster.
4. Repeat the unlock process for each of the levels until the Raster is ready.

If the type is set to `rwRASTERFORMATAUTOMIPMAP` the Raster:

1. Is Locked.
2. Then, read in the base Image bitmap using `RwRasterSetFromImage()`.
3. Unlock the Raster.

RenderWare Graphics will generate the remaining MIPMAPs for you using a simple re-scale algorithm. This issue is covered in more detail in the Textures section, later in this chapter.

Accessing Raster Properties

There is only one "set" function in the `RwRaster` functions in the API Reference. This is a conversion function.

The reasons for this were covered in the Basic Properties section earlier: Rasters are designed primarily as "look but don't touch" objects. Changing their properties is likely to impact on performance, so the `RwRaster` functions are designed to avoid that by not providing property-setting functions in the first place!

Pixels and palette data can be accessed. The only problem with this is that the data may be compressed or in some strange format so the Raster bitmap data may not be easily manipulated.

To get at the data the Raster needs to be locked. This process is also used to tell RenderWare Graphics if you intend to read and/or write to the data. If you write data into the Raster, RenderWare Graphics may need to re-generate MIPMAPs if the Raster is of type `rwRASTERFORMATAUTOMIPMAP` (this should be avoided wherever possible).

While most of the basic properties – width, height, depth, stride, etc. – are the same as their `RwlImage` equivalents, the following four need more explanation:

1. `NumLevels`

This property defines the number of MIP-levels defined in a Raster when a Texture object is using it. Such Rasters can contain multiple bitmaps.

The function to access this property is `RwRasterGetNumLevels()` .

2. Format

The Format of a Raster is usually dictated by the underlying hardware – particularly its bit-depth. The Format property can be used to query the actual bit-depth and format of the Raster palette.

For instance, a 16-bit Raster using a 1555 form for ARGB will return `RWRASTERFORMAT1555`

A current list of valid formats can be found in the API Reference. See the entry for `RwRasterGetFormat()` .

3. CurrentContext

The current Context is a pointer to the Raster used as the target for 2D rendering by RenderWare Graphics' rendering engine.

The Context mechanism uses a stack, consisting of two functions `RwRasterPushContext()` and `RwRasterPopContext()` .

4. Parent

There is a special kind of Raster, called a sub-Raster , which is simply a Raster that shares another Raster's bitmap data. The sub-Raster must therefore keep track of which Raster its actual bitmap data is stored in, hence the "Parent" property.

Sub-Rasters come into their own when used with Camera objects. For instance, split-screen rendering is usually achieved by using sub-Rasters. This is covered in the chapter on The Camera .

The `RwRasterSubRaster()` function is used to create these. `RwRasterGetParent()` will return the parent Raster containing the actual bitmap data.

6.4.2 The Raster as a Display Device

While Rasters can be used as sprites and textures, a Raster also represents the display device itself. This means that Rasters spend most of their time being rendered onto other Rasters.

RenderWare Graphics' rendering engine uses double buffering. This means that rendering takes place off-screen and you don't see anything updated until you explicitly swap the buffers over. (Most hardware supports this by simply flipping memory addresses, although some older devices require a physical blit operation instead.)

When 3D graphics are rendered to the virtual camera object, this is actually rendering to a Raster. This is why, once the rendering cycle is completed, the `RwRasterShowRaster()` function needs to be called in order to see anything. This performs the buffer swap needed for double-buffering to work.

The `RwRasterShowRaster()` function also takes a flag which lets you tell RenderWare Graphics to wait for the vertical blank interrupt ("Vsync" or "VBI") before performing the switch.

Rasters used as targets for 3D rendering are always attached to Camera objects. This means they must have the `rwRASTERTYPECAMERA` raster type.

6.4.3 Rendering Rasters

Rendering Rasters is a 2D render process and therefore involves an `RwRasterPushContext()` / `RwRasterPopContext()` function pair. All 2D rendering must be topped and tailed by these two functions.

In other words, the code should be structured something like this:

```
...

if (RwRasterPushContext(DestRaster))
{
    /* Do any rendering of Rasters here.*/
    RwRasterPopContext(); /* Done with our 2D rendering. */
}

...
```

There are three functions provided for Raster rendering.

1. `RwRasterRender()` , this performs a simple blit operation, copying the Raster to the target Raster – the Context – and taking the alpha channel transparency into account while doing so.
2. `RwRasterRenderFast()` , performs a faster blit operation, but takes no account of the alpha channel. This makes it considerably faster on most platforms.
3. `RwRasterRenderScaled()` , this operates much like `RwRasterRender()` while also taking an `RwRect` parameter defining the target rectangle to blit to. The Raster will be scaled during the blit operation to match the target rectangle. (The original data is not changed; the scaling takes place during the blit itself.)

In all three cases, the Z-buffer is ignored.

If you need this functionality, you should use Textures and the Immediate Mode APIs.

6.4.4 Accessing Rasters

Rasters do not support many processing functions. Two bitmap clearing functions exist:

`RwRasterClear()` - clears the entire bitmap array

`RwRasterClearRect()` - clears a defined rectangle within the bitmap.

A Raster might be using a non-standard format, so, Rasters need to be locked before any major processing is performed.

Locking is done using the `RwRasterLock()` function. The function takes both a MIP-level and lock mode. The MIP-level specifies the MIP-map level desired from the Raster. (0 is the highest resolution – the level displayed when the texture is close to the Camera.) The lock mode enables you to tell RenderWare Graphics what you intend to do with the Raster:

`rwRASTERLOCKREAD`

`rwRASTERLOCKWRITE`

`rwRASTERLOCKREADWRITE`

`rwRASTERLOCKNOFETCH` This mode lets you tell RenderWare Graphics that you intend to overwrite all the bitmap data without reading any of it. This can speed things up considerably on some platforms.

Once locked, the Raster can have its pixels accessed and manipulated.

Of course, if the Raster is in some non-standard format, it makes more sense to convert it to a more useful one. Usually, this is achieved by converting the Raster to an Image using `RwImageSetFromRaster()`.

This is a procedure often used for creating screen dumps, avoiding the need to point cameras at screens in order to create screenshots for packaging. In such cases, you simply lock the Camera Raster and convert it into a known Image format before either storing it somewhere in memory, or writing it to disk using one of the Image format Toolkits.

`RwRasterUnlock()` will reverse the locking process.

If your Raster has a palette, you will need to use the `RwRasterLockPalette()` and `RwRasterUnlockPalette()` functions as well if you need access to it.

`RwRasterGetFormat()` will tell you if you have a paletted Raster.

6.4.5 Reading Rasters from disk

Rasters can be read and written to disk using the `RwRasterRead()` and `RwRasterReadMaskedRaster()` functions. Both will read an Image from disk (using the `RwImage` API) and convert that Image to a Raster of type `rwRASTERTYPENORMAL`.

These functions are most often used to read data such as sprites or UI elements.

6.5 Textures & Rasters

RenderWare Graphics has a Texture (`RwTexture`) object. This is, as you would expect, used to provide the texture handling features in RenderWare Graphics.

Textures are actually thin wrappers for Rasters, which means Textures shouldn't be considered as entirely autonomous entities. For instance, we have already seen that MIPMAP features are actually handled at the Raster level.

The number of MIPMAP levels is returned by `RwRasterGetNumLevels()` .

Rasters that are to be used as MIPMAPs should have either the `rwRASTERFORMATMIPMAP` or `rwRASTERFORMATAUTOMIPMAP` Raster Type.

In this section, we'll take a close look at RenderWare Graphics' Textures and how they relate to Rasters.

6.5.1 Introducing Textures

Textures add some additional properties to the Raster:

Filtering

This determines how MIPMAPs are to be used (if at all) and rendered. A number of Filter Modes are provided and are listed in the MIPMAPs section, below.

Addressing modes

This determines how the texture is mapped to the polygons. "Texture Space" is always defined as running between 0 and 1, so that for any given Texture, the Texture's own U and V coordinates will be as follows:



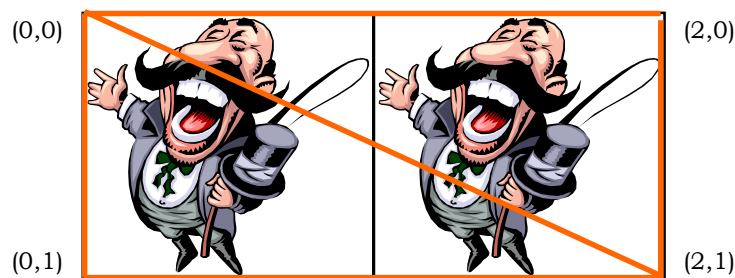
This Texture Space is mapped onto polygons during rendering by mapping the Texture's own U and V coordinates to those linked to each Material and/or Vertex. Thus, rendering the Texture with the U and V coordinates shown below would result in the Texture being drawn actual size:



But you can use the U and V mapping to stretch or shrink the Texture in one or two axes, or even to tile it or produce mirrored reflections of it depending on the Addressing Mode. The diagrams below show some options available:

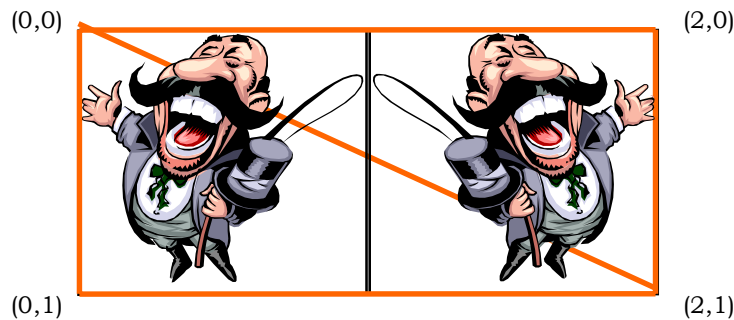


Here, the Texture is stretched across a wide polygon (built using two triangles). Stretching is simply a matter of stretching the polygons while retaining the same U/V mapping.



This is the same texture rendered using a tiled addressing mode called "rwTEXTUREADDRESSWRAP". Note the right-most U coordinates are set to 2, which is exactly twice the size of Texture Space, hence the Texture is repeated exactly once.

Other addressing modes let you modify this tiling behavior. For instance, if the addressing mode were set to `rwTEXTUREADDRESSMIRROR`, the result might look like this:



You can use the texture Example to experiment with different addressing modes and see their effects.

The full list of addressing modes is listed in the Addressing Modes section, below.

MIPMAPS

These are simply individual bitmaps, which are used to make up a texture. In RenderWare Graphics' case, all MIPMAPs for a particular Texture object are stored in a single Raster.

The purpose of MIPMAPs is to allow a Texture to use different bitmaps according to its distance from the virtual camera. This technique allows smaller, less-detailed bitmaps to be used when a model – and, hence, its Textures – are a long way from the camera. This results in a more efficient use of the platform's memory.

MIPMAPs are divided into MIP levels. Level 0 is the default and is always present in a Raster, which means that it is also always present in a Texture. Level 0 is usually the largest of the bitmaps with each additional MIP level half the size of the previous one.

MIPMAPs are enabled or disabled explicitly on a per-texture basis using the `RwTextureSetMipmapping()` function.

During the rendering process, RenderWare Graphics will use the Texture's Filtering settings to determine which MIP level(s) should be used. These are detailed below.

Filtering

Filtering is the process used to determine how – and if – the Texture object selects and displays individual MIP levels.

It is usual for the MIP levels to be blended smoothly from one to the other across the level threshold rather than simply switching between two states and the last two Filters listed below can be used to support this where available.

FILTER	EFFECT
<code>rwFILTERNEAREST</code>	No MIP mapping. Point-sampled textures.
<code>rwFILTERLINEAR</code>	No MIP mapping. Bilinear interpolated textures.
<code>rwFILTERMIPNEAREST</code>	Point-sampled MIP mapped textures. Individual MIPMAPs are drawn using point-sampling and will simply flip from one image to the next as the level changes over distance.
<code>rwFILTERMIPLINEAR</code>	Bilinear interpolated MIP mapped textures. MIPMAPs are drawn using bilinear interpolation. They still flip from one image to the next as the level changes over distance.
<code>rwFILTERLINEARMIPNEAREST</code>	MIPMAPs are interpolated between levels so giving a smooth blend from step to step rather than simply flipping from one image to the next at the threshold. MIPMAPs are drawn using point-sampling.
<code>rwFILTERLINEARMIPLINEAR</code>	Tri-linear interpolated textures. Interpolation works in three dimensions, running through U, V and MIP level axes. Gives the best visual results but can be expensive on some hardware.

The first two of these Filter types are for Textures that are not MIP mapped and therefore only have their MIP-level 0 bitmap defined. The remaining Filter types are MIPMAP blend modes, listed in order of complexity and visual quality, from least to best.

An ideal world would have every Texture mapped using the best quality Filtering – in this case, tri-linear interpolation. But this Filter can require a lot of processing power, so you will need to balance your mix of Filter modes.

SDK-Examples: "MIPMAP"

This Example demonstrates MIPMAP filters available on your platform.

Some filters may be unsupported on certain platforms. Attempting to use unsupported Filters may result in an error condition being returned from the `RwTextureSetFilterMode()` function.

Addressing Modes

Addressing modes define the process used to determine a particular texel color. These modes define how the indexing along the texture map's U and V axes behaves when the edge of the texture map is reached.

The addressing modes defined by RenderWare Graphics are:

ADDRESSING MODE	EFFECT
<code>rwTEXTUREADDRESSWRAP</code>	This mode is the default. When the edge of a texture map is reached, RenderWare Graphics wraps back to the opposite edge. The result is that the texture map is tiled.
<code>rwTEXTUREADDRESSMIRROR</code>	This is similar to the default mode except that alternate tiles are mirror images of the original.
<code>rwTEXTUREADDRESSCLAMP</code>	This mode clamps the U and V indices at the edges so that the edge-most pixel is repeated across the remainder of the surface. It is sometimes used for one-off decals.
<code>rwTEXTUREADDRESSBORDER</code>	This mode is similar to the Clamp mode. However, this mode applies the <code>BORDERCOLOR</code> Render State color to the polygon where the U and V indices fall outside the bitmap's range.

examples\texadrss

This Example demonstrates all the Addressing Modes available on your platform.

Addressing Modes can be varied by U/V axis. There are three API functions available to set the Addressing Mode for a Texture:

`RwTextureSetAddressing()` applies an Addressing Mode across both U and V axes so that behavior in both directions is the same. The appropriate property interrogator is `RwTextureGetAddressing()`.

`RwTextureSetAddressingU()` applies an Addressing Mode across the U axis only. The appropriate property interrogator is `RwTextureGetAddressingU()`.

`RwTextureSetAddressingV()` applies an Addressing Mode across the V axis only. The appropriate property interrogator is `RwTextureGetAddressingV()`.

A couple of important notes:

Firstly, if you are using different Addressing Modes across each axis, the `RwTextureGetAddressing()` function will return `rwTEXTUREADDRESSNATEXTUREADDRESS`.

Secondly, some Addressing Modes may be unsupported on particular platforms and will cause `RwTextureSetAddressing()`, `RwTextureSetAddressingU()` and `RwTextureSetAddressingV()` to return an error condition.

6.5.2 Loading Textures

Textures are loaded using the `RwTextureRead()` function.

This function makes use of the `RwlImage` API, so you must register any image file format readers with `RwlImageRegisterImageFormat()` before attempting to load Textures.

It is important to note that `RwTextureRead()` will also perform some additional operations:

A search is made in the default Texture Dictionary (`RwTexDictionary`) for an existing Texture with the same filename stored in its "name" property. If no such Texture is found, the function attempts to load it as expected. However, if a matching Texture is located, the function simply returns a pointer to this Texture.

On the first attempt to read a Texture, the function will, if the data was read successfully, store the filename in the Texture's "name" property.

The function also adds the Texture to the default Texture Dictionary if it is not already in there.

That last point is important: it means Textures will only ever be loaded once, unless you explicitly change the Texture's "name" property.

Texture Dictionaries are covered a little later on.

Platform Specific Information

RenderWare Graphics uses the platform's conventions for pixel and texel centers. These may vary per platform. For an example, look at the `imshadow` example.

MIPMAP Generation

During the load process, MIPMAPs can be generated automatically or additional MIPMAPs images can be read and added to the Texture. The first step is to tell RenderWare Graphics whether we want MIPMAPs generated at all. This is done through a call to `RwTextureSetMipmapping()`.

If you want MIPMAPs to be generated for your Textures, you need to call `RwTextureSetAutoMipmapping()`. This lets you choose:

1. To let RenderWare Graphics generate all MIPMAPs levels automatically from a single bitmap loaded from disk.

Call `RwTextureRead()` with the filename of the bitmap to load. RenderWare Graphics will then generate the MIP-levels itself by re-sampling the original image as required for each level.

2. To load all the individual MIPMAPs directly from disk.

Individual images are named something like: "image_m?.ext ", where "image " is the name of your image, "?" is a number between 1 and 9 inclusive and "ext " is the three-letter extension of the image file format used.

The Third Way: Generating MIPMAPs yourself

This system can be overridden by supplying the `RwTextureSetMipmapGenerationCallBack()` function with a pointer to a function that will be called to generate each MIP layer. (See the API Reference for the callback's prototype.)

Your callback function will be given a pointer to the destination Raster and the source Image: what it does with them is up to you.

This naming system is restricted by the ISO 9660 filename convention. ISO 9660 dictates an 8.3 format, which limits image filenames to a maximum of five useful characters on most platforms.

`RwTextureSetMipmapping()` and `RwTextureSetAutoMipmapping()` affect only the loading of Textures using the `RwTextureRead()` functionality. These settings will have no effect whatsoever on automatic Texture loading by Clumps, Worlds and other RenderWare Graphics objects.

Accessing MIPMAPs

Individual MIPMAPs can be accessed at the Raster level using the `RwRaster` API's `RwRasterLock()` and `RwRasterUnlock()` functions, as well as their palette equivalents. These were described in more detail in the section on Rasters.

6.5.3 Texture Dictionaries

It is often convenient to group Textures together using some form of simple database to make it easier to manage them. For this purpose, RenderWare Graphics provides the Texture Dictionary (`RwTexDictionary`) object.

Texture Dictionaries are collections of Textures. The Textures are indexed via their "name" property. (API functions: `RwTextureGetName()` to retrieve; `RwTextureSetName()` to set.)

Names are associated with both Textures and their Masks and, to keep object sizes constant, names are limited to a maximum of thirty-two characters in length.

The 32-character limit for Texture names may appear to contradict the earlier point about the ISO-9660 filename convention.

In fact, Texture names are not directly linked to filenames and can therefore provide a more convenient means of accessing Textures. You can use the `RwTextureSetName()` , `RwTextureGetName()` and the two Mask name equivalents; `RwTextureSetMaskName()` and `RwTextureGetMaskName()` , to manipulate and change Texture names directly.

Streaming Texture Dictionaries

In the section about loading Textures, RenderWare Graphics automatically maintains a default Texture Dictionary (`RwTexDictionary`) object was mentioned. This prevents RenderWare Graphics from making unnecessary reads from a disk or CD as it can check to see if the Texture is already loaded and just return a pointer to that if so.

Reading a Texture Dictionary from a binary stream is a very quick way of getting all Textures and Rasters into memory. It eliminates the need to load each individual Texture from its own file and perform the conversion of the bitmap from an Image to a device-dependent Raster.

If textures are being loaded individually, the time needed to locate and read each one from the storage media—usually a CD or DVD disk—the seek time alone slows the loading process down dramatically.

Generation of MIPMAPs can be particularly expensive on CPU time and system resources, so this should be kept to a minimum. This is why the Texture Dictionary mechanism was created: you can create them just once for a particular platform, store all the Textures into a Texture Dictionary object then stream said object out for re-use in your application.

examples\ texdict

This SDK example exercises the Texture Dictionary API and shows how to construct, write and then read a Texture Dictionary.

It can be found in the examples folder.

6.5.4 Using Texture Dictionaries

When you write a Texture Dictionary to a binary stream, the result is a file containing all your Textures with their associated Rasters ready for use.

The API functions to use Texture Dictionaries with RenderWare Graphics Binary Streams are the standard three:

`RwTexDictionaryStreamGetSize()` , `RwTexDictionaryStreamRead()` and `RwTexDictionaryStreamWrite()` .

Using Texture Dictionaries

It is possible to create your own Texture Dictionaries rather than using the default one created by RenderWare Graphics. This lets you have, for example, a separate Texture Dictionary for each level in a game. Multiple Texture Dictionaries can also be loaded into memory.

To get a pointer to the active Texture Dictionary, use `RwTexDictionaryGetCurrent()` . Which can also access the default database created by RenderWare Graphics.

To set the current active database `RwTexDictionarySetCurrent()` function needs to be called. Pass `NULL` if you do not want RenderWare Graphics to use a Texture Dictionary at all. (This can also be used to disable the default database created by RenderWare Graphics.)

Platform dependent texture dictionaries for PS2, Xbox and GCN can be generated on the PC using `nullsky`, `nullxbox` or `nullgc` libraries. They do not have to be generated on the target platform.

Other API features are:

Adding and removing Textures from the database is achieved through the `RwTexDictionaryAddTexture()` and `RwTexDictionaryRemoveTexture()` functions.

To locate a Texture in the database by name, use `RwTexDictionaryFindNamedTexture()`.

An iterator function, `RwTexDictionaryForAllTextures()`, uses the standard RenderWare Graphics callback mechanism to give you access to all Textures within the Texture Dictionary.

6.5.5 Textures & Binary Streams

While Textures can be read directly from disk or stored in Texture Dictionaries, they can also be streamed to and from RenderWare Graphics Binary Streams.

Once opened using the standard `RwStream` API the `RwTextureStreamRead()` and `RwTextureStreamWrite()` functions can be used to read and write Texture objects to the stream.

It is important to note that Textures are not serialized with their Raster data. Instead, writing a Texture object results in a data stream containing the Texture's settings and properties along with the filename from which the Texture's Raster was created. Therefore a reference to the filenames needs to be made by opening each one using the Image object API and performing the conversion into a Raster for the Texture to use.

Because of this, most developers will prefer to use the Texture Dictionary

Chapter 7

Immediate Mode

7.1 Introduction

Immediate Mode contains 2D and 3D graphics functionality. The API functions can be found in the Core Library. There are two Immediate Mode APIs `Rwlm2D` and `Rwlm3D`. They provide low-level control over the underlying hardware through a multi-platform API.

A quick glance at the API Reference reveals that both the Immediate Mode APIs are based around the rapid manipulation and rendering of primitives . Primitives are built up of lines or filled triangles. All other polygons are built up of combinations of triangles. The line-drawing features are typically used for very specialized effects, such as wire frame rendering or GUI features.

7.1.1 Properties & Render States

Before Immediate Mode features are looked at in more detail, there are some very important points to note that apply to both these 2D and 3D APIs. In particular, the properties of any and all RenderWare Graphics primitives are specified and defined by the developer.

These properties include:

- Color

- Position

- Render State

- Texture Coordinates

Colors are specified on a per-vertex basis for most rendering tasks. For example, a line drawn with a red vertex at the start and a blue vertex at the end will be drawn with the color blending smoothly from red to blue along the line.

Positioning vertices requires performing transformation operations on said vertices. RenderWare Graphics includes `RwV2d`, `RwV3d` and `RwMatrix` data types as well as the API to manipulate them.

Perhaps the most important property of the two Immediate Modes is the Render State . The Render State is a collective term for a group of settings that define how primitives are to be rendered. It is accessed through the `RwRenderStateSet()` and `RwRenderStateGet()` functions. It manages issues such as texturing, fogging and alpha blending.

The Render State system has its own API and this will be covered in more detail later in this chapter.

7.2 2D Immediate Mode

This mode forms the basis of RenderWare Graphics' rendering engine.

There are relatively few functions to grasp because the 2D Immediate Mode API performs line and triangle rendering. These basic shapes, known as primitives, form the basis of the Immediate Mode APIs.

Lines and triangles are built up using vertices stored as `RwIm2DVertex` objects. The properties of this object will be examined shortly, but first, fundamental concepts behind 2D Immediate Mode will be covered.

7.2.1 Basic Concepts

Coordinates

RenderWare Graphics' 2D Immediate Mode API works in screen space, also known as device space. This means that you work directly with display device coordinates and, in turn, this means the output is resolution and platform dependent.

The `RwIm2DVertex` object stores coordinates as `RwReal` types, making them floating point values rather than integers. This makes it easier to handle features, such as anti-aliasing, which require sub-pixel precision in order to work.

Coordinates can also be defined in terms of the camera space. This is of use mainly for layering rendered imagery over a previous rendering. (For example, layering an effect over a model rendered by a Retained Mode plugin.) Camera Space rendering takes place in a 3D co-ordinate system with the origin and orientation defined by the current Camera position and orientation.

Rt2d Toolkit

This resolution-dependency is what separates 2D Immediate Mode from the Rt2d Toolkit. The Rt2d Toolkit provides a higher-level 2D graphics API that is resolution independent. It also makes heavy use of the 3D graphics facilities in RenderWare Graphics to support a number of powerful hardware-accelerated features, including rotation, translation and alpha blending.

Clipping

The 2D Immediate Mode API does not make any attempt at clipping. If you attempt to draw using invalid coordinates, the results will be undefined and a crash is quite possible. You should make full use of debug builds when testing Immediate Mode applications.

Z-Buffers

The output of 2D Immediate Mode can make use of Z-buffers. This lets you position your 2D renderings along the Z-axis. In theory, a Z-axis is taken to point 'into' the screen.

However, in practice, hardware implementations of Z-buffers vary greatly from platform to platform. For instance, some may define the range of Z coordinates as running between 0 and 1 with values specified in floating point. Others may use integers running from 0 to -32768.

The upshot of this is that the RenderWare Graphics Z-buffer may work differently from platform to platform.

Near & Far Z

A common requirement for 2D graphics is the need to place the rendered imagery at either the closest or furthest point along the Z-axis. These limits are defined by the underlying hardware, which dictates the resolution of the Z-buffer.

Two functions are provided to support these requirements: `Rwlm2DGetFarScreenZ()` and `Rwlm2DGetNearScreenZ()`. These will return the minimum and maximum possible Z coordinates for the active Raster as a `RwReal` value. You should ensure any Z values are clipped to this range.

Vertices

While the result of working with 2D Immediate Mode is primitives rendered on a display, your code will actually work directly with vertices, so let's take a look at these now.

The vertex is the main data type for RenderWare Graphics' Immediate Mode APIs. There are two such types, one each for 2D and 3D Modes.

The 2D Immediate Mode form is `Rwlm2DVertex`. As with its `Rwlm3DVertex` counterpart, this data type's physical format is platform-dependent, so you shouldn't try to access it directly. Instead, the API provides a complete set of functions to prepare and manipulate the vertex. Note: These functions are implemented as C Macros in most cases to provide optimum performance.

7.2.2 Initializing an RwIm2DVertex object

As mentioned earlier, this object is defined in a platform-specific way. This prevents us from documenting its structure. Initializing such objects is therefore performed entirely through the API.

An `RwIm2DVertex` object contains the following properties:

- Red, Green, Blue and Alpha components

- Camera-space coordinates (X, Y and Z)

- Reciprocal of Camera's Z coordinate

- Screen-space coordinates (X, Y and Z)

- U and V coordinates

Setting & Retrieving Coordinates

2D Immediate Mode vertices are usually in screen space and this means the `RwIm2DVertex` objects are also known as "device vertices". Since this vertex object is not exposed as a structure, you need to use the API functions, `RwIm2DVertexSetScreenX()`, `RwIm2DVertexSetScreenY()` and `RwIm2DVertexSetScreenZ()`, to set these coordinates programmatically. These functions have counterparts that can be used to retrieve these same values: `RwIm2DVertexGetScreenX()`, `RwIm2DVertexGetScreenY()` and `RwIm2DVertexGetScreenZ()`.

It is also possible to specify coordinates in terms of camera space – the functions are: `RwIm2DVertexSetCameraX()`, `RwIm2DVertexSetCameraY()`, `RwIm2DVertexGetCameraX()` and `RwIm2DVertexGetCameraY()`.

Reciprocal Camera Z, Texture Coordinates & Platform Dependence

The underlying platform's hardware has a great impact on the vertex object. Often, the `RwIm2DVertex` (and its 3D Immediate Mode counterpart) are simply wrappers for a hardware-specific data type.

This means that even the format of the colors can change. For instance, some platforms require colors to be specified in floating point form while others prefer integer values. This is why, if you look at the API Reference entries for the two Immediate Mode APIs, you'll find color setting functions that accept both formats. This gives you the option of either sticking to one color format – the functions will convert where necessary – or changing the format from platform to platform if you prefer that level of control.

In order for some rendering techniques, including Gouraud shading and perspective-correct texture mapping, to work correctly, you may need to specify the reciprocal camera-space Z coordinate of a vector and/or the actual camera-space Z coordinate itself.

This is an important facet of Immediate Mode programming: some functions simply don't exist on some platforms; on others, different functions may be required to achieve the same effects.

In the case of the reciprocal of Camera Z, the function (if available) to obtain this is `Rwlm2DVertexGetRecipCameraZ()`. (There is an equivalent `Rwlm2DVertexSetRecipCameraZ()` function too if the former is available.)

The reciprocal camera Z value is generally required to enforce perspective-correct texture mapping. Texture coordinates can therefore also be specified if your primitive(s) are to be so rendered. Currently, the vertex object will only support one texture – multi-texturing is planned – so setting the U and V coordinates is a matter of calling the appropriate `Rwlm2DVertexSetU()` and `Rwlm2DVertexSetV()` functions respectively.

The API Reference is rebuilt specifically for each platform so it will only contain the relevant platform-specific entries. Other platforms have different builds of the API Reference with different platform-specific entries as appropriate.

7.2.3 Primitives

RenderWare Graphics supports two major types of primitive: Lines and Triangles. These are in turn split into subtypes. Lines are split into line lists and poly-lines .

Line Lists

These are lists of vertex pairs defining the start and endpoints of lines. This data format is ideal for rendering groups of unconnected lines.

Poly-lines

This is a list consisting of one start point, followed by any number of endpoints. The Immediate Mode rendering engine will start at the first point, draw a line to the first endpoint and draw another from the end of that line to the next endpoint in the list. This primitive is ideal for sequences of connected lines.

Except for the first line, the poly-line primitive type does not require two vertices to be processed per line. This means it needs the least data to work and is therefore the most efficient of the line primitives. If possible, use this primitive type when working with lines.

Rendering individual lines can be done using `RwIm2DRenderLine()` . This function allows you to draw individual lines between arbitrary vertices. (See the API Reference for more details.)

It should be noted, however, that drawing individual lines in this way is not recommended as it can be slow.

RenderWare Graphics also supports a number of triangle primitives:

Triangle Lists ("Tri-List")

These are similar to the Line Lists mentioned earlier. They consist of triplets of vertices defining individual triangles. This implies that triangles in a tri-list are entirely independent of each other.

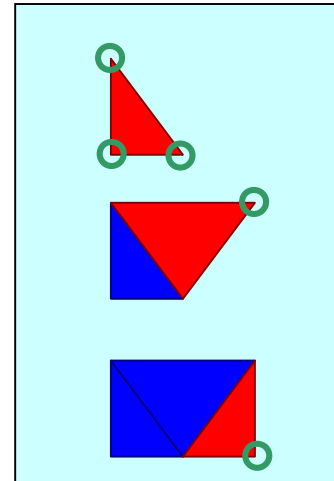
Triangle Strips ("Tri-Strip")

These define strips of adjacent triangles, each sharing an edge with another. This form requires fewer vertices than the Triangle Lists type as, after the first triangle is defined, only one additional vertex is needed to define each subsequent triangle. This has significant performance benefits on most platforms.

The diagram to the right demonstrates the advantage of tri-strips. Once the first three vertices (shown as rings in the diagram) have defined the first triangle, only one additional vertex is required to add each new triangle to the strip.

In the diagram, the second step shows a fourth vertex defining the second triangle (shown in red) and in the third step, we have just five vertices defining three triangles.

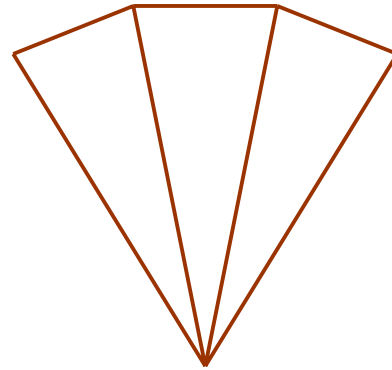
Obviously, this format is more efficient as you add more triangles.



Triangle Fans ("Tri-fan")

These are similar in concept to triangle strips, relying on contiguous triangles to reduce the number of vertices that need to be moved around and processed.

As you can see from the example to the right, a triangle fan requires that all triangles share a vertex and are contiguous. This form has much the same advantages as triangle strips.



Both the strip and fan forms are popular with transform and lighting processing hardware, so RenderWare Graphics is biased somewhat in their favor on most supported platforms.

The triangle fan and triangle strip primitives are not built on the fly by RenderWare Graphics' rendering engine. The RenderWare Graphics modeler exporter plugins allow you to export geometry optimized for triangle strips and/or fans.

7.2.4 Triangle Winding Order

Throughout RenderWare Graphics, the ordering of the vertices defining a triangle – the winding order – also define its visibility. RenderWare Graphics' use of a right-handed coordinate system means triangles with vertices defined in clockwise order (relative to the camera) have a normal pointing away from the camera and are therefore not visible, so they will not be drawn.

In summary: visible Triangles must have vertices defined in anti-clockwise order, relative to the camera, as the normal to the Triangle is pointing towards the camera.

7.2.5 Primitives vs. Indexed Primitives

2D Immediate Mode can work directly with arrays of primitives, which are in any case just arrays of vertices. It can also work with arrays of indices to such arrays of vertices. This second technique is particularly important as it opens up more efficient use of system memory. In particular, it lets you re-use vertices rather than having define duplicates.

The SDK includes a number of Examples, including the `im2d` Example. This illustrates 2D Immediate Mode by rendering a number of different primitives, including indexed and non-indexed forms.

7.2.6 Example 1: Rendering A Line.

This code fragment renders a single, simple, opaque red line:

```
Rwlm2DVertex vertex[2];

// Set up the vertices...
Rwlm2DVertexSetScreenX(&vertex[0], 20.0f );
Rwlm2DVertexSetScreenY(&vertex[0], 20.0f );
Rwlm2DVertexSetScreenZ(&vertex[0], 3276.0f );
Rwlm2DVertexSetRecipCameraZ(&vertex[0], (1.0f/6.0f) );

Rwlm2DVertexSetScreenX(&vertex[1], 620.0f );
Rwlm2DVertexSetScreenY(&vertex[1], 460.0f );
Rwlm2DVertexSetScreenZ(&vertex[1], 3276.0f );
Rwlm2DVertexSetRecipCameraZ(&vertex[1], 1.0f/6.0f );

// Opaque red...
Rwlm2DVertexSetIntRGBA(&vertex[0], 255, 0, 0, 255);

// ...and the same for the second vertex.
Rwlm2DVertexSetIntRGBA(&vertex[1], 255, 0, 0, 255);

// Texturing off...
RwRenderStateSet(rwRENDERSTATETEXTURERASTER, NULL);

// Flat shading on...
RwRenderStateSet(rwRENDERSTATESHADEMODE, (void
*)rwSHADEMODEFLAT);

// Alpha-transparency off...
RwRenderStateSet(rwRENDERSTATEVERTEXALPHAENABLE, (void
*)FALSE);

//Render line...
Rwlm2DRenderLine(vertex, 2, 0, 1);
```

Release Builds

There appear to be a lot of function calls involved in this code, but the overhead is not as bad as it first appears.

The RenderWare Graphics libraries are supplied in three builds : Release, Metrics and Debug.

Release builds redefine many trivial functions as macros. The upshot is that almost all property setting and access functions, such as those above, are inlined.

Example 2: Rendering A Triangle.

This code fragment renders a simple, yellow, flat shaded triangle:

```
RwIm2DVertex vertex[3];

//Set up the vertices...
RwIm2DVertexSetScreenX(&vertex[0], 20.0f );
RwIm2DVertexSetScreenY(&vertex[0], 20.0f );
RwIm2DVertexSetScreenZ(&vertex[0], 3276.0f );
RwIm2DVertexSetRecipCameraZ(&vertex[0], (1.0f/6.0f) );

RwIm2DVertexSetScreenX(&vertex[1], 20.0f );
RwIm2DVertexSetScreenY(&vertex[1], 40.0f );
RwIm2DVertexSetScreenZ(&vertex[1], 3276.0f );
RwIm2DVertexSetRecipCameraZ(&vertex[1], (1.0f/6.0f) );

RwIm2DVertexSetScreenX(&vertex[2], 40.0f );
RwIm2DVertexSetScreenY(&vertex[2], 40.0f );
RwIm2DVertexSetScreenZ(&vertex[2], 3276.0f );
RwIm2DVertexSetRecipCameraZ(&vertex[2], (1.0f/6.0f) );

// Opaque yellow...
RwIm2DVertexSetIntRGBA(&vertex[0], 255, 255, 0, 255);

// Opaque yellow...
RwIm2DVertexSetIntRGBA(&vertex[1], 255, 255, 0, 255);

// ...and, once again, opaque yellow...
RwIm2DVertexSetIntRGBA(&vertex[2], 255, 255, 0, 255);

// Texturing off...
RwRenderStateSet(rwRENDERSTATETEXTURERASTER, NULL);

// Flat shading mode...
RwRenderStateSet(rwRENDERSTATESHADEMODE, (void
*)rwSHADEMODEFLAT);

// Alpha-transparency off...
RwRenderStateSet(rwRENDERSTATEVERTEXALPHAENABLE, (void
*)FALSE);

// Render triangle...
RwIm2DRenderTriangle(vertex, 3, 0, 1, 2);
```

As you can see from the examples above, there is an overhead involved in setting up the `RenderState` data. Because of this, it makes sense to group all your primitives by `RenderState` to speed up the rendering process.

7.3 3D Immediate Mode

3D Immediate Mode provides low-level 3D rendering facilities. It is designed around the same concepts as its 2D counterpart, including the concepts of vertex lists , index lists and primitives . In addition, it makes heavy use of RenderWare Graphics' Render State API to control the rendering process.

7.3.1 Preparation for Rendering

Vertices & Indices

3D Immediate Mode has its own vertex object: `Rwlm3DVertex` . This holds coordinates defined using either world space (also known as 'scene space'), or in object space . This distinction is covered in more detail later.

As with the 2D Immediate Mode's own vertex object, `Rwlm3DVertex` is also an opaque, platform-dependent type. Access is entirely through the API.

In fact, there isn't all that much to say about the 3D vertex object: aside from the extra dimension, it's relatively straightforward and the API is similar to that for 2D Immediate Mode's own `Rwlm2DVertex` object.

Where the 3D Immediate Mode API does differ slightly from the 2D one is that setting coordinates is done through a single function call: `Rwlm3DVertexSetPos()` . This takes the X, Y and Z coordinates as well as a pointer to the `Rwlm3DVertex` where they are to be stored. A retrieval function, `Rwlm3DVertexGetPos()` , is also provided.

As with the 2D Immediate Mode, 3D Immediate Mode expects vertices to be stored in arrays. These arrays can be referenced either directly or using an index array. The latter has the expected advantage of requiring fewer vertices, but can cause problems if your model geometry is to be adjusted in real-time.

Unlike its 2D counterpart, the 3D Immediate Mode API provides clipping.

Transforming Space

To determine how a particular vertex maps to the display device, the 3D Immediate Mode API provides a function that takes an array of 3D vertices and transforms them to camera space (which is much the same thing as device space).

The function that performs this feat is the `Rwlm3DTransform()` function. This takes four parameters, the third of which is an optional parameter specifying an additional transformation matrix to be applied.

This transformation matrix should be set when working with vertex arrays, which use object space. The matrix will be used to map the object to world space and then to camera/device space.

The `Rwlm3DTransform()` function normally produces two new arrays, one of camera space vertices and – for those vertices that lie within the camera frustum – an array of device space (device-dependent) vertices.

Access to these intermediate data representations should be performed very carefully. They are platform-dependent, some hardware accelerators may store these new arrays in strange formats – or even not at all.

Additional Vertex Properties

As well as the usual three coordinates, each 3D Immediate Mode vertex can store RGBA values. These are set using the `Rwlm3DVertexSetRGBA()` function. (The function accepts 8-bit unsigned integers, so floating point RGBA formats will need to be converted first.)

The remaining two properties are U and V coordinates for texturing. The functions for setting these are `Rwlm3DVertexSetU()` and `Rwlm3DVertexSetV()` respectively. Both functions take an `RwReal` value for the texture coordinates.

The 'impick' Example

This example lets you pick and drag vertices in wireframe 3D Immediate Mode models. The commented code therefore illustrates a wide variety of Immediate Mode programming techniques.

Lighting

Lights are not supported in 3D Immediate Mode. You can simulate them by manipulating the vertex colors in accordance with a custom lighting model.

7.3.2 Rendering

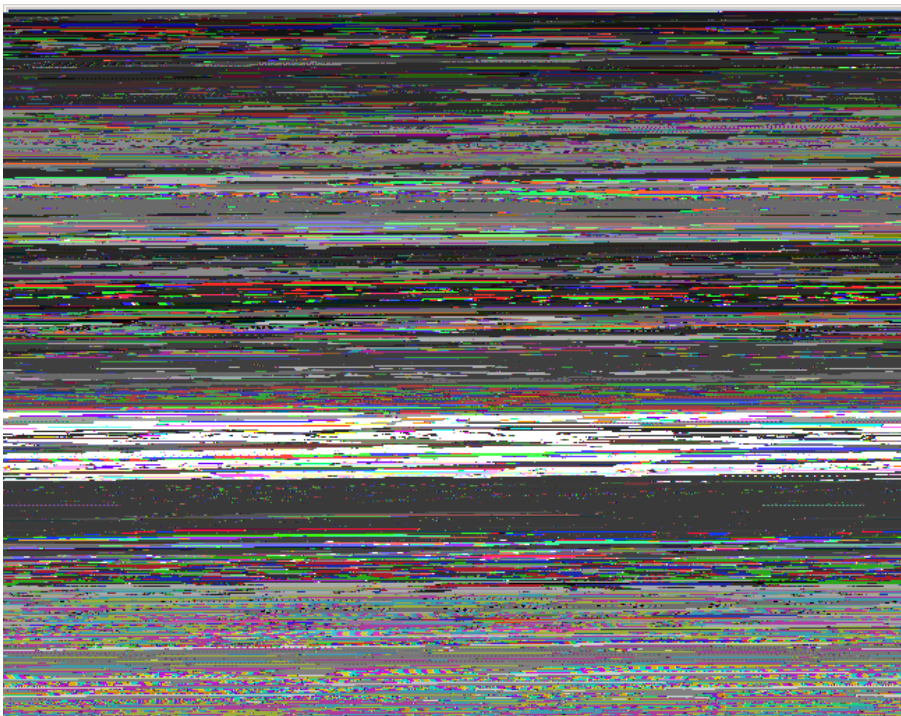
Rendering in 3D Immediate Mode requires following this sequence:

1. Transform the primitives into camera space
2. Perform any rendering with the primitives
3. Terminate pipeline execution

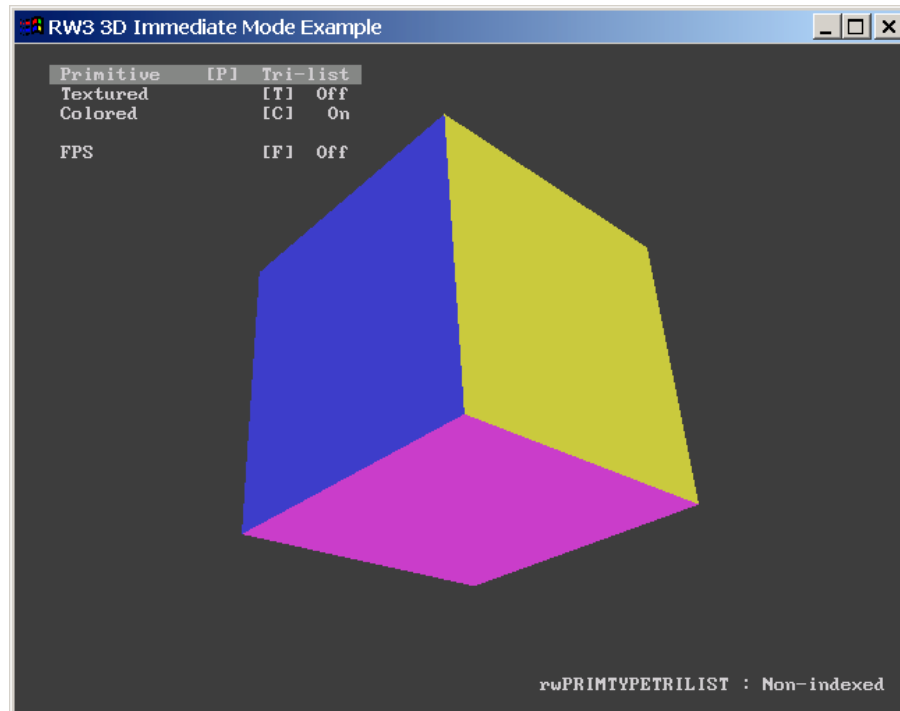
The primitives supported by 3D Immediate Mode are simply extensions of those found in the 2D mode: line lists, polylines, triangle lists, triangle strips and triangle fans. Again, the same trade-offs apply.

The screenshots below illustrate some of the primitives in action. All are from the im3d Example.

This first shot is the polyline form in action. (The code used to display this is contained within the `polyline.c` source file.)



The next shot shows the triangle list primitive. (The code that exercises this primitive can be found in `trilist.c` .)



We recommend you take a look through and modify the code that makes up this Example as it illustrates the salient features of this particular API.

Render States

3D Immediate Mode rendering makes heavy use of render state settings. More on these in 7.4 Render States. As with the 2D Immediate Mode API, the most efficient way to handle render states is to group primitives with the same state and render them in batches, switching render states between each batch.

Rendering primitives

Two rendering functions are available for this: `Rwlm3DRenderPrimitive()`, which renders non-indexed primitive types, and its counterpart `Rwlm3DRenderIndexedPrimitive()`, which renders indexed ones.

For instance, the following line...

```
Rwlm3DRenderIndexedPrimitive(rwPRIMTYPETRILIST, myIndices, 8);
```

...would be used to render an indexed triangle list. The relevant array indices are stored in the `myIndices` array, and there are 8 such indices to render.

Remember that the `Rwlm3DTransform()` function has already taken this array and transformed it into camera-space vertices. The PowerPipe Transform node caches these internally. It is these converted vertices that are referenced during any rendering.

If your target platform allows it, it may be possible to access these converted vertex arrays using RenderWare Graphics' platform-specific API. The exact functionality available will likely vary from platform to platform.

7.3.3 Closing the pipeline

Once rendering of your array of primitives is complete, you must call `Rwlm3DEnd()` to close and flush the rendering pipeline.

7.3.4 3D Immediate Mode & PowerPipe

3D Immediate Mode rendering makes use of RenderWare Graphics' PowerPipe rendering technology. When you transform an array of vertices with `Rwlm3DTransform()`, you are actually firing up PowerPipe and calling its default pipeline.

Many of our platforms are provided only with generic transform and render nodes. These nodes do not support all of the primitives. For instance, the generic transform node will expand triangle fans into ordinary triangle lists.

The transform node also does not support unindexed primitives and will create indices for them, so you should use indexed rendering wherever possible.

However, all PowerPipe nodes can be overridden by your own, so two functions are provided to give you direct access to the Transform and Rendering nodes. They are, respectively, `Rwlm3DGetTransformPipeline()` and `Rwlm3DGetRenderPipeline()`.

Should you wish to revert to the generic nodes, you can obtain pointers to the generic transform and render nodes using `Rwlm3DGetGenericTransformPipeline()` and `Rwlm3DgetGenericRenderPipeline()`.

On platforms that support Transform & Lighting in hardware, overloading the generic node will cause a software pipeline to be used. This will have a negative impact on performance so you may need to write your own hardware T&L node.

Once the pipeline is fired up, it transforms and renders the supplied vertices.

It's important to understand that the only way to transform some more vertices at this stage is to close down the pipeline using `Rwlm3DEnd()` and restart it with `Rwlm3DTransform()`. This is not the obstacle it appears, as you will usually render batches of primitives sorted by Render State anyway. It is not possible to nest Transform/End sequences.

As you can see, efficient management of this process is the key to getting maximum performance out of RenderWare Graphics' 3D Immediate Mode.

Uses for 3D Immediate Mode

RenderWare Graphics provides a very rich Retained Mode API, which would lead many to suspect 3D Immediate Mode of being of relatively little use. Nothing could be further from the truth!

3D Immediate Mode is ideally suited to the rendering of program-generated data. For example, particle system explosions and other scenery effects are often added to a scene using this API.

Other techniques include producing pseudo-3D imagery, such as that seen in some real-time strategy games. (Although produced long before RenderWare Graphics was released, Cavedog's "Total Annihilation" series is a good example of pseudo-3D.)

An important use is for rendering of scientific data or mathematical models that don't map easily to traditional modeling paradigms. One of our developers has taken advantage of this aspect of 3D Immediate Mode to create a groovy multicolor lava lamp simulator using a marching cube algorithm.

7.3.5 Platform Specific Information

Some platforms do their vertex transforms in hardware and cannot provide access to the transformed vertices. You should refer to the platform-specific documentation to see what functionality is available in this regard.

Similarly, texture coordinates may have restrictions imposed by their underlying hardware. Again, you should refer to the platform-specific documentation to see if this applies to your target platform.

7.3.6 Camera-space and Z-buffer depth equations

The mapping functions that convert from camera space to screen depth space are provided here for information:

Perspective Camera:

$$Z(z) = Z_{\min} + \frac{Z_{\max} - Z_{\min}}{R_{\max} - R_{\min}} \cdot \left(\frac{R_{\max}}{z} - \frac{R_{\min}}{R_{\max}} \right)$$

Parallel Camera:

$$Z(z) = Z_{\min} + \frac{Z_{\max} - Z_{\min}}{R_{\max} - R_{\min}} \cdot (z - R_{\min})$$

Where:

$Z(z)$ is the Z-buffer value corresponding to a camera space z-coordinate z .

$Z_{\max}$, $Z_{\min}$ are the maximum and minimum Z-buffer values, respectively.

They correspond to the Z-buffer values given by $Z(R_{\max})$ and $Z(R_{\min})$. Their values may be queried using the API functions

`Rwlm2DGetFarScreenZ()` and `Rwlm2DGetNearScreenZ()`.

$R_{\min}$, $R_{\max}$ are the distances to the far and near clip planes, respectively.

Their values may be queried using the API functions

`RwCameraGetFarClipPlane()` and `RwCameraGetNearClipPlane()`.

7.3.7 Rt2D

This Toolkit provides a 2D graphics API. Its power is such that it could justifiably be considered a kind of Retained Mode API for 2D graphics.

Unlike 2D Immediate Mode, which deals solely with the most basic rendering chores, the 2D Toolkit also supports features such as resolution-independence, scaleable fonts, paths, textured polygons, anti-aliasing, alpha blending effects, free rotation and scaling, clipping, rendering to any arbitrary Camera object, and more.

Its power is derived from RenderWare Graphics' 3D graphics capabilities. This means it can make full use of any 3D graphics acceleration hardware to produce great 2D imagery.

So why use 2D Immediate Mode?

2D Immediate Mode is a thin API. This makes it much better suited to fast, simple 2D operations of the type often required to create a user interface. Icons, debugging messages, quick bitmap font routines, side-panels, simple sprites and so forth are generally easier to produce using the 2D Immediate Mode API.

7.4 Render States

Briefly, the Render State object is a basic state machine. It is an opaque object, with two API functions provided to access its properties: `RwRenderStateSet()` and `RwRenderStateGet()` .

7.4.1 Key Features

Render States are a shared hardware resource.

There is only one Render State structure and it is shared throughout RenderWare Graphics. Therefore, 2D Immediate Mode shares this with both the 2D and 3D Retained Mode APIs – `Rt2D` and `RpWorld` , respectively.

There is no multi-platform default setting for the Render State structure.

This is because some platforms may not support all the Render State settings, or support them in different ways. Instead, the default setting for the Render State varies from platform to platform.

The application is responsible for ensuring any required state(s) are set correctly.

Some high-level portions of the RenderWare Graphics API, such as `RpWorld` , may change the Render State settings if they need to. You should not assume that a particular Render State has been left untouched by, say, a call to `RpWorldRender()` .

Some Render State settings are unsupported on certain platforms.

RenderWare Graphics does not provide any software emulation as such, so if a platform does not support a particular Render State feature, setting it will have no effect.

7.4.2 API

The Render State API (`RwRenderState`) differs from most other APIs in that there is only one Set/Get pair of functions. They are `RwRenderStateSet()` and `RwRenderStateGet()` respectively.

Both functions take two parameters: an enumerated type and a void pointer. The enum or 'render state specifier' determines which render state the function is to set or get. The void pointer holds the data. So the void pointer is used as an arbitrary 32-bit value whose contents depend on the value of the render state specifier. The programmer is responsible for checking the type of this parameter as the compiler will not check the type of void pointers. (Types cast to void in these function calls include `RwRaster *` , `RwBool` , and other enumerated types.)

Currently, the following render states are supported:

`rwRENDERSTATETEXTURERASTER`

Use this render state to set the Raster object to texture with.

For example:

```
RwRaster * myRaster;
RwRasterCreate( myRaster );
RwRenderStateSet( rwRENDERSTATETEXTURERASTER,
                  (void *)myRaster);
```

`rwRENDERSTATETEXTUREADDRESS`

Use this render state to tell RenderWare Graphics how to handle the texture coordinate space. The options are: `rwTEXTUREADDRESSWRAP`, `rwTEXTUREADDRESSCLAMP`, `rwTEXTUREADDRESSMIRROR`, or `rwTEXTUREADDRESSBORDER`

The values to pass as the void pointer are those described in the API Reference under `RwTextureGetAddressing()`.

This render state will set both U and V handling to the same operation. The following two enumerated values can be used to set the U and V spaces individually so you can have different behavior for each:

- `rwRENDERSTATETEXTUREADDRESSU`
- `rwRENDERSTATETEXTUREADDRESSV`

Example 1: Set both U and V coordinate spaces to Clamp mode:

```
RwRenderStateSet( rwRENDERSTATETEXTUREADDRESS, (void *)
rwTEXTUREADDRESSCLAMP);
```

Example 2: Set V coordinate space to Wrap mode:

```
RwRenderStateSet( rwRENDERSTATETEXTUREADDRESSV, (void *)
rwTEXTUREADDRESSWRAP);
```

`rwRENDERSTATETEXTUREPERSPECTIVE`

Pass `TRUE` or `FALSE` in the void pointer. `TRUE` enables perspective correction.

`rwRENDERSTATEZTESTENABLE`

Pass `TRUE` or `FALSE` in the void pointer. `TRUE` enables Z-buffer tests.

`rwRENDERSTATESHADEMODE`

Can be set to either `rwSHADEMODEFLAT` or `rwSHADEMODEGOURAUD`

`rwRENDERSTATEZWRITEENABLE`

Pass `TRUE` or `FALSE` in the void pointer. `TRUE` enables Z-buffer writes.

rwRENDERSTATETEXTUREFILTER

Texture filtering method. The values you can pass are as follows:

- **rwFILTERNEAREST**,
- **rwFILTERLINEAR**,
- **rwFILTERMIPNEAREST**,
- **rwFILTERMIPLINEAR**,
- **rwFILTERLINEARMIPNEAREST**,
- **rwFILTERLINEARMIPLINEAR**

See the Rasters, Images and Textures chapter for more detailed information on these.

rwRENDERSTATESRCBLEND

Used to set the source blending factor.

rwRENDERSTATEDESTBLEND

Used to set the destination blending factor.

rwRENDERSTATEVERTEXALPHAENABLE

Pass **TRUE** or **FALSE** in the void pointer. **TRUE** enables vertex alpha transparency.

rwRENDERSTATEBORDERCOLOR

Border color for texturing with borders. The argument should be RGBA components packed into an **RwRGBA** value and cast to a void pointer.

rwRENDERSTATEFOGENABLE

Pass **TRUE** or **FALSE** in the void pointer. **TRUE** enables fogging effects. (All polygons rendered from this point on until this is set to **FALSE** will be affected.)

rwRENDERSTATEFOGCOLOR

Color used for fogging. The argument should be RGBA components packed into an **RwRGBA** value cast to a void pointer.

rwRENDERSTATEFOGTYPE

Select the type of fogging to use. Valid arguments are:

- **rwFOGTYPELINEAR**,
- **rwFOGTYPEEXPONENTIAL**,
- **rwFOGTYPEEXPONENTIAL2**

The precise algorithms used vary from platform to platform, but are broadly similar. Read the documentation supplied with your target platform for more details on how fogging is implemented.

On current platforms, linear fog will be faster than exponential forms.

Not all platforms support all three fogging formats.

rwRENDERSTATEFOGDENSITY

Defines the density of exp and exp2 fogging. The density should be an RwReal value cast to a void pointer. See the platform-specific documentation for the range(s) allowed.

rwRENDERSTATEFOGTABLE

This render state accepts a pointer to a 256-byte fog table (a RwUInt8[256] cast to a void pointer) placed between fog distance and far clip plane. This is one of the faster forms of fog algorithm and gives better control over the density than the standard rwFOGTYPELINEAR form. However Fog Tables are not supported on all platforms.

In all cases, the RwRenderStateGet() and RwRenderStateSet() functions will return FALSE if a particular render state is unsupported.

7.4.3 Blending

The Render State structure includes two blending settings, rwRENDERSTATESRCBLEND and rwRENDERSTATEDESTBLEND. These functions are used to control the blending of the rendered imagery with the data already extant in the target Raster.

The core blend formula is as follows:

$$(R_s S_r + R_d D_r, G_s S_g + G_d D_g, B_s S_b + B_d D_b, A_s S_a + A_d D_a)$$

Where:

(S_r , S_g , S_b , S_a) represents the source RGBA blend factor;

(D_r , D_g , D_b , D_a) represents the destination RGBA blend factor;

The RGBA values of the source and destination are indicated by a subscript of s or d respectively.

The following blend functions are available:

rwBLENDZERO

Blend factor = (0, 0, 0, 0)

rwBLENDONE

Blend factor = (1, 1, 1, 1)

rwBLENDSRCCOLOR

Blend factor = (R_s , G_s , B_s , A_s)

rwBLENDINVSRCOLOR

Blend factor = (1, 1, 1, 1) - (R_s , G_s , B_s , A_s)

rwBLENDDESTCOLOR

Blend factor = (R_d, G_d, B_d, A_d)

rwBLENDINVDESTCOLOR

Blend factor = (1, 1, 1, 1) - (R_d, G_d, B_d, A_d)

rwBLENDSRCALPHA

Blend factor = (A_s, A_s, A_s, A_s)

rwBLENDINVSRCALPHA

Blend factor = (1, 1, 1, 1) - (A_s, A_s, A_s, A_s)

rwBLENDDESTALPHA

Blend factor = (A_d, A_d, A_d, A_d)

rwBLENDINVDESTALPHA

Blend factor = (1, 1, 1, 1) - (A_d, A_d, A_d, A_d)

rwBLENDSRCALPHASAT

Blend factor = (f, f, f, 1), where f = min(A_s, 1 - A_d)

7.4.4 Sorting Alpha Primitives

Current versions of RenderWare Graphics do not support sorting of alpha primitives for you. It is therefore necessary to perform such processing explicitly in your application. This issue is explored in one of our FAQs.

The `rwRENDERSTATEALPHAPRIMITIVEBUFFER` render state is not implemented on any platform.

7.4.5

Chapter 8

Serialization

8.1 Introduction

RenderWare Graphics supports both a low-level file I/O API and a high-level streaming API which is used for serialization of objects.

This chapter covers the low-level file I/O API, which is similar in design to most ANSI-based operating system APIs. However, the main focus is on the streaming API.

8.2 File I/O API

The low-level file I/O API deals with file-level access to storage devices. It exposes a file I/O interface based on the standard ANSI interface included with most C compilers.

The interface is exposed through the `RwFileFunctions` structure, a pointer to which can be obtained through a call to `RwOsGetFileInterface()`. This structure contains pointers to the file I/O functions and a default set is installed by default by RenderWare Graphics upon initialization.

The default functions implement the functionality either by using the platform's C compiler's default implementation (if available), or by passing the file I/O requests to a platform's underlying operating system.

The following table lists the `RwFileFunctions` element names, its associated ANSI file I/O function, and its purpose. Due to the popularity of the ANSI file I/O interface, it is assumed developers are familiar with these functions and their usage.

STRUCTURE ENTRY	ANSI NAME	PURPOSE
<code>rwfexist</code>	<code>fexist()</code>	Returns TRUE if the file exists
<code>rwfopen</code>	<code>fopen ()</code>	Opens a file for reading/writing
<code>rwfclose</code>	<code>fclose ()</code>	Closes an opened file
<code>rwfread</code>	<code>fread ()</code>	Reads data from a file
<code>rwfwrite</code>	<code>fwrite ()</code>	Writes data to a file
<code>rwfgets()</code>	<code>fgets ()</code>	Reads a null-terminated string from a file
<code>rwfputs()</code>	<code>fputs ()</code>	Writes a null-terminated string to a file
<code>rwfeof()</code>	<code>feof ()</code>	Returns TRUE if the end of the file has been passed
<code>rwfseek()</code>	<code>fseek ()</code>	Moves the file pointer to the specified point in the file
<code>rwfflush()</code>	<code>fflush ()</code>	Flushes any data written to a file from the internal cache
<code>rwftell()</code>	<code>ftell ()</code>	Returns the current position of the file pointer in a file

The application can replace the default functions by overwriting the entries in the `RwFileFunctions` structure with pointers to custom functions.

The replacements must use the relevant function prototype. For this purpose, suitable prototype typedefs have been defined. These typedefs are highlighted in bold within the function prototypes given in the table below.

ANSI NAME	PROTOTYPE
fexist()	RwBool (* rwFnFexist)(const RwChar *name);
fopen()	void *(* rwFnFopen)(const RwChar *name, const RwChar *access);
fclose()	int (* rwFnFclose)(void *fptr);
fread()	size_t (* rwFnFread)(void *addr, size_t size, size_t count, void *fptr);
fwrite()	size_t (* rwFnFwrite)(const void *addr, size_t size, size_t count, void *fptr);
fgets()	RwChar *(* rwFnFgets)(RwChar *buffer, int maxLen, void *fptr);
fputs()	int (* rwFnFputs)(const RwChar *buffer, void *fptr);
feof()	int (* rwFnFeof)(void *fptr);
fseek()	int (* rwFnFseek)(void *fptr, long offset, int origin);
fflush()	int (* rwFnFflush)(void *fptr);
ftell()	int (* rwFnFtell)(void *fptr);

8.3 RenderWare Binary Streams

RenderWare Graphics supports the concept of binary streams .

A binary stream is a data format which allows data to be streamed to files or through a network connection. Only the former is supported explicitly.

The binary stream mechanism is also used to support serialization of objects and this API relies on the low-level file I/O API described earlier, in section 1.2.

8.3.1 Binary Stream Structure

The binary stream format is used when serializing all serializable objects. For example, static worlds (RpWorld) can be serialized to a binary stream and so can dynamic models (RpClump).

Binary streams are normally written to, or read from, files. Such files can contain one or more serializable objects, so a system of chunk IDs —also known as object type IDs —is used to identify the specific entities within the stream.

For example, the RenderWare Graphics SDK comes with a number of demonstrations and examples. Most of these load model geometry and other related data from individual files with an extension of either ".dff " for dynamic models, or ".bsp " for static models. Both of these file extensions are used purely to remind the programmer what is in each file. Both file types are fundamentally the same: both are binary streams and accessed using the same binary stream API.

Although the examples provided with the SDK use individual files for each dynamic model or world, an application could keep all its data in a single binary stream if needed.

The Header

When an object is serialized, the data is preceded by a chunk header , represented by the RwChunkHeaderInfo structure:

length – length of the chunk data in bytes

type – chunk ID, which can either be one of the following or a custom application-created ID:

rwID_ATOMIC - an atomic (type RpAtomic).

rwID_CAMERA- a camera (type RwCamera).

rwID_CLUMP - a clump (type RpClump).

rwID_GEOMETRY- a geometry (type RpGeometry).

`rwID_IMAGE` - an image (type `RwImage`).

`rwID_LIGHT` - a light (type `RpLight`).

`rwID_MATERIAL` - a material (type `RpMaterial`).

`rwID_MATRIX` - a matrix (type `RwMatrix`).

`rwID_TEXDICTIONARY` - a texture dictionary (type `RwTexDictionary`).

`rwID_TEXTURE` - a texture (type `RwTexture`).

`rwID_WORLD`- a world (type `RpWorld`).

`version` - version of the chunk data

Chunk Types

The key to the streaming mechanism is in the `type` element. This element denotes the type of object stored in the chunk referred to by the chunk header. When a plugin is attached, it registers its supported object type IDs with the streaming API.

The type ID should be created using the `MAKECHUNKID` macro. This macro is also used to create plugin IDs, but plugins rarely implement more than one serializable object, so the plugin ID can also be used as the object type ID.

If a custom plugin does implement more than one object, you should ensure that the additional object IDs are unique. If you need more than 256 unique IDs, you should contact Criterion Software Ltd. to obtain additional Vendor IDs, each of which can be used to create a further 256 unique IDs.

Complex Chunks

Chunks can contain other chunks, so a complete stream can be parsed as a tree-like hierarchy. Such chunks are known as complex chunks .

The `strview` applet, which can be located in `[SDK-ROOT]/tool/strview` , illustrates this aspect of RenderWare Graphics' binary streams by displaying the structure of any valid stream as a hierarchical view. An example of the stream viewer in action can be seen in the screenshot on the next page.

Complex chunks are handled transparently by callbacks which are registered with the streaming API. In the screenshot below, the structure of a typical `RpClump` container object is clearly visible, with its list of `RwFrame` objects, `RpGeometry` objects and `RpAtomic` objects.

The stream viewer applet

8.3.2 Serializing Objects

Although there is a trend towards originating 3D graphics data algorithmically, most 3D graphics data is streamed into memory from a storage device. Given RenderWare Graphics' target market, this is usually either a CD, DVD or hard disk.

The streaming of extended or derived object data requires the creation of additional functions. We saw these in the ClumpPhysics example (see the [Plugins: Creation & Usage](#) chapter), but didn't see them in action.

Look for `TestPluginStream()` in the ClumpPhysics example's `main.c` file. This function, which is predicated on a build-time `#define` called `TESTSTREAM`, was included in this source code so you can see how the RenderWare Graphics Binary Stream facility is used.

You may want to activate it, by defining the `TESTSTREAM` constant, and step through the code in your debugger to watch what it does.

Registering the Stream Functions

If you look at the source code, you'll notice that none of the three stream functions created for the new plugin appear to be called directly. This is because these functions are called directly by the clump object's own stream functions.

Objects that support extension via the plugin mechanism and support for binary streams have two registration functions. Respectively, these are:

`Rp[PLUGIN-NAME]RegisterPlugin()`

`Rp[PLUGIN-NAME]RegisterPluginStream()`

In the `ClumpPhysics` plugin example, both functions are called in the `RpClumpPhysicsPluginAttach()` function. The second of the two registration functions, `RpClumpRegisterPluginStream()`, is the one that tells the clump object about our own stream functions.

The `RpClumpRegisterPluginStream()` function accepts three function pointers. Respectively, these are for reading, writing and returning the size of the additional data our functions will place into the stream.

It is important to note that `RpClumpRegisterPluginStream()` is part of the `RpClump` plugin, not the `ClumpPhysics` plugin. All plugins that need to support extension of their binary streams must implement their own registration function to allow this. For example, the `ClumpPhysics` plugin would need to implement an `RpClumpPhysicsRegisterPluginStream()` function.

Determining the Size of a Chunk

A stream chunk is not assumed to be fixed in size.

One reason for this is that plugins may extend existing chunk types rather than utilizing their own. Another issue is that some platforms work better if data is padded to a certain minimum block size. (This is often the case on consoles which make heavy use of caches and other hardware tricks.)

Every chunk type needs to have its own chunk size function. This function will call the chunk size functions for any extensions within that chunk so, if another plugin extends the same chunk, it too must implement a chunk size function.

This gives `RenderWare Graphics` a means by which it can determine the full size of a chunk: adding up the results of all the chunk size functions implemented for a chunk type gives the total size of the complex chunk.

Reading and Writing

Writing platform independent data to a stream is not as straightforward as it sounds. `RenderWare Graphics`' multi-platform nature means we have to take into account such things as packing sizes, default datatype sizes and even the endianness of the data. Otherwise, data written out on one platform may not be readable on another.

Endian-ness

Depending on the processor used at the heart of your target platform, data can be stored in one of two formats: big-endian or little-endian. This defines the byte-order of datatypes larger than a single byte when these are stored in RAM.

The table below uses hexadecimal notation to show the difference between the two forms when storing the hexadecimal 4-byte number "0x12345678":

LITTLE-ENDIAN	BIG-ENDIAN
0x12 0x34 0x56 0x78	0x78 0x56 0x34 0x12

As you can see, all the endian-ness means is which order bytes are stored. Little-endian format places the most significant byte ("MSB") first, while big-endian format places the least significant byte ("LSB") first.

This means you cannot simply throw your data structures at the stream as there's no guarantee that the raw bytes will be read back correctly on another platform. The solution is to standardize on some basic datatypes and make sure the endian-ness of the streamed data is consistent across all platforms.

In RenderWare Graphics, we've chosen little-endian format as our standard. This was picked mainly because we use PCs to develop on and these use little-endian format. This standard is used for all streams, regardless of platform, so streams written on a big-endian platform will be converted to little-endian form. On platforms that use little-endian format, no endian-ness conversion takes place.

The table below shows which of our key platforms uses which storage format:

PLATFORM	CPU	STORAGE FORMAT
Sony PlayStation 2	MIPS-based core	Little-endian
Microsoft Xbox	Intel Pentium III	Little-endian
PC (Direct3D / OpenGL)	Various x86-compatible CPUs	Little-endian
MacOS (OpenGL)	Motorola / IBM PowerPC	Big-endian
NINTENDO GAMECUBE	Motorola / IBM PowerPC	Big-endian

Endian-ness Conversion

The endian-ness of individual platforms makes it impossible to create data streams that are cross-platform compatible without some conversion work. The `RwMem` part of the core library API provides such conversion functions.

Some of the streaming functions—see the next section—perform these conversions internally. However, you must use the conversion functions described here if your application uses only the basic `RwStreamRead()` / `RwStreamWrite()` functions.

An important point to note is that none of the `RwMem` conversion functions will change the physical size of the data being converted, so a "Float32" occupies the same number of bytes as an `RwReal` value. This makes calculating the size of the data to be read or written easier.

The functions are:

RwMemFloat32ToReal()

Used when reading from the stream: this function reverses the conversion process performed by `RwMemRealToFloat32()` and gives you back your original array of `RwReal` values. The `RwMemNative32()` function should be called first.

RwMemLittleEndian32()**RwMemLittleEndian16()**

Used when writing to the stream: this function converts the integer data you've pointed it at and converts the data to little-endian format if necessary. (On platforms that natively use little-endian format, this function has no effect.)

RwMemNative32()**RwMemNative16()**

Used when reading from the stream: this function converts the little-endian data read in from the stream into the native endian-ness of the platform. (As with its counterpart, this function has no effect if the platform uses little-endian format.)

RwMemRealToFloat32()

Used when writing to the stream: this function converts an array of `RwReals` to a standard 32-bit floating point format for streaming. The appropriate `RwMemLittleEndian32()` function should be called after this.

Reading & Writing to a Stream

Once the data has been converted, the `RwStream` API is used to write the individual data elements to the stream. When reading data, the process is reversed.

Endian-ness Conversion

Endian-ness conversion must be performed explicitly if using the `RwStreamRead()` / `RwStreamWrite()` functions. However, when using the streaming functions that know about their type—such as `RwStreamReadInt32()` or `RwStreamWriteReal()`—you do not need to perform explicit endian-ness conversion: these functions perform the conversion internally. It is usual for developers to avoid using `RwStreamRead()` / `RwStreamWrite()` wherever possible.

The following two tables list the read and write functions respectively.

Stream Reading Functions

These functions read data from a binary stream. Those functions with an explicit datatype as part of their name will perform any endian-ness conversions internally, if necessary.

READ FUNCTION	PURPOSE
RwStreamRead()	Reads the specified number of bytes from the specified stream into the given data buffer. This function requires explicit endian-ness conversion.
RwStreamReadChunkHeaderInfo()	Reads the Chunk Header information and stores it into the RwChunkHeaderInfo structure provided. (Intended mainly for diagnostics.)
RwStreamReadInt16()	Reads an array of 16-bit RwInt16 values from the stream into the specified buffer.
RwStreamReadInt32()	Reads an array of 32-bit RwInt32 values from the stream into the specified buffer.
RwStreamReadReal()	Used to read an array of RwReal data values from the stream into the specified buffer.
RwStreamSkip()	Used to skip the specified number of bytes in the given stream.

Stream Writing Functions

These functions write data to a binary stream. Those functions with an explicit datatype as part of their name will perform any endian-ness conversions internally, if necessary.

WRITE FUNCTION	PURPOSE
<code>RwStreamWrite()</code>	Writes the specified number of bytes to the specified stream from the given data buffer. This function requires explicit endian-ness conversion.
<code>RwStreamWriteChunkHeader()</code>	Used to write a chunk header to the specified stream. This function would typically be used by an application wishing to write application specific data to a file.
<code>RwStreamWriteInt16()</code>	Writes an array of 16-bit <code>RwInt16</code> values to the stream from the specified buffer.
<code>RwStreamWriteInt32()</code>	Writes an array of 32-bit <code>RwInt32</code> values to the stream from the specified buffer.
<code>RwStreamWriteReal()</code>	Used to write an array of <code>RwReal</code> values to the stream from the specified buffer.

8.3.3 Explicit Streaming Functions

In many cases, RenderWare Graphics' supplied plugins implement and expose streaming functions to serialize their objects explicitly. The `RpClumpStreamRead()` function is an example of such a function.

When developing custom plugins and objects, it is worth considering exposing similar, high-level functions. This makes it easier for developers using the plugin to stream the plugin's data without having to explicitly parse its individual components—a task which becomes a chore if the plugin is a complex one.

Another, very important, advantage of implementing high-level functions in this way is so that the function can perform any required pre- or post-processing transparently. This avoids the need for developers to remember to perform such processes explicitly.

In most cases, you will have already implemented the necessary low-level functionality needed to implement high-level functions, so adding them to your plugin's API is usually trivial.

Using Explicit Streaming Functions

In the previous section, 8.3.2 Serializing Objects , we looked at serialization from within a plugin. This is important to know, but what if your application needs to read or write an object to a stream directly?

Although it is not entirely necessary for it to do so, it is good practice for your plugin to implement high-level stream-read and stream-write functions. The application to which the plugin is attached should be responsible for managing the streams.

This is best illustrated with an example.

Reading a Clump Object

The clump (RpClump) object, part of the World plugin (RpWorld), is a complex object, containing additional objects such as frames (RwFrame) and atomics (RpAtomic).

All this complexity is handled by callback functions set up by the plugin itself when it is attached to your application. This makes reading a clump object very simple.

First, we need to declare our stream and clump objects:

```
RwStream *stream;  
RpClump *newClump;
```

Next, we open our binary stream:

```
stream = RwStreamOpen(rwSTREAMFILENAME, rwSTREAMREAD,  
                    "myclump.dff");
```

(Remember, the .dff filename extension is purely a reminder that this file contains a dynamic model—a clump object.)

Now we can check that the stream was opened and locate our clump object within the stream:

```
if( stream )  
{  
    if( RwStreamFindChunk(stream, rwID_CLUMP, NULL, NULL) )
```

The `RwStreamFindChunk()` function call above locates the first occurrence of a chunk with a chunk ID of `rwID_CLUMP`. By convention, a .dff file usually contains only one clump, so there is no need for further identification.

With the correct chunk located, the application can now read the clump data.

```
{
    newClump = RpClumpStreamRead(stream);
}
```

Finally, we should close the stream as we no longer need to read anything else from it.

```
RwStreamClose(stream, NULL);
}
```

Writing a Clump Object

This process is rare within a game, but applications written as part of a tool-chain, such as an exporter for a modeling package, will need to perform this operation frequently.

Writing an object is very similar to reading one. Again, we'll use a clump object for our example, which will assume that myClump contains a valid clump object for writing.

First, we need to declare our stream object:

```
RwStream *stream;
```

Next, we need to open a stream for writing. This uses the same function as for reading, but with a different flag. (These flags are covered in section 8.3.4, Stream Types .)

```
stream = RwStreamOpen(rwSTREAMFILENAME, rwSTREAMWRITE,
"myclump.dff");
```

At this point we should check that the stream was opened:

```
if( stream )
{
```

Now the data can be written to the stream:

```
    RpClumpStreamWrite( myClump, stream );
}
```

Finally, the stream should be closed:

```
RwStreamClose(stream, NULL);
```

Designing an Explicit Streaming API

It is not a requirement that a plugin exposes a high-level streaming API as there are a number of different ways in which a plugin can be implemented.

Two examples of plugins that may not need such an API are:

Plugins that extend existing objects – if a plugin extends an existing object, there is usually no need for a high-level API. This is because streaming the original object will automatically cause RenderWare Graphics' streaming API to call the appropriate callbacks for the extended data.

8.3.4 Stream Types

RenderWare binary streams are not limited to file-based streams.

The `RwStreamOpen()` function therefore has two flag parameters. The first defines the type of stream to use and the possible flags are listed in the table below:

FLAG	PURPOSE
<code>rwSTREAMFILE</code>	The stream is to a disk file that has been set up by the user. The access mode should match that used when the file was opened. The type-specific argument should be set to the file's pointer (usually of type <code>FILE *</code>).
<code>rwSTREAMFILENAME</code>	The stream is to a disk file that has not been set up by the user. The type-specific argument should be set to the desired filename.
<code>rwSTREAMMEMORY</code>	The stream is to a chunk of memory. If access type is <code>rwSTREAMAPPEND</code> then the chunk of memory must have been created using <code>RwMalloc</code> as RenderWare may subsequently try to use <code>RwRealloc</code> to gain more memory. The type specific argument should be a pointer to the memory chunk (<code>RwMemory *</code>) giving the position and size of the chunk to use.

The second flag parameter determines how the stream is opened:

FLAG	PURPOSE
<code>rwSTREAMREAD</code>	Opens a stream for reading only.
<code>rwSTREAMWRITE</code>	Opens a stream for writing only. If the stream is of type <code>rwSTREAMFILE</code> or <code>rwSTREAMFILENAME</code> the file size will be reduced to zero when the stream is opened.
<code>rwSTREAMAPPEND</code>	Opens a stream to which data will be appended.

8.4 Summary

RenderWare Graphics supports both a low-level file I/O API and a high-level streaming API.

8.4.1 File I/O API

Low-level access to files on disk is accessed through the `RwFileFunctions` structure. This structure, which can be overwritten with custom functions, exposes an ANSI-compliant set of file I/O functions:

8.4.2 RenderWare Binary Streams

Binary streams can contain simple objects, or complex ones. Complex objects can contain other objects—including other complex objects—so a stream's structure can be viewed as a tree.

The Stream Viewer application, `strview.exe`, can be used to examine binary streams. It can be found in the `[SDK-ROOT]/tool/strview/` folder.

Stream Types

a function that returns the size of the chunk.

The developer can choose to implement an object as either:

a container object, (e.g. RpClump and RpWorld)

an extension to another object, (e.g. RpMatFX)

a stand-alone object (e.g. RpLight).

Container objects should expose a high-level serialization API, as should stand-alone objects. Objects that extend an existing object rarely need to expose a high-level serialization API, unless the object can also be used as a stand-alone object.

In all cases, callback functions for serialization are necessary if the object is to support serialization at all.

Registering the Stream Functions

Plugin objects that support both the RenderWare Graphics binary stream API and extension via the plugin mechanism must call two plugin registration functions.

Chapter 9

Debugging and Error Handling

9.1 RenderWare Graphics Errors

RenderWare Graphics functions return error codes using the `RwError` API. This is a simple, minimal impact, general-purpose error reporting mechanism that has minimal impact on application performance even if heavily used.

The API has two functions, `RwErrorSet()` and `RwErrorGet()`, which are used to set or retrieve error codes respectively.

By convention, most RenderWare Graphics functions will return:

- a pointer to an object if successful;

- NULL to represent failure.

Should an API function fail, developers can check what the error code of the last function was using `RwErrorGet()`.

Error codes take the form of an integer matching an enumerated value, such as `E_RW_INVRASTERSIZE`("Invalid Raster Size").

Error code enumerations can be found in the appropriate [SDK-ROOT]/rwsdk/include/ folder corresponding to the build target. The files have one of two extensions:

- `.def` – for common error codes and those used by the core library;

- `.rpe` – for error codes used by specific plugins and toolkits.

9.2 RenderWare Graphics Builds

RenderWare Graphics libraries are supplied in three different builds: Release, Metrics and Debug.

Release Build , as its name implies, is for release builds of your applications. It has no debugging information or unnecessary hooks; it's a lean, mean rendering machine.

Metrics Build is a modified version of the Release Build, exposing hooks in the rendering engine that can be queried with `RwEngineGetMetrics()` . The metrics structure accumulates values since the last call to either `RwCameraShowRaster()` or `RwRasterShowRaster()` , and is most useful when examined immediately before calls to these functions.

The format of the data structure returned by this function will vary by platform – on Sony's PlayStation 2, for instance, it will cover aspects such as DMA and Vector Unit utilization.

Finally, we come to the Debug Build . This set of libraries has been built with debug message logging enabled, so that a log file (with a file extension of .log) is created while running. This log file lists assert messages, errors, bad arguments and similar reports generated by the RenderWare Graphics API.

The log file is generated through the `RwDebug` object, which we'll look at shortly.

9.3 The Debug Object

When using a Debug Build of RenderWare Graphics, the `RwDebug` object becomes active. In the other library builds, the object does nothing.

This object maintains a simple debugging information reporting system, with output being handled by a Debug Stream Handler function. The default function simply diverts the messages to a file called `"rwdebug.log"`, or, in the case of some console platforms, it sends the data across a network or other target/host link.

This handler can be replaced by your own function with the `RwDebugSetStreamHandler()` function. This function controls output for all RenderWare Graphics messages, including those emitted by `assert` and `trace` (covered in 9.4) messages. These messages are all disabled in release and metrics builds.

An important, but often missed, point to understand is that a Debug Build of RenderWare Graphics is designed to behave exactly as if it were a Release Build. The only difference is the activation of the `RwDebugAPI` and the logging of any asserts and errors raised while running.

This is an intentional design feature. Our view is that if an application is going to crash in Release Build, it should also crash in Debug Build. This may sound extreme, but it has the advantage of dramatically reducing unpleasant surprises when you're ready to build a release. In particular, it avoids the problem of bugs that cannot be repeated in a debug build.

It is also important to note that overuse of the `RwDebug` functionality is very likely to affect performance, particularly if the output is being streamed to a file. For example, we do not advise placing long, descriptive messages in the stream for functions likely to be frequently called within the inner loop of a rendering cycle.

9.3.1 The Default Debug Stream Handler

As we saw earlier, the default Debug Stream Handler outputs to a log file named `"rwdebug.log"`. This file is either stored in the current working directory, or, on platforms where this isn't possible, is transmitted to the host (development) computer.

If you have replaced the default debug stream handler with your own functionality, then passing `NULL` to `RwDebugSetStreamHandler()` will re-instate it.

9.3.2 Sending a Message to the Debug Stream

The Debug Stream is not limited to RenderWare Graphics' own functions: you can send your own messages to the stream using `RwDebugSendMessage()`. These messages will be chronologically ordered with any other messages, including those resulting from asserts and the trace message mechanism.

The function takes the following parameters:

Type – Signifies whether this message is an Assert, an Error message, an informational Message, or a simple Trace ("where am I?") message. These are defined as `rwDEBUGASSERT`, `rwDEBUGERROR`, `rwDEBUGMESSAGE`, and `rwDEBUGTRACE` respectively.

Filename – The filename from where the message was sent. Usually, this is the source file containing the function sending the message and can be obtained using the ANSI C compiler directive `__FILE__`.

Line – The line number where this particular call to `RwDebugSendMessage()` is taking place. Again, this is usually obtained using the ANSI C compiler directive `__LINE__`.

Function Name – The name of the function from whence the message was sent.

Message – The message itself.

These are the arguments passed to `RwDebugSendMessage()`. Most of this information is combined before passing to the Debug Stream Handler itself. The stream handler only receives two parameters: the Type and a Message. The Message is a single string, usually containing the information outlined above.

9.4 Tracing RenderWare Graphics Activity

RenderWare Graphics' functions can be made to output simple Trace messages using the `RwDebugSendMessage()` mechanism.

Trace messages are simple, informational "I Am Here" messages which can be used during debugging to track the flow of execution of your application. They are specified by passing `rwDEBUGTRACE` as the `Type` parameter to `RwDebugSendMessage()`.

To enable these Trace messages, use:

```
RwDebugSetTraceState(TRUE);
```

Passing `FALSE` will disable them.

All of RenderWare Graphics public API will output Trace messages when the Trace State is `TRUE`. (This can impact performance, so the default Trace State is `FALSE`.) Messages are output both on entry to and on exit from each function.

The text below is an extract from an example Debug log with Trace enabled:

```
...
D:/rel/rwsdk/world/baclump.c(1740): TRACE: RpClumpForAllAtoms: Enter
d:/rel/rwsdk/plugin/morph/rpmorph.c(1151): TRACE: RpMorphAtomicAddTime: Enter
d:/rel/rwsdk/plugin/morph/rpmorph.c(1203): TRACE: RpMorphAtomicAddTime: Exit
D:/rel/rwsdk/world/baclump.c(1763): TRACE: RpClumpForAllAtoms: Exit
D:/rel/rwsdk/src/bacamera.c(1580): TRACE: RwCameraClear: Enter
D:/rel/rwsdk/src/bacamera.c(1592): TRACE: RwCameraClear: Exit
D:/rel/rwsdk/src/bacamera.c(957): TRACE: RwCameraBeginUpdate: Enter
D:/rel/rwsdk/src/bacamera.c(961): TRACE: RwCameraBeginUpdate: Exit
D:/rel/rwsdk/world/baworobj.c(331): TRACE: WorldCameraBeginUpdate: Enter
D:/rel/rwsdk/world/baworobj.c(342): TRACE: WorldCameraBeginUpdate: Exit
D:/rel/rwsdk/src/bacamera.c(776): TRACE: CameraBeginUpdate: Enter
D:/rel/rwsdk/src/bacamera.c(810): TRACE: CameraBeginUpdate: Exit
D:/rel/rwsdk/world/baworld.c(1874): TRACE: RpWorldRender: Enter
D:/rel/rwsdk/world/baworld.c(1885): TRACE: RpWorldRender: Exit
D:/rel/rwsdk/src/bacamera.c(1118): TRACE: RwCameraGetRaster: Enter
D:/rel/rwsdk/src/bacamera.c(1123): TRACE: RwCameraGetRaster: Exit
D:/rel/rwsdk/src/baraster.c(347): TRACE: RwRasterGetWidth: Enter
D:/rel/rwsdk/src/baraster.c(351): TRACE: RwRasterGetWidth: Exit
D:/rel/rwsdk/src/bacamera.c(1118): TRACE: RwCameraGetRaster: Enter
D:/rel/rwsdk/src/bacamera.c(1123): TRACE: RwCameraGetRaster: Exit
D:/rel/rwsdk/src/baraster.c(371): TRACE: RwRasterGetHeight: Enter
```

```
D:/rel/rwsdk/src/baraster.c(375): TRACE: RwRasterGetHeight: Exit  
d:/rel/rwsdk/tool/charse/rtcharse.c(943): TRACE: RtCharsetGetDesc:  
Enter  
d:/rel/rwsdk/tool/charse/rtcharse.c(953): TRACE: RtCharsetGetDesc:  
Exit  
...
```


9.5 Replacing The Stream Handler

We saw earlier that the Debug Stream Handler can be replaced by your own code. This is achieved through a standard callback mechanism.

The callback function prototype is defined by a typedef named `RwDebugHandler`. Your callback function's prototype must match this.

Secondly, either your main program or your handler will need to open or create any files it intends to use. Disabling the default handler will close the "rwdebug.log" file it uses automatically, so your new handler cannot make the assumption that it is still open for writing.

To replace the default RenderWare Graphics handler with your own, you should call `RwDebugSetHandler()`, passing a pointer to your replacement callback function as the parameter.

9.5.1 Example

The following code replaces the existing handler with a dummy handler that outputs the supplied text only if it is an `rwDEBUGTRACE` type.

(This example assumes your program's global data is stored in a structure called `GlobalData` and that this structure has an entry named `debugFile` representing a `FILE *` type.)

```
static void
MyDebugHandler(RwDebugType type, const RwChar * debugText)
{
    /* First check if output file is available... */
    if ( !GlobalData.debugFile )
    {
        /*
         * output file not initialized, better create it...
         */

        GlobalData.debugFile = Rwfopen(RWSTRING("bug_out.txt"),
RWSTRING("at"));
    }

    /* Output message if it's the right type... */

    if ( (GlobalData.debugFile) && (rwDEBUGTRACE == type) )
    {
        Rwfwrite(debugText, (rwstrlen(debugText) * sizeof (RwChar)), 1,
RWSRCGLOBAL(debugFile));
        Rwfwrite(cr, (rwstrlen(cr) * sizeof (RwChar)), 1,
```

```
        RWSRCGLOBAL(debugFile));  
        RwFlush(RWSRCGLOBAL(debugFile));  
  
    }  
  
    return;  
}
```

This example handler is for illustration purposes only and has little error checking of its own. Naturally you will want to remedy this in production code.

Setting the above handler in your RenderWare Graphics application would be achieved by adding the following line to your code after the Engine has been initialized:

```
RwDebugSetHandler(MyDebugHandler);
```

We can reset the default debug handler by passing NULL to the above function, therefore there is no need to store the returned function pointer unless you are using multiple handlers.

Part B

World Library

Chapter 10

World and Static Models

10.1 Introduction

This chapter explains scenes and static models, which are represented by `RpWorld` and `RpWorldSector`. These objects are part of the `World` plugin (`RpWorld`) which provides RenderWare Graphics' Retained Mode API.

This Plugin provides the following objects:

`RpAtomic`; `RpClump`; `RpGeometry`; `RpInterpolator`; `RpLight`; `RpMaterial`; `RpMorphTarget`; `RpWorld` and `RpWorldSector`. It also adds some extensions to the Core Library's `RwCameraAPI`.

These objects can be further extended through the plugin mechanism.

10.2 Scenes & Static Models

10.2.1 Scenes

A typical scene will consist of:

- Static models – the ‘scenery’

- Dynamic models – objects that can be moved or animated

- Dynamic lights

- Cameras – one or more depending on the number of views required

This chapter is primarily concerned with static models. Separate chapters are devoted to dynamic models (Atomics), Lights and Cameras.

10.2.2 RpWorld Object

The `RpWorld` object is a container for scenes. It links the dynamic and static components and provides a cohesive world space in which they can exist.

A World is bounded by a single box, which is internally subdivided into static sectors (`RpWorldSector` objects). Each of these defines a cuboid volume within the scene, and encloses a section of the static scenery.

The sectors are created by dividing up the static scenery using a binary space partition (BSP) tree and this sectorization process is usually performed at the exporter stage using the functions supplied by the `RtWorldImport` Toolkit. This structure is used to speed up the rendering process.

Dynamic objects such as Atomics, Lights and Cameras can be added to a World and this allows links to be maintained between the objects and any World Sectors to which they are relevant. When dynamic objects are moved around, the World plugin will decide which World Sector(s) they should be linked to, based on their location. This makes the rendering process more efficient since:

- Only World Sectors that are visible from the Camera need be rendered.

- Dynamic objects that lie entirely within non-visible sectors can be eliminated.

- Only the dynamic lights affecting visible sectors need be considered in lighting calculations.

10.2.3 RpWorldSector Object

A World Sector is the static counterpart to the Geometry object, which is used for dynamic models.

A World Sector performs two tasks:

1. Contains all Vertices, Triangles, Materials and other data that define the scenery within the World Sector's space.
2. Maintains links with Atomics, Cameras and Lights that have been positioned within the World Sector.

The data for the static scenery is usually divided up across multiple World Sectors to increase the rendering engine's efficiency. This division is in the form of a binary space partitioning ('BSP') tree.

Binary Space Partitioning

The World is sectorized by recursively slicing the geometry along axis-aligned planes to produce a BSP tree. The resulting box regions are sectors. Geometry cannot always be cleanly divided and sectors often overlap slightly.

RenderWare Graphics uses a modified form of BSP, known as a K Dimensional Tree (KD-tree). The difference between the two forms is that KD-trees are always axis-aligned.

Normals & Bounding Boxes

Normals and bounding boxes are encountered when creating import and conversion tools for World and World Sector objects.

Normals

These are required for lighting calculations. During the world creation process, the normals are stored as `RwV3d` for full floating-point precision, but in the final World Sectors they are compressed to triples of type `RwInt8`.

Bounding Box

World Sectors have bounding boxes (`RwBBox`). Since models can rarely be cut into neat, straight lines, the bounding boxes of neighboring World Sectors often overlap.

10.3 World Offset

A World's origin may be moved during an import or load process, depending on the requirements on the underlying hardware. For instance, a particular platform might only accept positive coordinates. In such cases, the entire World must be offset so that it can be entirely referenced using positive coordinates.

RenderWare Graphics therefore provides the `RpWorldGetOrigin()` function. This returns the offset of the specified World from the origin and is fixed at the time the world is created. It describes the translation required to move the world's bounding box back to its original position.

In current versions of RenderWare Graphics, Worlds are not offset and the above function will therefore always return (0,0,0). However, developers using data created with previous versions of RenderWare Graphics may need to use this function as offsets may have been applied.

Although we do not envisage having to support World Offsets in the near future, it is possible that new handheld and wireless gaming devices may appear that require this functionality.

10.4 Iterator Functions

A number of simple iterator functions are provided to access the contents of both `RpWorld` and `RpWorldSector` objects:

10.4.1 RpWorld Iterators

There are four `RpWorld` iterators:

1. `RpWorldForAllClumps()`

This function takes a callback (type `RpClumpCallBack()`), which is called for each `Clump` within the specified `RpWorld` object. This function will only locate `Clumps` that have been added to the World using `RpWorldAddClump()` .

2. `RpWorldForAllLights()`

This function takes a callback (type `RpLightCallBack()`), which is called for each `Light` within the specified `RpWorld` object. This function will only locate `Lights` that have been added to the World using `RpWorldAddLight()` .

3. `RpWorldForAllMaterials()`

This function takes a callback (type `RpMaterialCallBack()`), which is called for each `Material` object within the specified `RpWorld` object. This function iterates only through `Materials` contained within `World Sectors`. `Materials` define how a model's surface should be rendered and are covered in more detail in the `Dynamic Models` chapter.

4. `RpWorldForAllWorldSectors()`

This function takes a callback (type `RpWorldSectorCallBack()`), which is called for each `World Sector` within the specified `RpWorld` object.

This function is one of the most often-used iterators and it is often used together with `RpWorldSectorForAllAtoms()` .

10.4.2 RpWorldSector Iterators

There are three RpWorldSector iterators:

1. RpWorldSectorForAllAtomsics()

This function takes a callback (type RpAtomicCallBack()), which is called for each Atomic within the specified RpWorldSector object. This function will only locate Atomics that have been added to the World either explicitly, using RpWorldAddAtomic() , or implicitly using RpWorldAddClump() . It is often used in conjunction with RpWorldForAllWorldSectors() to access all Atomics with in a World.

2. RpWorldSectorForAllLights()

This function takes a callback (type RpLightCallBack()), which is called for each Light within the specified RpWorldSector object. This function will only locate Lights that have been added to the World using RpWorldAddLight() .

3. RpWorldSectorForAllMeshes()

This function takes a callback (type RpMeshCallBack()), which is called for each Mesh within the specified RpWorldSector object. This function will only locate Meshes that are part of the static model data.

10.4.3 Collision Detection

Originally, the RpWorld API contained some basic collision detection functionality. This has now been moved into a separate RenderWare Graphics Plugin: RpCollision .

10.5 Modeling Tools

The RenderWare Graphics modeling package exporters supplied with the SDK can be used to export static models. Artists should read the associated documentation installed with these exporters for details on how they work.

Developers are also advised to read these documents, as exporters for particular modelers may not necessarily support all features.

Exported files can be read by applications.

10.5.1 Viewers

The static models exported by artists can be tested using the RenderWare Visualizer viewer supplied with the RenderWare Graphics SDK. Using the RenderWare Visualizer is covered in the RenderWare Visualizer Viewers document.

The screenshot below shows this viewer displaying a static model of a building's interior.



This viewer can display most serialized RenderWare Graphics objects including those supported directly by the World Plugin.



User Guide

Volume I

SDK Release 3.3